# INDEX


| Sr.No. | Title of Topic | Page No. |
|---|---|---|
| 1). | Data structure introduction. | 3 |
| 2). | classification of data structure | 7 |
| 3). | introduction to algorithm | 9 |
| 4). | asymptotic analysis | 15 |
| 5). | DS- pointer | 18 |
| 6). | DS- structure | 20 |
| 7). | DS- Array | 23 |
| 8). | DS- linked list | 30 |
| 9). | DS- skip list | 36. |
| 10). | DS- stack | 41 |
| 11). | DS- Queue | 44 |
| 12) | DS- Tree | 48. |
| 13). | Types of Tree | 58 |
| 14). | DS- Graph | 62 |


CodeNeurax

| Sr. No | Topic Name | Page No. |
|---|---|---|
| 15). | Graph Traversal Algorithms | 65 |
| 16). | Searching | 71 |
| 17). | Searching Algorithms with example | 82 |
| 18). | Sorting Algorithms | 86. |
| 19). | Implementations of sorting Algorithms | 89. |
| 20). | Data Structure Interview questions with Answers (50 questions with Answers) | 95. |
| 21). | DATA STRUCTURE CODING QUE. | 104. |
| 22). | END | |

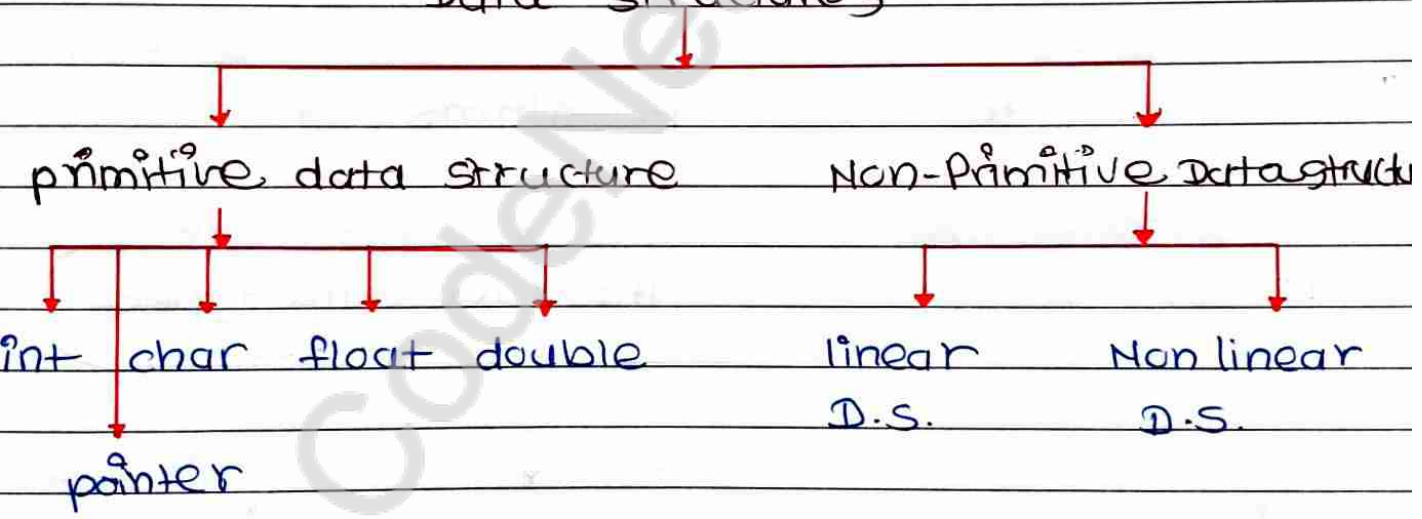

CodeNeuray

## What is Data Structure ?

→ Data structure is a way to store and organize data so that it can be used efficiently.

As per name indicates itself that organizing the data in memory.

The data structure is not any programming language like c, c++, Java etc. It is set of algorithms that we can use in any programming language to structure data in memory.




### Linear Data structure :-

The arrangement of data in the sequential manner is known as linear data structure. The data structure used for this purpose are Arrays, linked list, Stacks and queues.

In this data structures, one element is connected to only one another element in a

linear form.

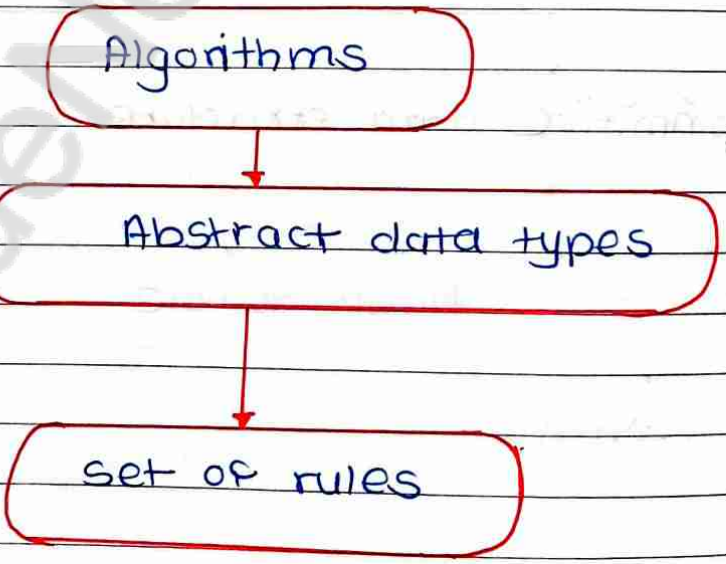
Non-linear data structure :-

When one element is connected to the 'n' number of elements known as non-linear data structures.

Example :- trees and graphs.

In this case, elements are arranged in a random manner.

## Algorithms and Abstract Data types ??




Why →

To structure the data in memory, 'n' number of algorithms are proposed, and all these algorithms are known as Abstract Data Types.

Codeleurax

An Abstract Data Type tells what is to be done and data structure tells how is to be done ?

ADT gives us the blueprint while data structure provides the implementation part.

What is Data ?

Data can be defined as the elementary value / collection of values.
forexample :- student's name and its id are the data about student.

What is Record ?

Record can be defined as collection of various data items
example :- student entity → name, address, course and marks can be grouped together to form record.

What is File ?

File is a collection of various records of one type of entity
example :- if there are 60 employees in class, then there will be 20 records in related file where record contains info of employee

What is Attribute and Entity ?

An entity represents class of certain objects. it contains various attributes. each attribute represents particular property of that entity.

codeNeural

## What is need of data structures?

As applications are getting complexed and amount of data is increasing day by day, there may arise following problems :-

**Processor speed :-** As data is growing day by day to the billions of files per entity, processor may fail to deal with that amount of data.

**Data Structure :-** consider an inventory size of 106 items in store, if our application needs to search for a particular item, it needs to transverse 106 items every time, results in slowing down process.

**multiple requests :-** If thousands of users are searching data simultaneously on a web server, then there are chances that to be failed to search during that process.

To solve this problems, data structures are used. Data is organized to form a data structure in a such way that all items are not required to be searched and require data can be searched instantly.

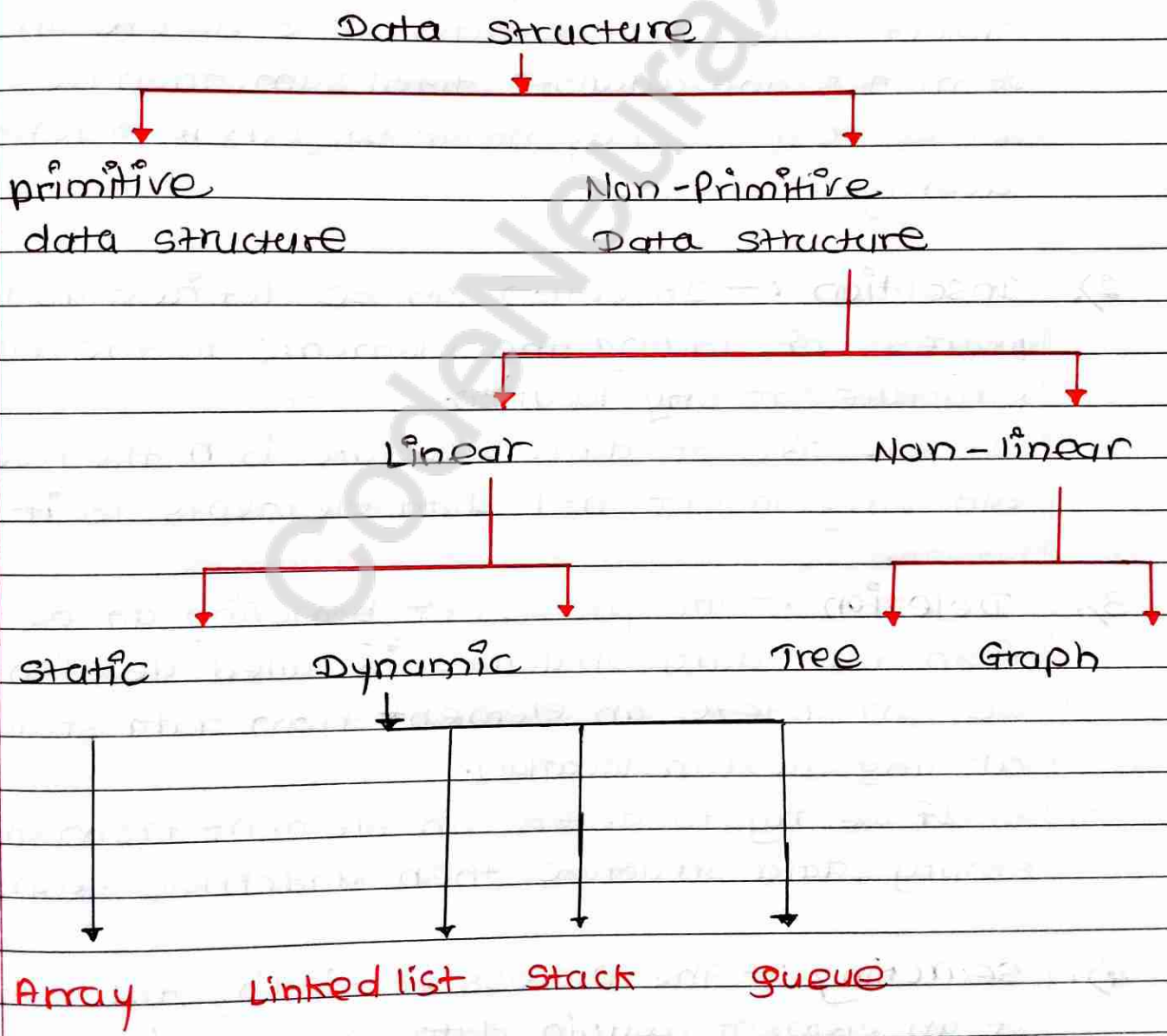
## Advantages of data Structure :-

**Efficiency :-** If the choice of a data structure for implementing a particular ADT is proper, it makes program very efficient in terms of time and space.

**Reusability :-** The data structure provides reusability means that multiple client programs can use the data structure.

Abstraction :- The data structure specified by the ADT also provides level of abstraction. The client cannot see internal working of data structure, so it does not have to worry about implementation.

* Data structure classification :-




CodeNeuraX

## Operations on data structure :-

1). Traversing :- Every data structure contains a set of data elements. Traversing data structure means visiting each element of data structure in order to perform some specific operation like searching or sorting.

Example :- If we need to calculate average of marks obtained by a student in 6 different subject, we need to traverse complete array of marks and calculate total sum, then we will devide that sum by no. of subjects ie. 6 to find average.

2). Insertion :- Insertion can be defined as the process of adding the elements to the data structure at any location.

If the size of data structure is n then we can only insert n-1 data elements to it.

3). Deletion :- The process of removing an element from the data structure is called deletion. We can delete an element from data structure at any random location.

If we try to delete an element from an empty data structure then underflow occurs.

4). Searching :- The process of finding the location of an element within data structure is called searching. There are two algorithms to perform

searching, linear search and Binary search.

5). Sorting :- The process of arranging the data structure in a specific order is called as sorting. There are many algorithms that can be used to perform sorting, for example, insertion sort, selection sort, bubble sort etc.

6). merging :- When two lists list A and list B of size M and N respectively, of similar type of elements, clubbed or joined to produce third list, list C of size (M+N), then this process is called merging.

# DATA STRUCTURES AND ALGORITHM

## What is Algorithm ?

An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer.

It is not complete program or code ; it is just a solution (logic) of a problem, which can be represented either as an informal description using a flowchart or pseudocode.

characteristics of an algorithm.

Input :- An algorithm has some input values. We can pass 0 or some input value to an algorithm.

Output :- We will get 1 | more output at end of an algorithm.

Unambiguity :- An algorithm should be unambiguous which means that instruction in an algorithm should be clear and simple.

Finiteness :- An algorithm should have finiteness. means limited number of instructions.

Effectiveness :- An algorithm should have finite as each instruction in an algorithm affects the overall process.

## Approches in Algorithm :-

1). Brute force Algorithm :- The general logic structure is applied to design an algorithm. It is also known as exhaustive search algorithm that searches all possible to provide required solution.

such algorithms have two types :-

1). optimizing
finding all solutions of a problem and then take out the best solution is known then it will terminate if the best solution is known.

2). sacrificing
As soon as the best solution is found, then it will stop.

Code Nagrat

classmate
Date
Page 11

Divide and conquer :- This breaks down the algorithm to solve the problem in different methods. It allows you to break down problem into different methods, and valid output is produced for the valid input. This valid output is passed to some other function.

Greedy algorithm :- It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting best solution. It is easy to implement and has faster execution time. But there are very rare cases in which it provides the optimal solution.

The major categories of algorithms are given below:
Sort :- Algorithm developed for sorting the items in a certain order.

search :- Algorithm developed for searching the items inside a data structure.

Delete :- Algorithm developed for deleting the existing element from the data structure.

Insert :- Algorithm developed for inserting an item inside a data structure.

Update :- Algorithm developed for updating the existing element inside a data structure.

CodeNeural

## Algorithm Analysis :-

The algorithm can be analyzed in two levels ie first is before creating the algorithm, and second is after creating the algorithm.
There are two analysis of an algorithm.

### Priori Analysis :-

Here, priori analysis is the theoretical analysis of an algorithm which is done before implementing the algorithm.

### Posterion Analysis :-

Here, posterior analysis is a practical analysis of an algorithm. The practical analysis is achieved by implementing algorithm using any programming language.

## Algorithm complexity :-

The performance of the algorithm can be measured in two factors :

### Time complexity :-

The time complexity of an algorithm is the amount of time required to complete the execution. The time complexity of an algorithm is denoted by the big O notation.

Here big O notation is the asymptotic notation to represent time complexity. The time complexity is mainly calculated by counting the number of steps to finish execution.

CodeAvenue

```
sum = 0 ;
// suppose we have to calculate the sum of n
 numbers.
for i=1 to n
sum = sum + i ;
// when the loop ends then sum holds the sum
of n numbers.
return sum;
```

In above code, the time complexity of the loop statement will be atleast n, and if value of n increases, then time complexity also increases.

We generally consider the worst-time complexity as it is maximum time taken for any given input size.

Space complexity :—

An algorithm's space complexity is the amount of space required to solve a problem and produce an output. similar to the time complexity, space complexity is also expressed in big o notation.

Space complexity = Auxiliary space + Input size.

CodeNaurax

The following are the types of algorithms:

Search Algorithm :-

on each day, we search for something in our day to day life.

similarly, with the case of computer, huge data is stored in a computer that whenever user asks for any data then the computer searches for that data in the memory and provides that data to the user. There are mainly two techniques available to search data in an array:

- Linear search
- Binary search

Sorting Algorithms :-

sorting algorithms are used to rearrange elements in an array or a given data structure either in an ascending or descending order. The comparison operator decides the new order of the elements:

CodeNerd

## Asymptotic Analysis :-

The time required by an algorithm comes under three types :

Worst case :- It defines the input for which the algorithm takes a huge time.

Average case :- It takes average time for the program execution.

Best case :- It defines the input for which the algorithm takes the lowest time.

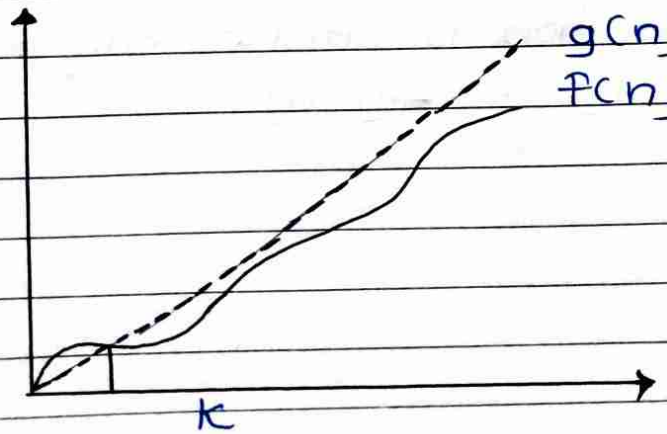
## Asymptotic Notations :-

The commonly used asymptotic notations used for calculating the running time complexity of an algorithm is given below :

1. Big oh notation (O) :-

This measures the performance of an algorithm by simply providing the order of growth of the function.

This notation provides an upper bound on a function which ensures that function never grows faster than the upper bound.



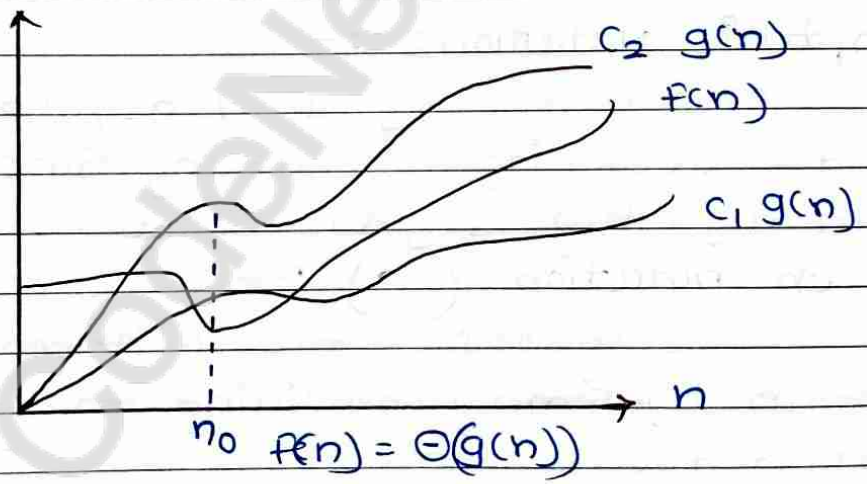

CodeNeuraX

Example :- If f(n) and g(n) are two functions defined for positive integer,
then f(n) = O g(n) as f(n) is big oh of g(n) or f(n) is on order of g(n)) if there exists constants c and no such that:

$f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

2). Omega Notation ($\Omega$) :-

It basically describes best case scenario which is opposite to big o notation. It is the formal way to represent lower bound of an algorithm's running time.




Example :- let f(n) and g(n) be functions of n where n is steps required to execute program:

$f(n) = \Omega\ g(n)$

The above condition is satisfied only if when:

$c1 \cdot g(n) <= f(n) <= c2 \cdot g(n)$

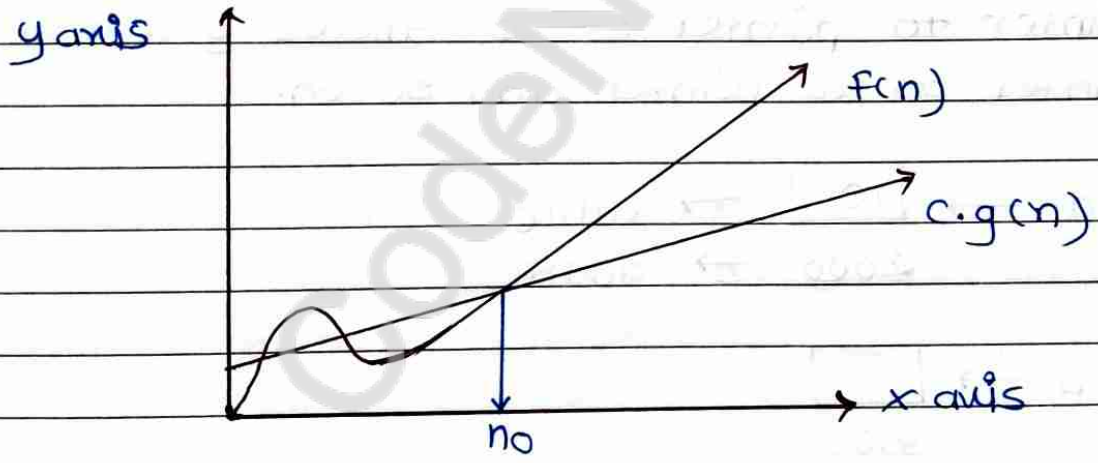
2). omega Notation ($\Omega$)

It basically describes best-case scenario which is opposite to big-O notation It is formal way to represent lower bound to an algorithm's running time. It measures the best amount of time an algorithm can possibly take to complete or best case time complexity.

Example :- If f(n) and g(n) are two functions defined for positive integers,

then $f(n) = \Omega g(n)$ as f(n) is omega of g(n) or f(n) is on the order of g(n) if there exists constants c and no such that:

$f(n) >= c \cdot g(n)$ for all $n \geq n_0$ and $c > 0$




3). Theta Notation ($\Theta$)

The theta notation mainly describes average case scenarios.

It represents realistic time complexity of an algorithm. Big theta is mainly used when the value of worst case and best case is same.

CodeNeural

## Pointer :-

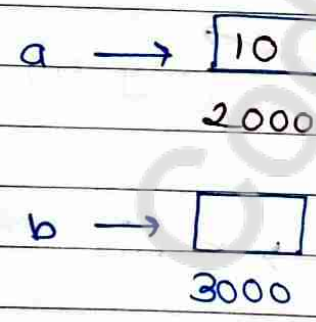
Pointer is used to points the address of the value stored anywhere in the computer memory. To obtain the value stored at location is known as dereferencing pointer.

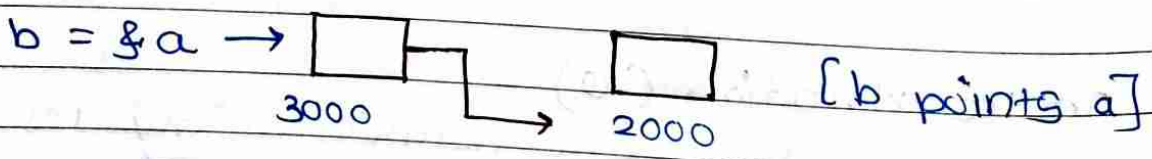
## Pointer arithmatic :-

4 arithmatic operaters that can be used in pointers : ++, --, +, -

**Array of pointers :-** You can define array of to hold a number of pointers.

**Pointer to pointer :-** c allows you to have pointer on a pointer and so on.




Program
Pointer →

```
#include <stdio.h>
int main()
```

```
{
int a = 5;
int *b;
b = &a;
printf ("value of a = %d \n", a);
printf ("value of a = %d \n", *(&a));
printf ("value of a = %d \n", *b);
printf ("address of a = %u \n", &a);
printf ("address of a = %d \n", b);
printf ("address of b = %u \n", &b)
printf ("value of b = address of a = %u ", b);
return 0;
}
```

Output

```
value of a = 5
value of a = 5
address of a = 3010491492
address of a = -1284473004
address of b = 3010491496
value of b = address of a = 3010491492.
```

Program :—

Pointer to pointer :—

```
#include <stdio.h>
int main ()
{
int a = 5;
int *b;
int **c;
```

```
b = &a;
c = &b;
printf ("value of a = %d \n", a);
printf ("value of b = address of a = %u \n", b);
printf ("value of c = address of b = %u \n", c);
printf ("address of b = %u \n", c);
printf ("address of c = %u \n", &c);
return 0;
}
```

output →

```
value of a = 5
value of b = address of a = 2831685116
value of c = address of b = 2831685120
address of b = 2831685120
address of c = 2831685128
```

## Structure :-

A structure is a composite data type that defines a grouped list of variables that are to be placed under one name in block of memory.

Program :-

structure →

```
struct structure_name
{
  data-type member 1;
  data-type member 2;
```

:
:
:

data-type member ;
};

Advantages of structure :-
- It can hold variables of different data types.
- We can create objects containing different types of attributes.
- It allows us to re-use the data layout across programs.
- It is used to implement other data structure like linked list, queues, trees and graphs.

Program :-
how to use structure in program →

```
#include <stdio.h>
#include <conio.h>
Void main ()
{
struct employee
{
int id ;
float salary ;
int mobile ;
};
```

CodeNeuron

```
struct employee e1, e2, e3;
printf ("\n Enter ids, salary & mobile no. \n");
scanf ("%d %f %d", &e1.id, &e1.salary, &e1.mobile);
scanf ("%d %f %d", &e2.id, &e2.salary, &e2.mobile);
printf ("%d %f %d", &e3.id, &e3.salary, &e3.mobile);
printf ("\n Entered result");
printf ("\n %d %f %d", e1.id, e1.salary, e1.mobile);
printf ("\n %d %f %d", e2.id, e2.salary, e2.mobile);
printf ("\n %d %f %d", e3.id, e3.salary, e3.mobile);
getch ();
}
```

output →

guess the output

And write it here....

CodeNeural

**Array :-** Arrays are defined as collection of similar type of data items stored at contigous memory locations.

Array is the simplest data structure where each data element can be randomly accessed by using its index number.

Array declaration :-

```
int arr[10] ; char arr[10] ; float arr[5]
```

Program without Array :-

```
#include <stdio.h>
void main ()
{
 int marks-1 = 56 ; marks-2 = 78, marks-3 = 89 ;
 float avg = (marks-1 + marks-2 + marks-3)/3 ;
 print (avg) ;
}
```

Program by using Array :-

```
#include <stdio.h>
void main
 {
 int marks [3] = { 56, 78, 89} ;
 int i ;
 float avg ;
 for (i=0 ; i<3 ; i++)
```

CodeNeural

```
    {
    avg = avg + marks[i];
    }
  printf(avg);
}
```

Complexity of Array operations :-

1). Time complexity :-

| Algorithm | Average case | worst case |
|---|---|---|
| Access | O(1) | O(1) |
| search | O(n) | O(n) |
| insertion | O(n) | O(n) |
| Deletion | O(n) | O(n) |

2). Space complexity :-

In Array space complexity for worst case is O(n)

Memory Allocation of the Array :-

Each element in Array represented by indexing Indexing of array can be defined in three ways:

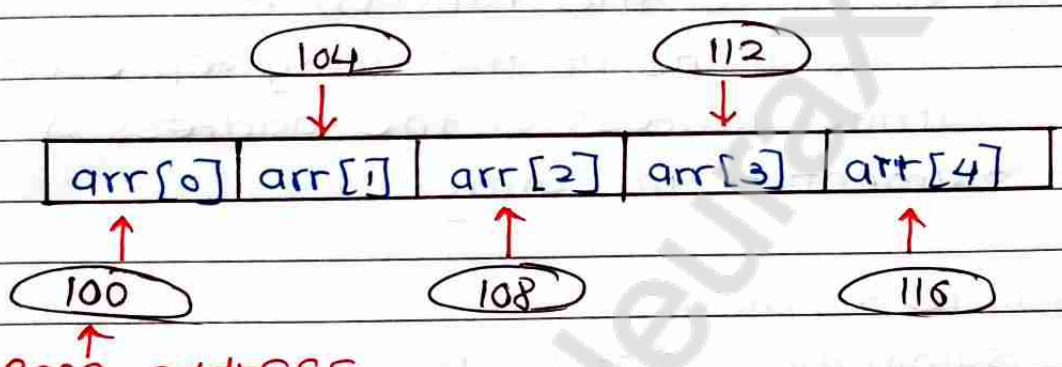
1. 0 (Zero Based indexing) :-

The first element of the array will be arr[0].

CodeNeural

2. 1 (one-based indexing) :-
The first element of array will be arr[1].

3. n ( n-based indexing ) :-
The first element of array can reside at any random index number.




(fig : int arr[5])

Accessing elements of an Array :-
To access any random element of an array we need the following information :
1. Base address of the array
2. size of an element in bytes.
3. which type of indexing, array follows.

Address of any element of 1D array can be calculate

Byte address of element A[i] = base address + size * ( first - index )

Example: In an array, A[-10 .... +2] Base address (BA) = ggg, size of an element = 2 bytes, find location of A[-1].

Solution: $L(A[-1]) = ggg + [(-1) - (-10)] \times 2$

$= ggg + 18$

$= 1017.$

$\therefore$ location of $A[-1] = 1017.$

**Passing array to the function :-**

The name of the array represents the starting address or the address of the first element of the array.

Program:

```
#include <stdio.h>
int summation (int []);
void main ()
{
 int arr [5] = {0,1,2,3,4};
 int sum = summation (arr);
 printf ("%d ", sum);
}
int summation (int arr[])
 {
  int sum =0, i;
  for(i=0 ; i<5 ; i++)
    {
     sum = sum + arr [i];
    }
  return sum ;
 }
```

**2D Array :-** 2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as collection of rows and columns.

How to declare 2D Array :-

The syntax for declaration of two dimensional array is as follows :

```
int arr[max-rows][max-columns];
```

However, it produces the data structure which looks like following :

| | 0 | 1 | 2 | ... | n-1 |
|---|---|---|---|---|---|
| 0 | a[0][0] | a[0][1] | a[0][2] | ..... | a[0][n-1] |
| 1 | a[1][0] | a[1][1] | a[1][2] | .... | a[1][n-1] |
| 2 | a[2][0] | a[2][1] | a[2][2] | ... | a[2][n-1] |
| : | : | : | : | ... | : |
| n-1 | a[n-1][0] | a[n-1][1] | a[n-1][2] | | a[n-1][n-1] |

a[n][n]

(Fig : a[n][n])

How to access data in 2D-array :-

Due to fact that elements of 2D arrays can be random accessed.

CodeNeuraX

```
int x = a[i][j];
```

where i, j are the rows and coloumns respectively.

**Initializing** 2D arrays :-

The syntax to declare and initialize the 2D array is given as follows :

```
int arr[2][2] = {0,1,2,3};
```

number of elements in 2D arrays
= number of rows * number of coloumns.

**Mapping** 2D array to 1 D array :-

The size of a two dimensional array is equal to the multiplication of number of rows and number of coloumns present in the array.

A 3x3 two dimensional array is as shown:-

| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | (0,0) | (0,1) | (0,2) |
| 1 | (1,0) | (1,1) | (1,2) |
| 2 | (2,0) | (2,1) | (2,2) |

coloumn index

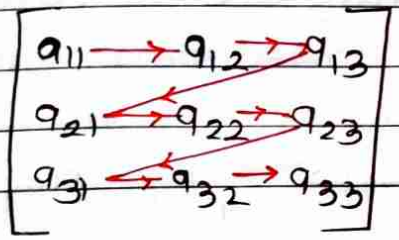
row index

There are two main techniques of storing 2D array elements into memory.

CodeNeural

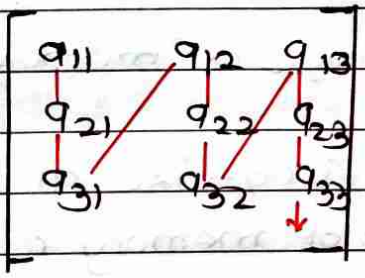
1. Row major ordering :—

In row major ordering, all the rows of 2D array are stored into memory contiguously.




2. Column major ordering :—

According to column major ordering, all the columns of 2D array are stored into the memory contigously.




Calculating address of random element of a 2D array:—

1). By row major order :—

If array is declared a[m][n] where m is the number of rows while n is number of columns. then address of an element a[i][j] is calculated as,

Address (a[i][j]) = B.A + (i * n + j) * size

B.A → Base Address

2). By column major order :—

Address (a[i][i] = (j * m) + i) * size + B.A.

CodeNeuraX

## Linked list :-

✓ Why there is a need of linked list ?

If we declare an array of size 3. As we know that all the values of an array are stored in a continous manner, so all three values of an array are stored in a sequential fashion.

Then, total memory space occupied by array would be $3 * 4 = 12$ bytes.

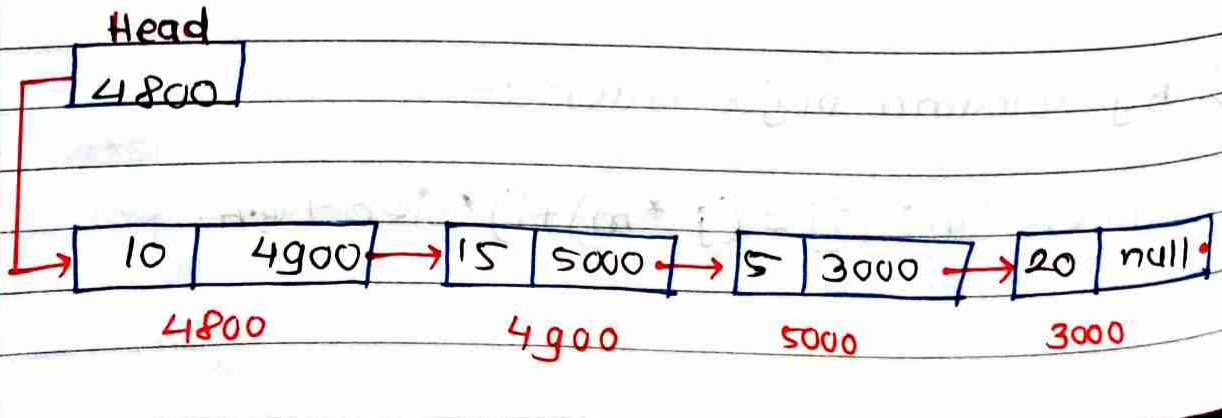
Drawbacks of using array :-

- we cannot insert more than 3 elements in above example because only 3 spaces are allocated by 3 elements.
- In case of array, the wastage of memory can occur.
- In array, we are providing fixed-size at compile time, due to which wastage of memory occurs.

The solution to this problem is to use linked list.

What is Linked list ?

A linked list is also a collection of elements, but the elements are not stored in a consecutive location. or linked list is a collection of the nodes in which one node is connected to another node and node consists of two parts i.e. one is data part and second one is the address part.



## declaration of linked list :-

In linked list, one is variable and second one is pointer variable. we can declare linked list by using user-defined data type called as structure.

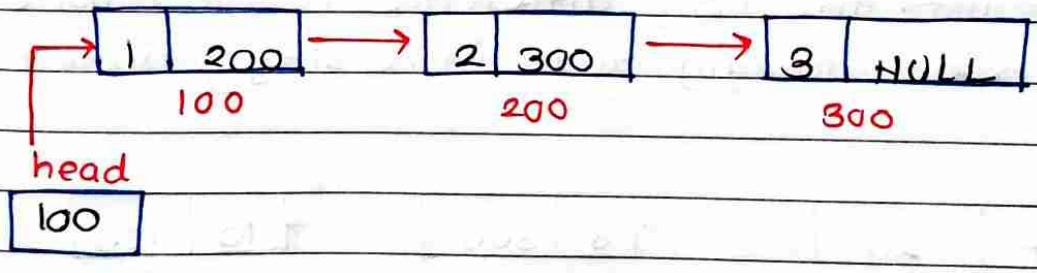
```
struct node
{
  int data;
  struct node *next;
}
```

## Types of linked list :-

1). singly linked list :-

The singly linked list is most common. which consists of data part and address part. The address part in the node is known as a pointer.

Example :- suppose we have three nodes and addresses of these three nodes are 100, 200 and 300 :

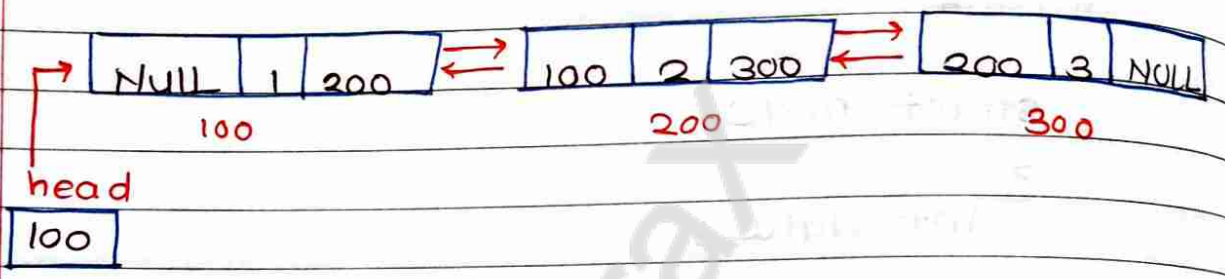



NULL means its address part does not point to any node. The pointer that holds the address of the initial node is known as a head pointer.

CodeNeurax

2. Doubly linked list :-

As name suggests, the doubly linked list contains two pointers. We define it in three parts the data part and the two address part.

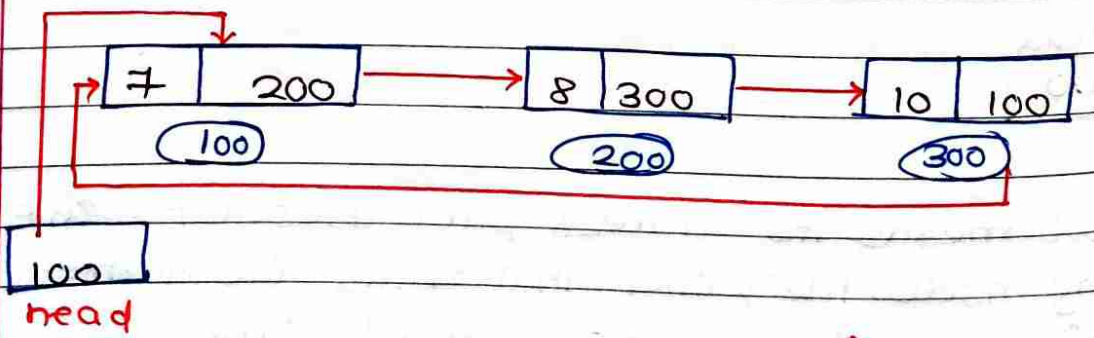



Representation of doubly linked list :-

```
struct node
 {
  int data ;
  struct node * next ;
  struct node * prev;
 }
```

3. Circular linked list :-

A circular linked list is a variation of a singly linked list. The only difference is "last node does not point to any node in a singly linked list".



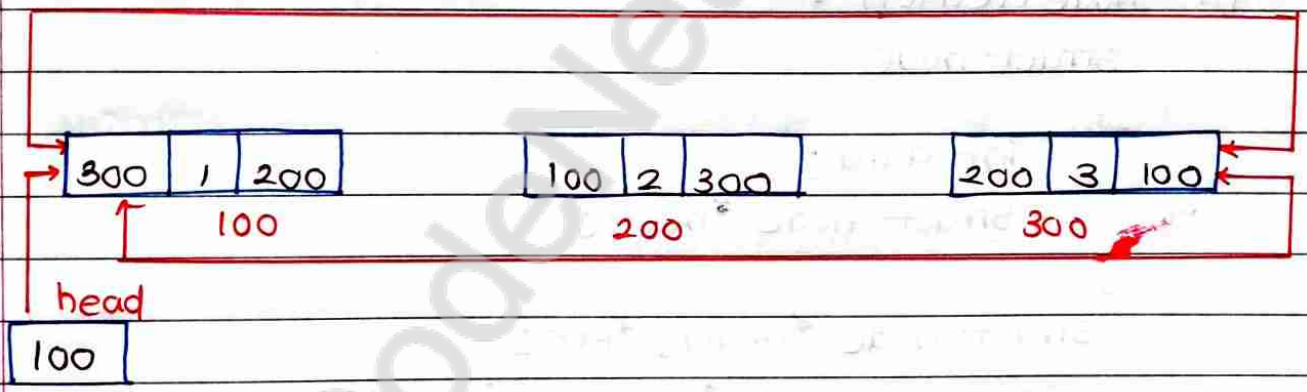

CodeNeuraX

Representation of circular linked list :-

```
struct node
{
  int data;
  struct node *next;
}
```

4. Doubly circular linked list :-

The doubly circular linked list has the features of both the circular linked list and doubly linked list.




The last node is attached to the first node and thus creates a circle.
The main difference is that doubly circular linked list does not contain NULL value in previous field of the node.

Representation of doubly circular linked list :-

```
struct node
{
  int data;
  struct node *next;
  struct node *prev;
}
```

CodeNeuraX

Complexity :-

| | Average | | | | Space complexity |
|---|---|---|---|---|---|
| | Access | search | Insertion | deletion | Worst |
| Singly linked list | O(n) | O(n) | O(1) | O(1) | O(n) |

| | Worst | | | |
|---|---|---|---|---|
| | Access | search | Insertion | deletion |
| singly linked list | O(n) | O(n) | O(1) | O(1) |

Operations on singly linked list :-

1). Node creation :-

```
struct node
{
    int data;
    struct node *next;
};
struct node *head, *ptr;
ptr = (struct node*) malloc (sizeof (struct node*))
```

2). Insertion :-

① Insertion at beginning :- It involves inserting any element at the front of the list. We just need a few link adjustment to make new node as head of list.

② Insertion at end of list :- The new node can be inserted as the only node in the list / it can be inserted as last one.

③ Insertion after specified node :- we need to skip desired number of nodes in order to reach node after which the new node will be inserted.

3). 3) Deletion and Traversing :-

① Deletion at beginning :- It just needs few adjustments in the node pointers

② Deletion at end of list :- The list can either be empty or full. Different logic is implemented for different scenario's.

Traversing :- In traversing, we simply visit each node of the list at least once in order to perform some specific operation in it, for example, printing data part of each node present in the list.

Searching :- In searching, we match each element of the list with the given element. If the element is found on any of the location of that element is returned otherwise null is returned.

operations on doubly linked list :-

1) Node creation :-

```
struct node
{
  struct node *prev;
  int data;
  struct node *next;
};
struct node *head;
```

2). Insertion :-

① Insertion at beginning :- Adding the node into the linked list at beginning.

② Insertion at end :- Adding the node into the linked list to the end.

CodeNeuraX

3). Deletion and Traversing :-

① Deletion at beginning :- Removing the node from beginning of the list.

② Deletion at end :- Removing the node from end of the list.

Traversing :- visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display etc.

Searching :- comparing each node data with the item to be searched and return location of the item in the list if the item found else return null.

## Skip list :-

* What is a skip list ?

A skip list is a probalistic data structure. The skip list is used to store a linked list of elements or data with a linked list. In one single step, it skips several elements of the entire list, which is why it is known as skip list.

### Structure of skip list :-

skip list is built in two layers : The lowest layer and the top layer. The lowest layer of the skip list is a common sorted linked list, and the top layers of the skip list are the like an "express line" where elements are skipped.

CodeNeuraX

complexity table :-

| Sr.No. | complexity | Average case | Worst case |
|---|---|---|---|
| 1). | Access complexity | O(log n) | O(n) |
| 2). | search comple. | O(log n) | O(n) |
| 3). | delete comple. | O(log n) | O(n) |
| 4). | Insert comple. | O(log n) | O(n) |
| 5). | Space comple. | - | O(n log n). |

Basic operations and its algorithms :-

1). Insertion operation :- It is used to add new node to a particular location in a specific situation.

2). Deletion operation :- It is used to delete a node in a specific situation.

3). search operation :- The search operation is used to search a particular node in a skip list.

Algorithm of insertion operation :-

Insertion (L, key)

local update [0 .... max - level + 1]

q = L → header

for i = L → level down to 0 do.

while q → forward [i] → key forward [i]

update [i] = q

```
a = a → forward[0]
lvl = random_level()
if lvl > L → level then
for i = L → level + 1 to lvl do
    update[i] = L → header
    L → level = lvl
a = make node (lvl, key, value)
for i = 0 to level do
    a → forward[i] = update[i] → forward[i]
update[i] → forward[i] = a
```

Algorithm of deletion operation :-

```
Deletion (L, key)
local update[0 .... max level + 1]
a = L → header
for i = L → level down 0 to do
    while a → forward[i] → key forward[i]
        update[i] = a
a = a → forward[0]
if a → key = key then
    for i = 0 to L → level do
    if update[i] → forward[i] ≠ a then break
    update[i] → forward[i] → forward[i]
free(a)
while L → level > 0 and L → header → forward[L → level]
                                    = NIL do
L → level = L → level - 1.
```

Collegelearn

Algorithm of searching operation :-

searching (L, skey)

q = L → header

loop invariant : q → key level down to 0 do.

while q → forward [i] → key < forward [i]

q = q → forward [q]

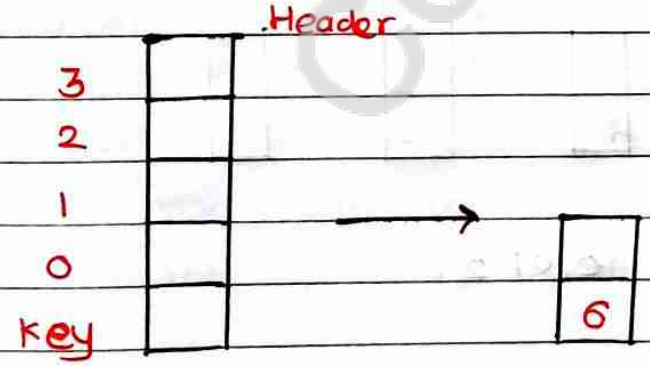
if q → key = skey then return q → value

else return failure.

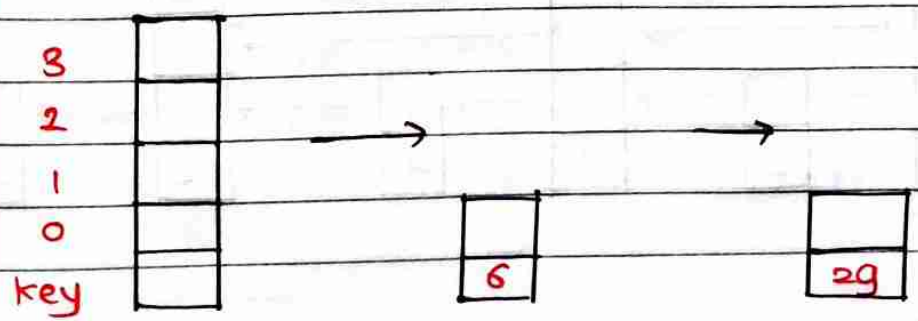
Example : create a skip list, we want to insert these following keys in empty skip list.

1. 6 with level 1
2. 29 with level 1
3. 22 with level 4.
4. 9 with level 3.
5. 17 with level 1.
6. 4 with level 2.

→ solution :- Insert 6 with level 1.

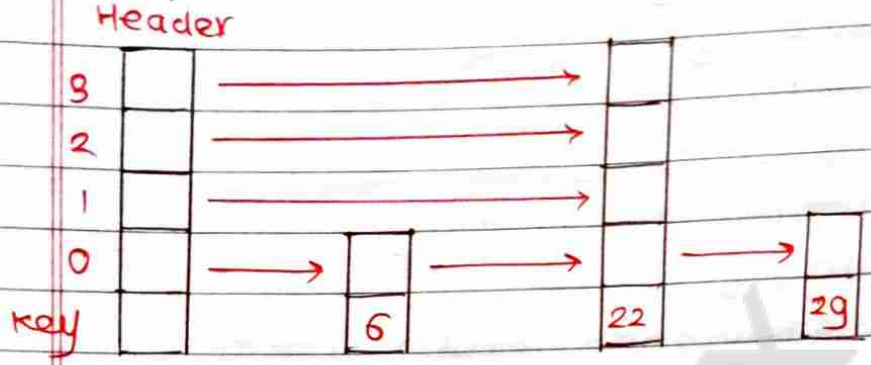



step 2 :- Insert 29 with level 1.

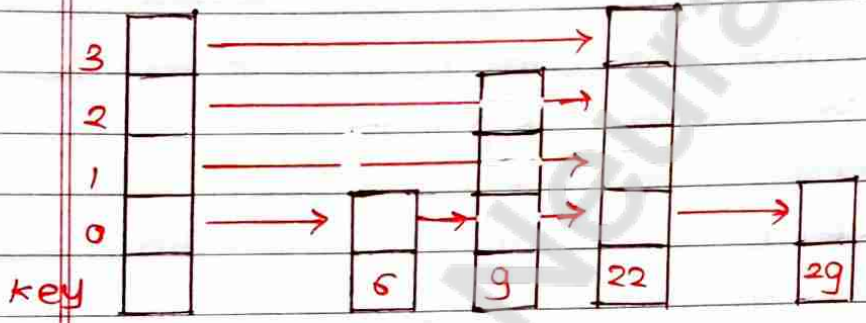



CodeNeural

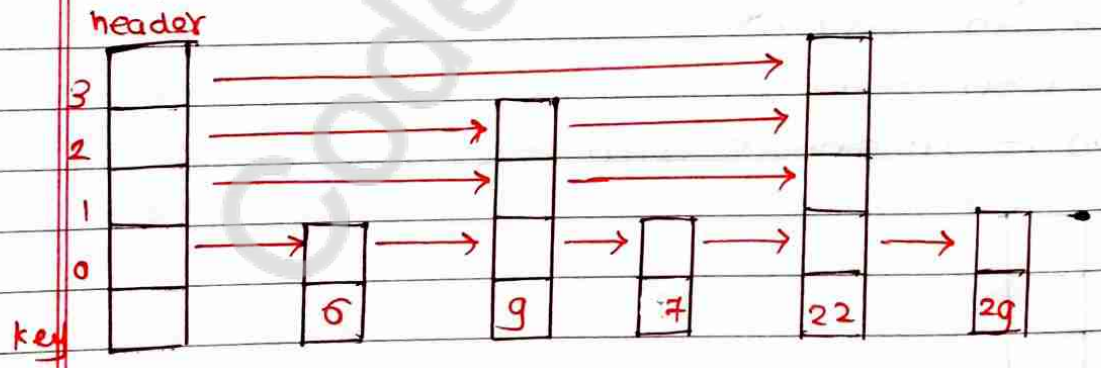
Step 3 : Insert 22 with level 4.



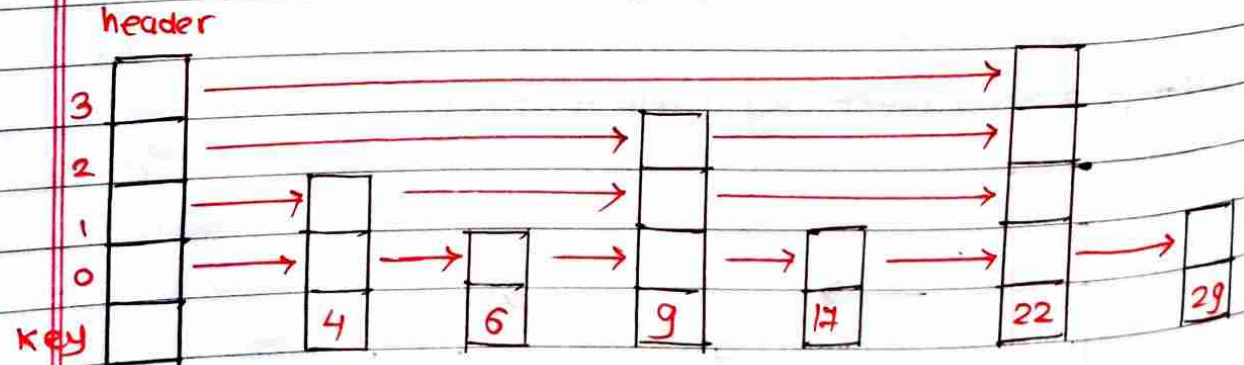

Step 4: Insert 9 with level 3.




Step 5: Insert 17 with level 1




Step 6: Insert 4 with level 2.




Codewyrax

Stack :- A stack is a linear data structure that follows LIFO (Last-In-First-Out) principle. Stack has one end, whereas queue has two ends (front and rear).

A stack is a container in which insertion and deletion can be done from the end (one) known as the top of the stack.

A stack is an Abstract Data Type with a pre-defined capacity, which means that it can store elements of limited size.

## Operations on the stack :-

1). push ( ) :- When we insert an element in a stack then the operation is known as push. If stack is full overflow condition occurs.

2). pop ( ) :- when we delete an element from stack, the operation is called as pop ( ). If stack is empty means no element exists in the stack, this state is known as an underflow state.

3). peek ( ) :- It returns the element at a given position.

4). count ( ) :- It returns the total number of elements available in a stack.

5). change ( ) :- It changes the element at the given position.

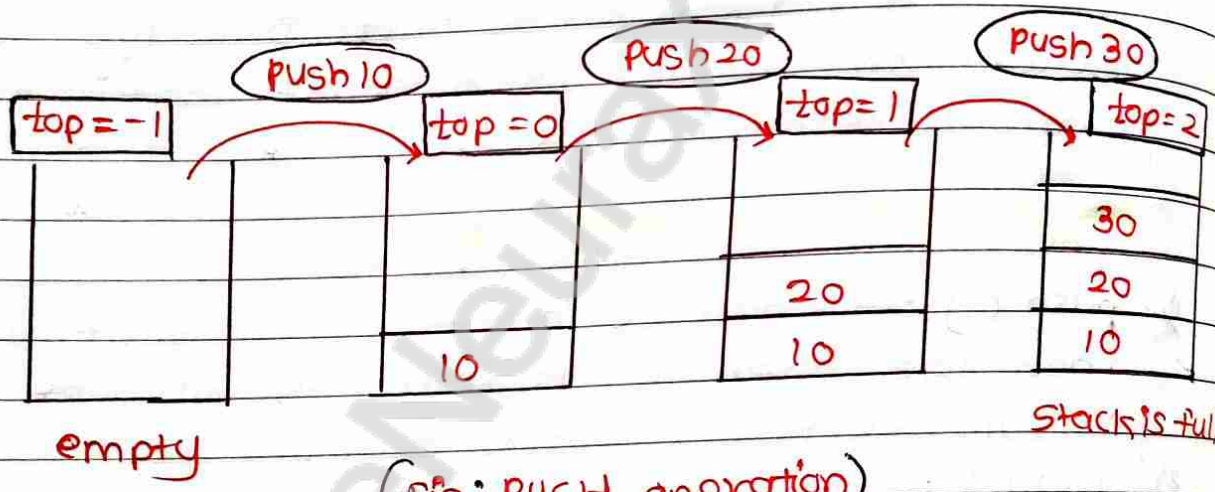
6). display ( ) :- It prints all the elements available in the stacks.

## PUSH operation :-

Steps — Before inserting an element in the a stack, we check whether the stack is full.

CodeFuray

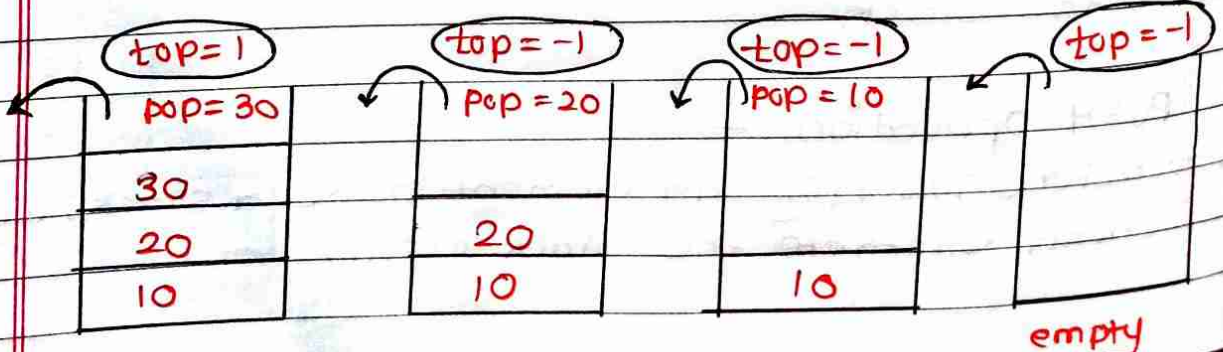
- If we try to insert element in a stack, and the stack is full, then overflow condition occurs.
- when we initialised a stack, we set the value of top as -1 to check that stack is empty
- The elements will be inserted until we reach the max size of the stack. top = top + 1.




(fig: PUSH operation)

## POP Operation :-

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from empty stack, then underflow condition occurs.
- first access the element which is pointed by top.
- once the top operation is performed, top is decremented by 1 i.e. top = top - 1.



Applications of stack :-

1) Recursion :- The recursion means that the function is calling itself again. To maintain the previous states, the compiler creates a system stack in which all previous records of function are maintained.

2) DFS (Depth first search) :- This search is implemented on a graph, graph uses stack d.s.

3) Backtracking :- If we have to create a path to solve maze problem, If we are moving in particular path and we realise that we come on the wrongway. In order to come at beginning of the path to create a new path, we use stack d.s.

4) Memory management :- The stack manages the memory. The memory is assign in the contiguous memory blocks.

Algo :- push operation :-

```
begin
  if top = n then stack full
  top = top + 1
  stack(top) := item ;
end
```

Time Complexity : O(1)

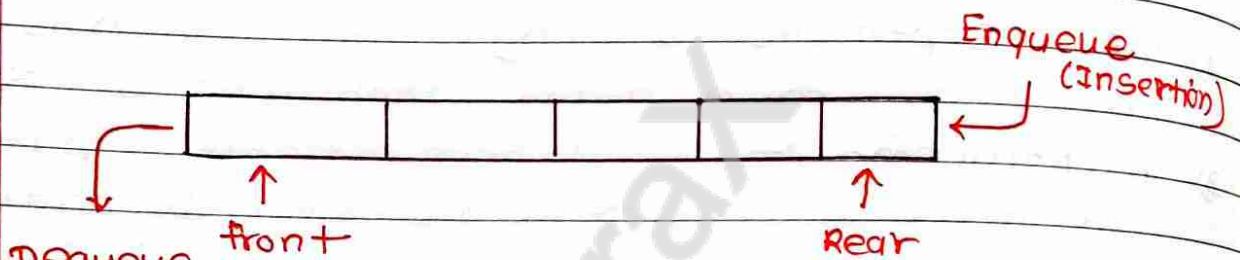
pop operation :-

```
begin
  if top = 0 then empty
  item := stack(top) ;
  top = top - 1 ;
end.
```

Time complexity : O(1)

**Queue :-** A queue can be defined as ordered list which enables insert operations to be performed at one end called REAR and delete operations to be performed at another end called FRONT.

- Queue can be referred as to be first In first out list.




Complexity of queue :-

| | Average | | | | Space comp |
|---|---|---|---|---|---|
| | Access | search | Deletion | Insertion | Worst |
| Queue | O(n) | O(n) | O(1) | O(1) | O(n) |

| | Worst | | | |
|---|---|---|---|---|
| | Access | search | Insertion | Deletion |
| Queue | O(n) | O(n) | O(1) | O(1) |

Operations on queue :-

1). Enqueue :- Enqueue is used to insert element at rear end of the queue. It returns void.

2). Dequeue :- dequeue operations performs the deletion from front end of queue. The deque operation can also be designed to void.

CodeNeural

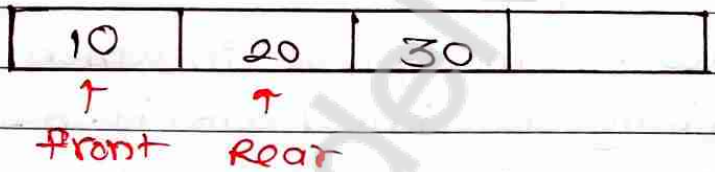
3). peek :- This returns, element which is pointed by front pointer in the queue but does not delete it.

4). queue overflow (is full) :- when queue is completely full, then it shows overflow condition.

5). queue underflow (isempty) :- when there is no element in the queue then it throws underflow condition.

## Types of queue :-

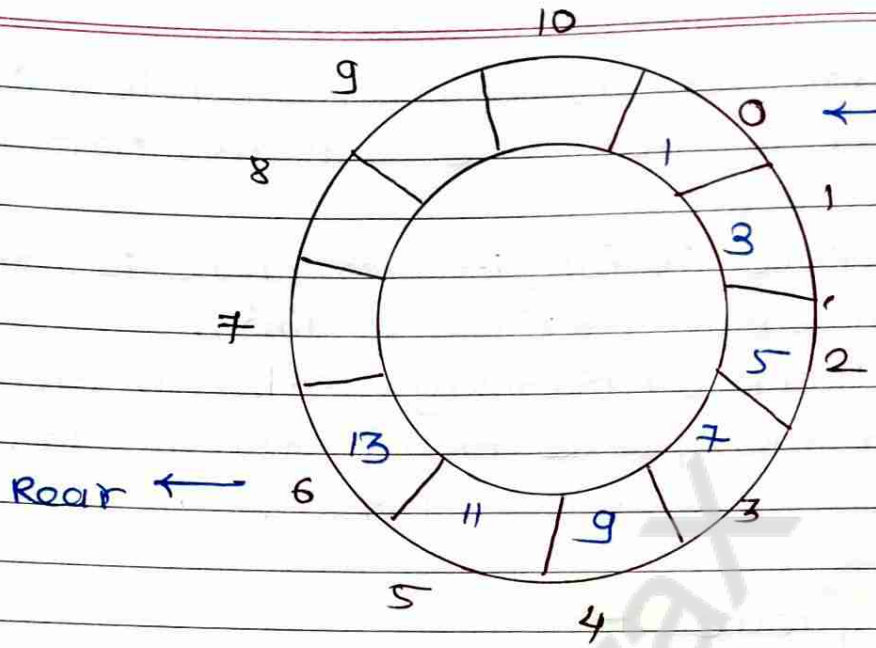
1). Linear queue :- In linear queue, an insertion takes place from one end while deletion occurs from another end. It strictly follows FIFO rule. The linear queue can be represented, as shown :




The elements are inserted from rear end, and if we insert more elements in queue, then rear value gets incremented on every insertion.
drawback is using linear queue is : insertion is done only from rear end. The linear queue shows the overflow condition as rear is pointing to last element of the queue.

2). circular queue :- In circular queue, all nodes are represented as circular. It is similar to linear queue except that last element of the queue is connected to the first element. It is also known as ring buffer. as all ends are connected to another end.

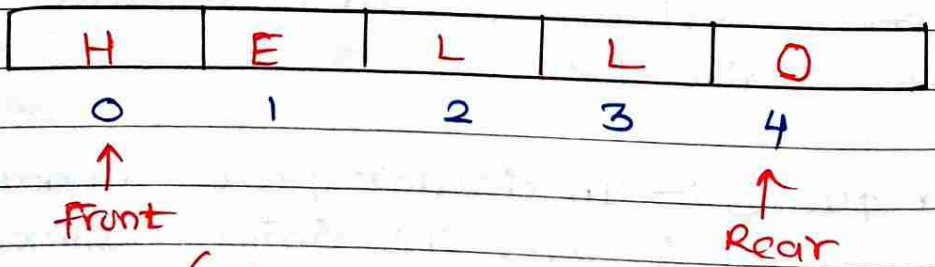
CodeVeluAx




Drawback of linear queue is overcomes in circular queue. If empty space is available, new element can be added in empty space by simply incrementing value of rear.

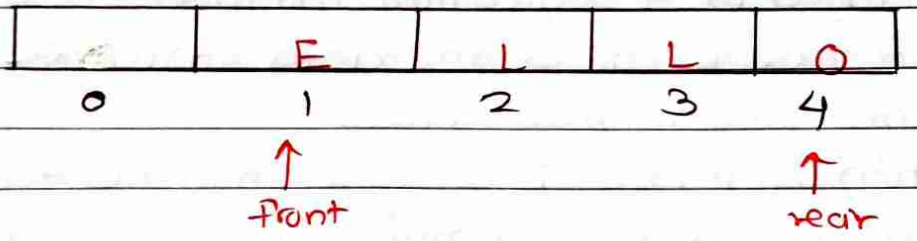
3). Priority queue :— The queue in which each element has some priority associated with it. Based on priority of the element, elements are arranged in a priority queue. If elements occur with same priority, then they are served according to FIFO principle.

* Array representation of queue :—




(Fig : queue after inserting an element)

After deleting element, value of front will increase from -1 to 0, the queue will look like :




(Fig : queue after deleting an element)

Algorithm to insert any element in a queue :-
check if queue is already full by comparing rear to max -1.

Algo: step 1 :- IF REAR = MAX -1
write OVERFLOW
Go to step [END OF IF]
step 2 :- IF FRONT = -1 and REAR = -1
SET FRONT = REAR = 0
ELSE
SET REAR = REAR + 1 [END OF IF].
step 3 :- SET QUEUE [REAR] = NUM
step 4 :- EXIT.

Algorithm to delete an element from queue :-

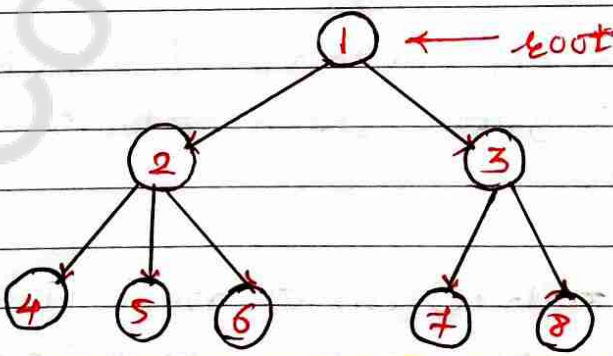
Algo: step 1 :- IF FRONT = -1 or FRONT > REAR
write UNDERFLOW
ELSE
SET VAL = QUEUE [FRONT]
SET FRONT = FRONT + 1
[END OF IF]
step 2 :- EXIT.

**Tree :-** We read data structure, like an array, linked list, stack and queue in which all elements are arranged in a sequential manner.

A tree is one of the data structures that represents hierarchical data.

**defination :-** A tree is a data structure defined as collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.

- A tree is a non-linear data structure because it does not store in a sequential manner. It is a hierarchical structure as elements in tree are arranged in multiple levels.
- In tree data structure topmost node is called as root node. Each node contains some data & data can be of any type.
- Each node contains some data & links or reference of other nodes that can be called children.




**Some Basic terms of tree :-**

1). **link :-** Each node is labeled with some number. Each array shown in fig is known as link between two nodes.

2). **Root :-** The root node is top most node in tree hierarchy. root node is one that doesn't have any parent. If node is directly linked to some other

node, then it would be called a parent-child relation-ship.

3). child node :- If the node is a descendant of any node, then node is called as child node.

4). parent :- If node contains any sub-node, then node is said to be parent of that sub-node.

5). sibling :- The nodes that have same parents are called siblings.

6). leaf node :- node which doesn't have any child node, a leaf a bottom-most node of tree.

7). ancestor node :- It is any predessor node on a path from root to that node. In the given fig, 1, 2, 5 are ancestors of node 10.

8). Descendant :- The immediate successor of given node is known as descendant of a node.

## * Properties of tree data structures :-

1). Recursive data structure :- Tree is also known as recursive data structure. Recursion means reducing something in a self-similar manner.

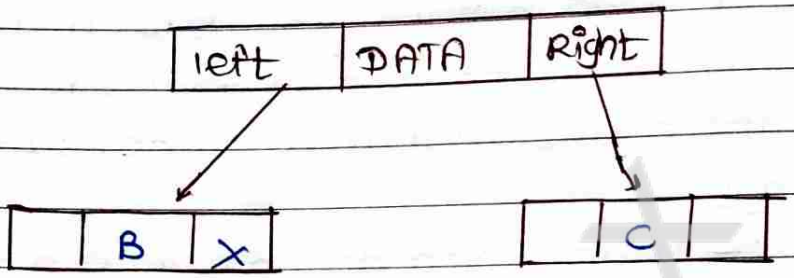
2). Number of edges :- If there are (n) nodes, then there would be (n-1) edges. each node, except root node, will have atleast one incoming line known as an edge.

3). Depth of node x :- It can be defined as length of path from root to node x. one edge contributes one unit length in the path, depth can be defined as no. of edges between root node and node (x)
The root node has depth 0.

4). Height of node x :- It is defined as longest path from node x to leaf node.

## Implementation of tree :-

The tree data structure can be created nodes dynamically with help of pointers. The tree in memory can be represented as shown :




```
Struct node
 {
  int data;
 struct node *left;
 struct node *right;
 }
```

The above structure can only be defined for binary trees because binary tree can have utmost two children, and generic trees.

## Application of trees :-

1). storing naturally hierarchical data :- File system., stored on disc drive, file and folder are in form of naturally heirarchical data and store in form of trees.

2). organize data :- It is used to organize data for efficient insertion, deletion and searching.

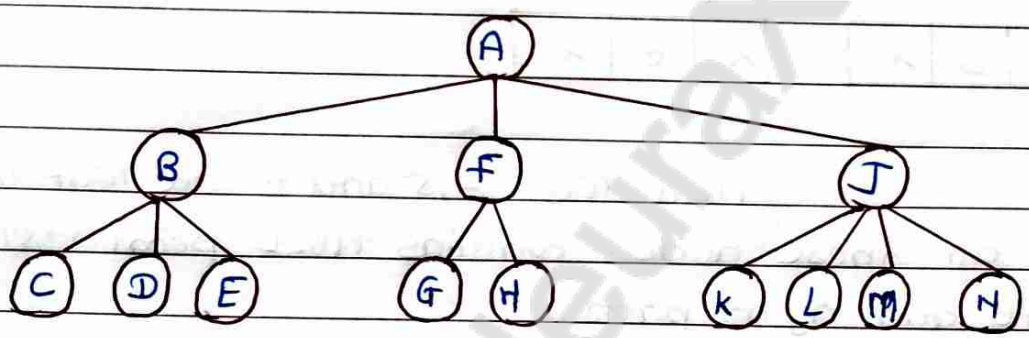
3). Trie :- It is special kind of tree that is used to store dictionary. It is fast and efficient way for dynamic spell checking.

4). Heap :- It is also a tree data structure implemented using arrays. It is used to implement priority queues.

Codenewra

## Types of Tree data structure :—

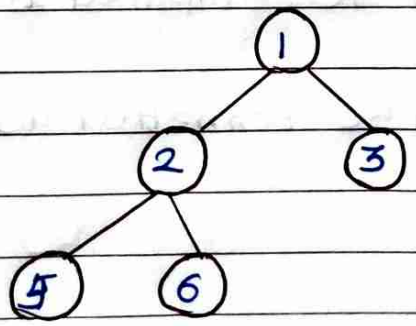
1). General Type :— In a general tree, a node can have either 0 or maximum n number of nodes. There is no restrictions imposed on the degree of node (number of nodes that a node can contain) The topmost node in a general tree is known as root node. The children of parent node are known as subtree.




There can be n number of subtrees in general tree. In general tree, subtrees are unordered as nodes in subtree cannot be ordered.

Every non empty tree has a downward edge, and these edges are connected to nodes known as child nodes. The nodes that have same parent are known as siblings.

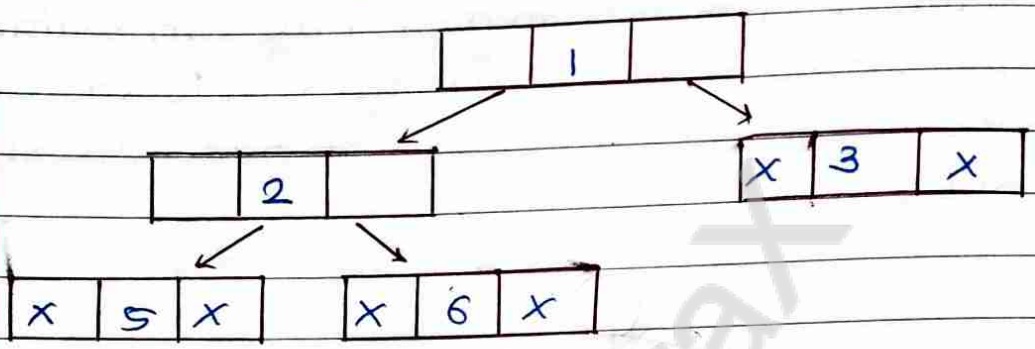
2). Binary Tree :— Binary tree means that the node can have maximum two children.




← given tree is a binary tree because
- each node contains atmost two children.

logical representation of above tree :-

In above tree, node 1 contains two pointers i.e. left and right pointer pointing to left and right node respectively.




The nodes 3, 5 and 6 are leaf nodes, so all these nodes contains NULL pointer on both left and right parts.

Properties of Binary tree :-

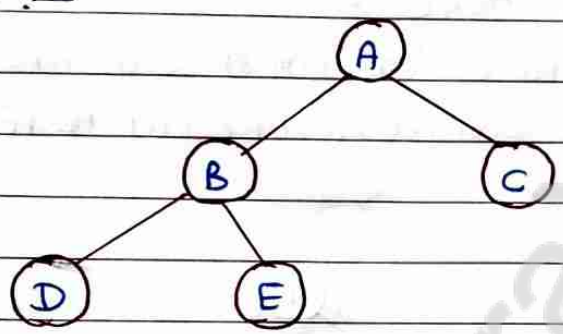
- At each level of i, the maximum number of nodes is $2^i$.
- The height of tree is longest path from root node to leaf node. In general, maximum number of nodes possible at height is $(2^0 + 2^1 + 2^2 + .. 2^h)$
- The minimum number of nodes possible at height h is equal to h+1.
- If number of nodes is minimum, then height of tree would be maximum.
- minimum height can be computed as:
  $h = \log_2(n+1) - 1$
- maximum height can be computed as:
  $h = n - 1$.

## Types of Binary tree :-

1). full / proper / strict Binary tree :-

If each node contains either 0 or two children. The tree in which each node must contain 2 children except left nodes.

Example :-



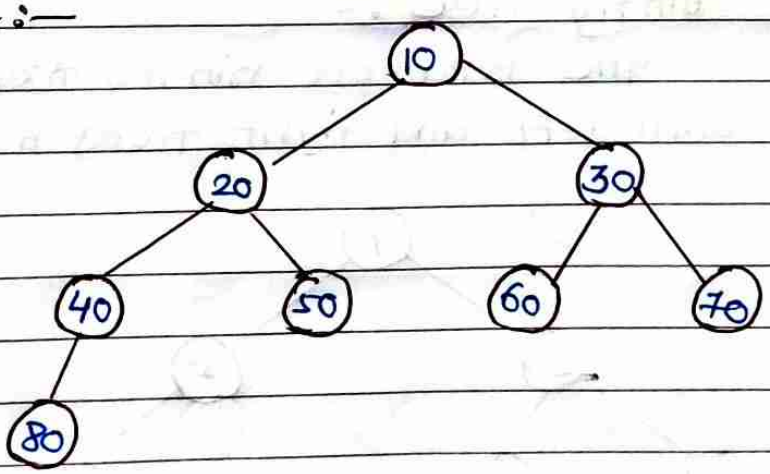

Properties :-

- maximum number of nodes : $2^{h+1} - 1$.
- minimum number of nodes : $2 * h - 1$
- minimum height $\log_2 (n+1) - 1$
- maximum height $h = \frac{n+1}{2}$

2). complete Binary tree :-

The tree in which all nodes are completely filled except the last level. In complete Binary tree nodes should be added from left.

Example :-




Codelibrary

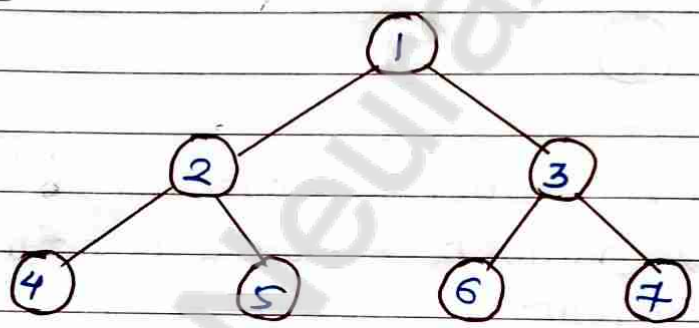
properties :-

- maximum number of nodes → $2^{h+1} - 1$.
- minimum number of nodes → $2^h$
- minimum height → $\log_2 (n+1) - 1$.

3). Perfect Binary tree :-

A tree in which all the internal nodes have 2 children. and all leaf nodes are at the same level.

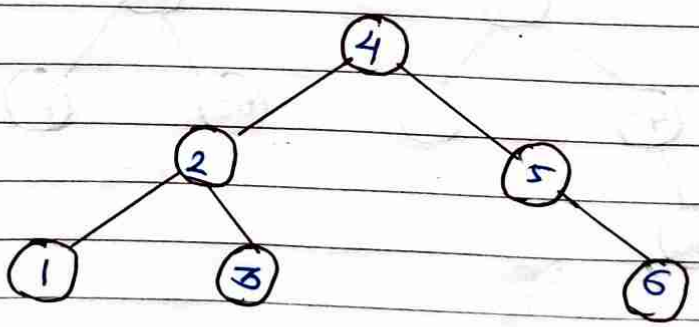
Example :-




Note :- All the perfect Binary trees are complete binary trees as well as the full Binary trees as But, vice versa is not true, all complete binary trees and full binary trees are the perfect Binary trees

4). Balanced Binary Tree :-

The balanced binary tree is a tree in which both left and right trees by almost 1.




above tree is balance : diff bet^n left subtree & right S.T. is zero

Binary Tree implementation :-

```
struct node
 {
  int data;
  struct node *left, *right;
 }
```

Tree Traversal :-

The process of visiting nodes is called as tree traversal.

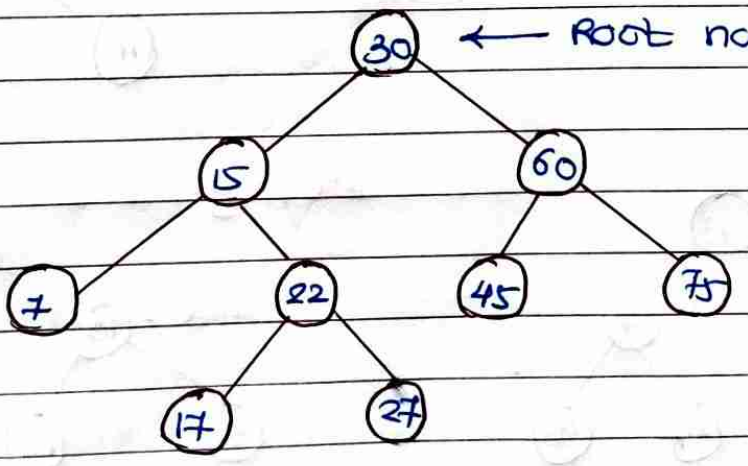
There are three types of traversals used to visit a node :

1). Inorder Traversal
2). preorder Traversal
3). postorder Traversal

3). Binary Search Tree :-

defn :- Binary search tree can be defined as a class of binary trees, in which a nodes are arranged in a specific order. also called as ordered BinaryTree.

- similarly value of all nodes in right subtree is greater than or equal to value of root.



Example : create binary search tree using following data elements :-

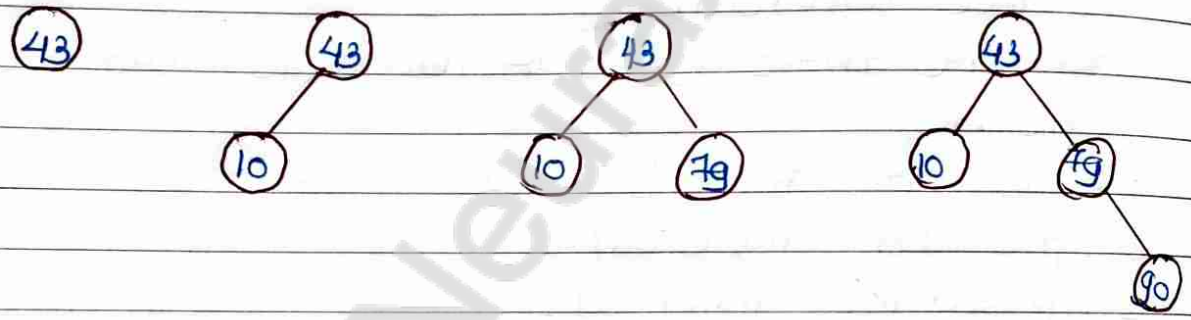
43, 10, 79, 90, 12, 54, 11, 9, 50.

→ 1). Insert 43 into tree as root of tree.

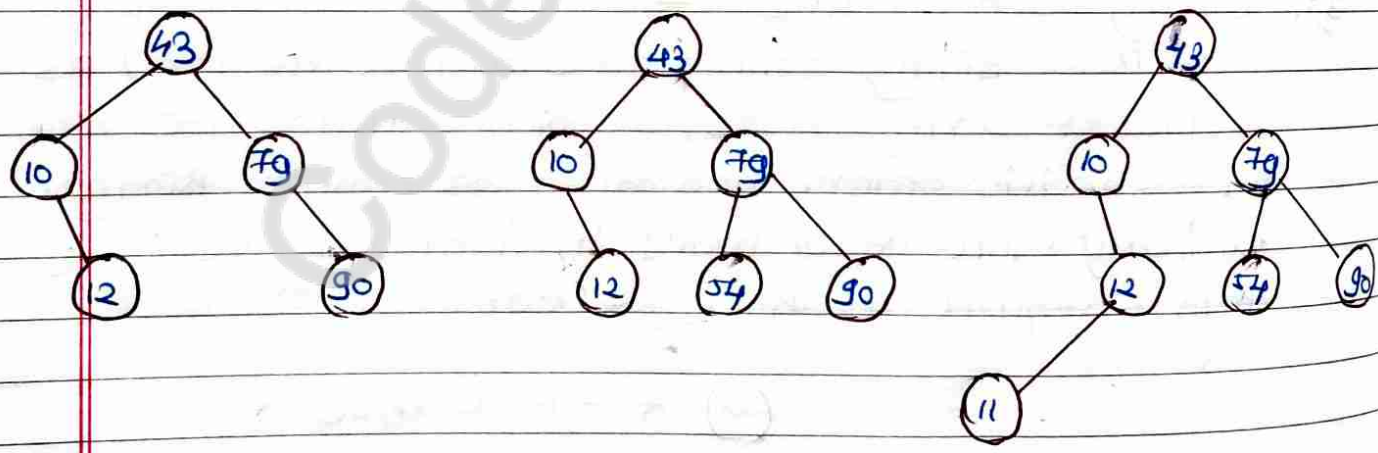
2). Read next element, if it is lesser root node element, insert it as root of left sub-tree.

3). otherwise, insert it as root of right of right subtree.

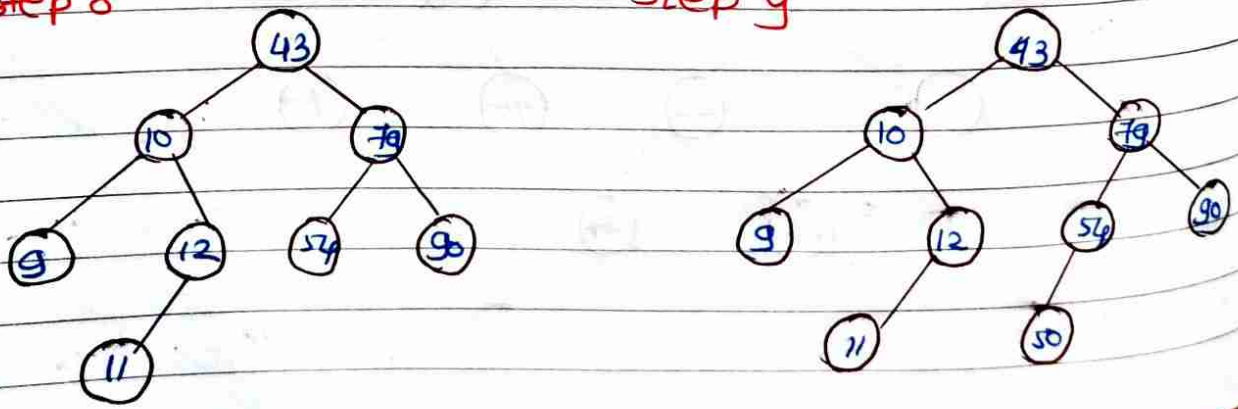
step 1 step 2 step 3 step 4




step 5 step 6 step 7




step 8 step 9




CodeNeuro

## Operations on Binary Search Tree (BST) :—

| Sr.No. | Operation | Description |
|---|---|---|
| 1). | searching in BST | finding location of some specific element in a Binary search tree. |
| 2). | Insertion in BST | Adding a new element to the binary search tree at appropriate location so that property of BST do not violate. |
| 3). | Deletion in BST. | Deleting some specific node from a BST, However, tree there can be various cases in deletion depending upon number of children, node have. |

4). AVL Tree :— AVL tree is invented by GM Adelson-Velsky and EM Landis in 1962. The tree is named as AVL in honour of its inventors.

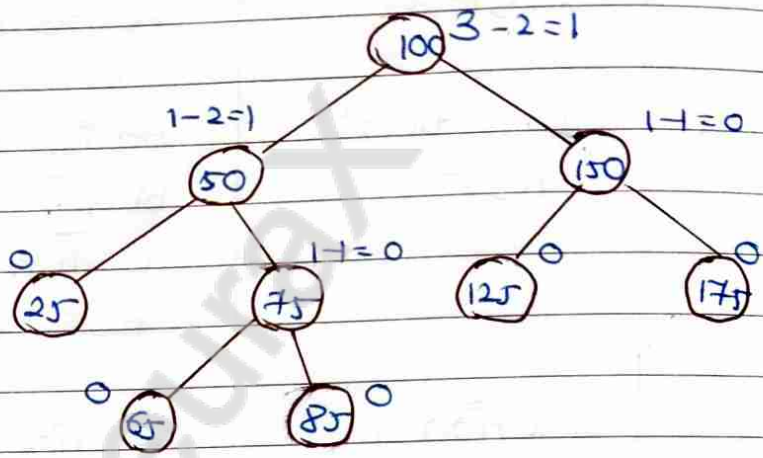
AVL tree is defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its R. subtree from its left subtree.

Balance factor (k) = height(left(k)) − height(right(k))

— If balance factor of any node is 1, it means that left sub-tree is one level higher than right subtree.

- If balance factor of any node is 0, it means that left sub-tree and right sub-tree contain equal height.
- If balance factor of any node is -1, it means that left sub-tree is one level lower than right subtree.

Example :-




Here we see that, balance factor associated with each node is between -1 and +1.
∴ It is an example of AVL tree.

Complexity :-

| Algorithm | Average case | Worst case |
|---|---|---|
| space | O(n) | O(n) |
| search | O(log n) | O(log n) |
| Insert | O(log n) | O(log n) |
| Delete | O(log n) | O(log n) |

Why AVL Tree ? → AVL tree controls height of binary search tree by not letting it to be skewed. The time taken by all operations in BST is O(h) However it will be extended to O(n) if BST became

skewed (worst case). By limiting this height to log n, AVL tree imposes an upper bound on each operation to be O(log n), where n is number of nodes.

Operations on AVL tree :-

| Sr. No. | Operation | Description. |
|---|---|---|
| 1). | Insertion | Insertion is performed in same way it performed in BST. However, it may lead to violation in the AVL tree property and so tree may need balancing. and tree can be balanced by rotation. |
| 2). | Deletion | Deletion is also same way performed as BST It can be also disturb balance of tree, so various types of rotations are used to rebalance tree. |

AVL Rotations :-

We perform rotations in AVL tree only in case if Balance factor is other than -1, 0 and 1.
There are Basically four types of rotations which are as follows :

CodeNeural

1). L-L rotation :- Inserted node is in the left subtree of left subtree of A.

2). R-R rotation :- Inserted node is in the right subtree of right subtree of A.

3). L-R rotation :- Inserted node is in right subtree of left subtree of A.

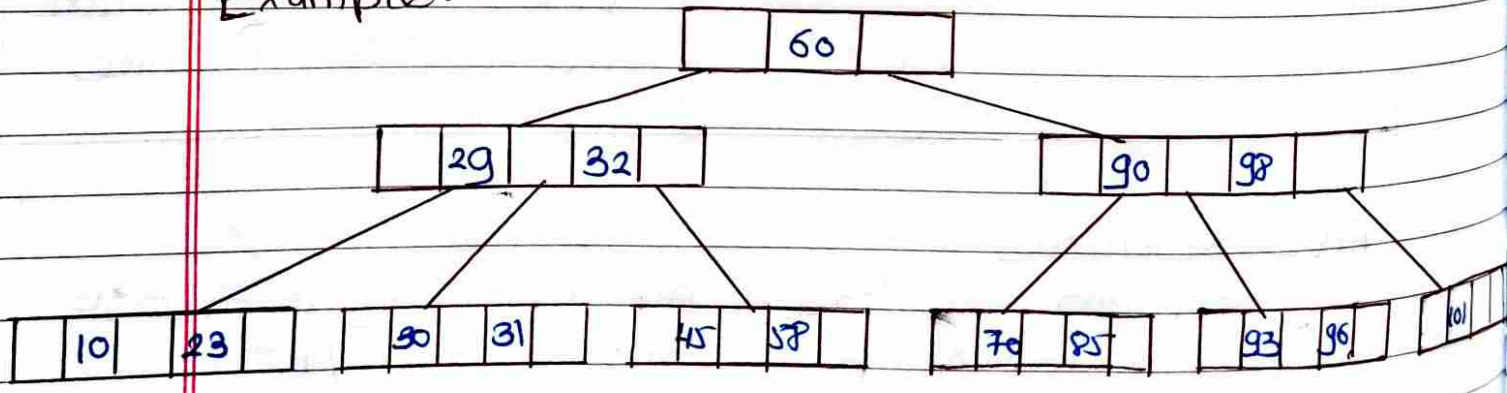
4). R-L rotation :- Inserted node is in the left subtree of right subtree of A.

5). B Tree :- B Tree is specialized m-way tree that can be widely used for disk access. A B-tree of order m can have at most m-1 keys and m children.

Properties :-

1. Every node in B-Tree contains at most m children.
2. Every node in B-Tree except root node and leaf node contain at least m/2 children.
3. The root nodes must have at least 2 nodes.
4. All leaf nodes must be at the same level.

Example :-



## Operations :-

1). searching :- The searching in B tree is similar to searching in Binary tree. For example, we search for an item 49 in following B Tree. The process will be :
① compare item 49 with root node 78. since $49 < 78$ hence, move its left sub-tree.
② since, $40 < 49 < 56$, traverse right subtree of 40.
③ $49 > 45$, move to right compare 49.
④ match found, return.

Searching in B tree depends upon height of the tree. The search algorithm takes $O(\log n)$ time to search any element in B tree.

2). Inserting :- Insertion are done at leaf node level. The following algorithm needs to be followed in order to insert an item into B tree.
① Traverse B tree in order to find appropriate leaf node at which node can be inserted.
② If leaf node contains less than m-1 keys then insert element in increasing order.
③ Else, if leaf node contains m-1 keys, then follow following steps:
- Insert new element in increasing order of elements.
- split node into two nodes at median.
- Push median element up to its parent node.
- If parent node also contain m-1 number of keys, then split it too by steps.

## Application of B tree :-

- B tree is used to index data and provides fast access to actual data stored on disks since, the

to value stored in a large database that is stored on a disk is a very time consuming process.

searching an un-indexed and unsorted database containing n key values needs O(n) running time.

6). B + Tree :-

B+tree is an extension of B tree which allows efficient insertion, deletion and search operations.

The leaf nodes of B+ tree are linked together in form of the singly linked list to make search queries more efficient.

Advantages of B+ tree :-

1). Records can be fetched in equal number of disk accesses.

2). Height of tree remains balanced and less as compare to B tree.

3). We can access data stored in B+ tree sequentially as well as directly.

4). Keys are used for indexing.

## Graph :-

A graph can be defined as group of vertices and edges that are used to connect these vertices.

Defination :-

A graph G can be defined as an ordered set G(V, E) where V(G) represents set of vertices and E(G) represents set of edges.

CodeNeural

which are used to connect these vertices.

Directed and Undirected Graph :-

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with directions with them.

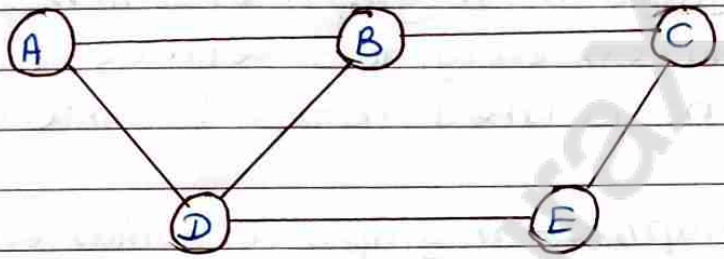



fig : Undirected graph

As above figure edges are not attached with any of the directions.

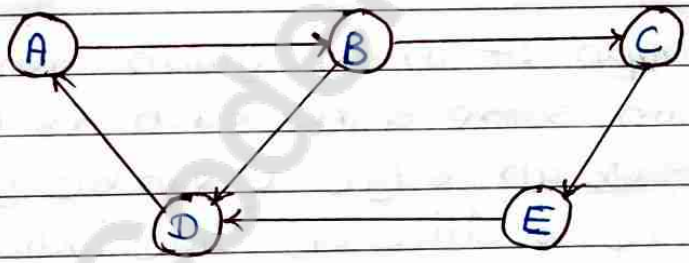



fig : directed graph.

In above figure, directed graph edges form an ordered pair.

## Graph Terminology :-

1). Path :- A path can be defined as sequence of nodes that are followed in order to reach some terminal node v from initial node v.

2). closed path :- A path will be called as closed if initial node is same as terminal node. $V_0 = V_N$

3). Simple path :- If all nodes of graph are distinct with an exception $V_0 = V_N$, then such path p is called as closed simple path.

4). Cycle :- A cycle is a path which has no repeated edges or vertices except first and last vertices.

5). Connected graph :- A graph in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.

6). Complete graph :- A graph in which every node is connected with all other nodes. A complete graph contain $\frac{n(n-1)}{2}$ edges where n is number of nodes in graph.

7). Weighted graph :- In this graph each node is assigned with some data such as length or width The weight of an edge e can be given as w(e) which must be positive (+) value indicating cost of traversing edge.

8). Diagraph :- A diagraph is directed graph in which each edge is associated with some direction and traversing can be done only in specified direction.

9). Loop :- An edge that is associated with the similar end points can be called as loop.

10). Adjacent Nodes :- If two nodes u and v are connected via an edge e, then nodes u and v

are called as neighbours or adjacent nodes.

ii). Degree of a Node :- A degree of a node is a number of edges that are connected with that node. A node with degree 0 is called isolated.

## Graph Representation :-

We simply mean, technique which is to be used to in order to store some graph into the computers memory.

(1). Sequential Representation :- In this we use adjacency matrix to store mapping represented by vertices and edges. A graph having n vertices, will have a dimension n x n.

An entry Mij in adjacency matrix representation of an ~~not~~ undirected graph G will be 1 if there exists an edge between Vi and Vj.

An undirected graph and its adjacency matrix representation is shown in following :

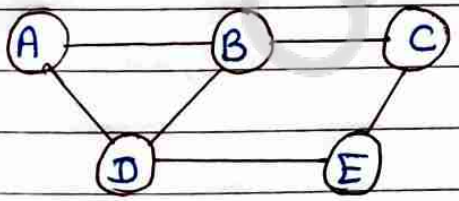



fig : Undirected graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 | 0 |
| B | 1 | 0 | 1 | 1 | 0 |
| C | 0 | 1 | 0 | 0 | 1 |
| D | 1 | 1 | 0 | 0 | 1 |
| E | 0 | 0 | 1 | 1 | 0 |

Fig: Adjacency matrix

In above figure, we can see mapping among vertices (A, B, C, D, E) is represented by using adjacency matrix which is also shown in fig.

A directed graph and its adjacency matrix representation is shown in figure :

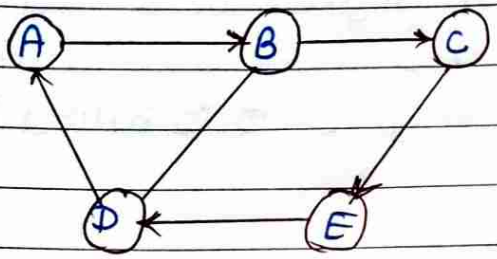



Fig : Directed Graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 1 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

fig : Adjacency matrix

Representation of weighted directed graph is different. Instead of filling entry by 1, non zero entries of adjacency matrix are represented by weight of respective edges.

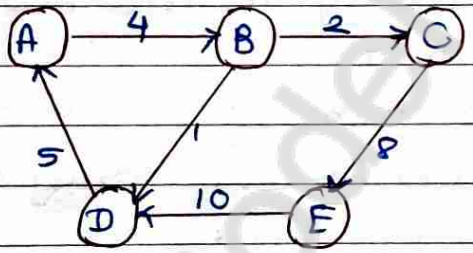



Fig : weighted directed graph

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 4 | 0 | 0 | 0 |
| B | 0 | 0 | 2 | 1 | 0 |
| C | 0 | 0 | 0 | 0 | 8 |
| D | 5 | 0 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 10 | 0 |

fig : Adjancy matrix

(2) linked representation :-

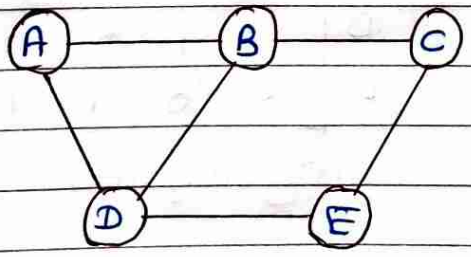



fig : undirected graph

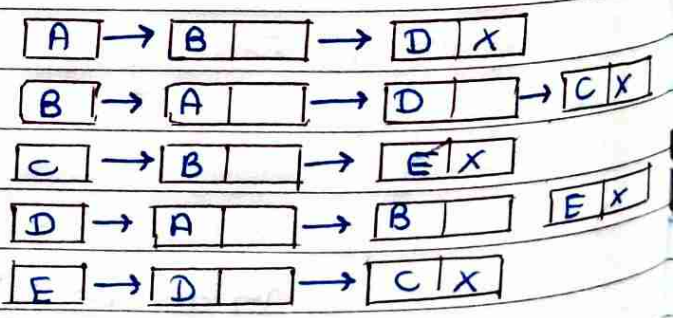



fig : Adjacency list.

An adjacency list is maintained for each node present in graph which stores node value and a pointer to next adjacent node to respective node.

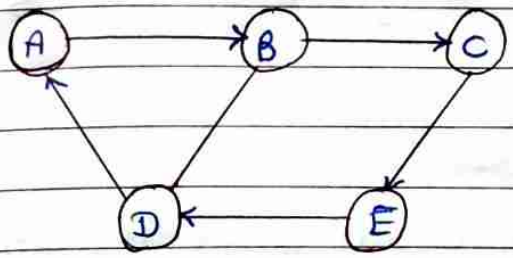



fig : Directed graph

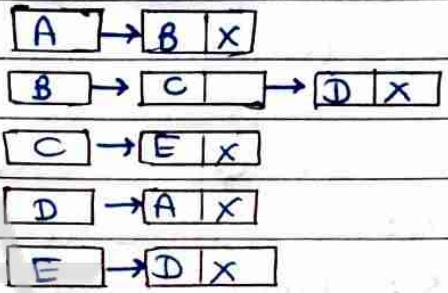



fig : Adjacy list

In directed graph, sum of lengths of all the adjacy lists is equal to the number of edges preseht in the graph.

## Graph Traversal Algorithm :-

In this tutorial we will learn all techniques by using which, we can traverse all the vertices of the graph. Traversing means examining all nodes and vertices of graph. There are two standard methods by using which, we can traverse graphs.

- Breadth first search
- Depth first search

### 1. Breadth first search (BFS) algorithm :-

Breadth first search is a graph traversal algorithm that starts traversing graph from root node and explores all the neighbouring nodes.

Then, it selects nearest node and explore all unexplored nodes. The algorithm follows same process for each of nearest node until it finds goal.

Algorithm :-

step 1: SET STATUS = 1 (ready state) for each node in G.

step 2: Enqueue starting node A & set its STATUS = 2 (waiting state).

step 3: Repeat steps 4 and 5 until QUE E is empty.

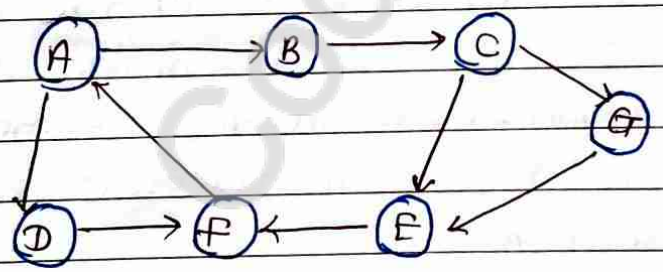
step 4: Dequeue a node N. Process it & set its STATUS = 3

step 5: Enqueue all neighbours of N that are in ready state (whose STATUS = 1) & set (STATUS = 2) [END OF LOOP].

step 6: EXIT.

Example :-

Consider graph G shown in following image, calculate minimum path P from node A to node E. Given that each edge has a length of 1.




Adjacency lists :

A : B, D
B : C, F
C : E, G
G : E
E : B, F
F : A
D : F.

Solution :-

Minimum Path P can be found by applying Breadth first search algorithm that will begin at node A and will end at E.

A → B → C → E

## Depth first search Algorithm :-

DFS algorithm starts with initial node of graph G, & goes to deeper & deeper until we find goal node / node which has no children. The data structure used in DFS is stack.

Algorithm :-

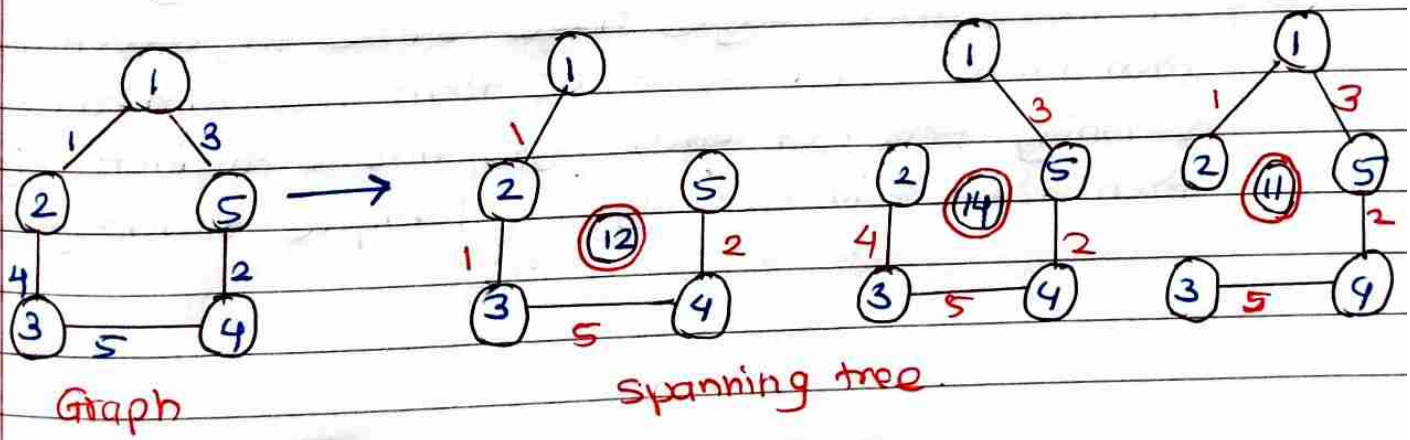
step 1: SET STATUS = 1 (ready state) for each node in G.
step 2: Push starting node A on stack & set its STATUS = 2 (waiting state).
step 3: Repeat steps 4 and 5 until stack is empty.
step 4: Pop top node N. Process it & set its STATUS = 3.
step 5: Push on stack all neighbours of N that are in ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state) [END OF LOOP].
step 6: EXIT.

## Spanning Tree :-

If we have a graph containing V vertices and E edges, then graph can be represented as: G(V, E). If we create spanning tree from above graph, then spanning tree would have same number of vertices as the graph, but vertices are not equal.

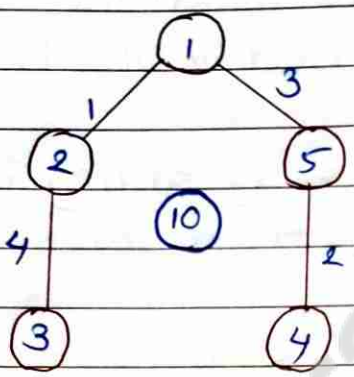
edges (spanning tree) = no. of edge (in graph) - 1.

Example :-



## Minimum Spanning Trees :-

The minimum spanning tree is a tree whose sum of edge weights is minimum.




In above tree, total edge weight is less than above spanning trees, therefore a minimum spanning tree is a tree which is having an edge weight i.e. 10.

## Properties of Spanning tree :-

- A connected graph can contain more than one spanning tree.
- All possible spanning trees that can be created from given graph G would have same number of vertices in given graph minus 1.
- Spanning tree does not contain any cycle, let's understand this property through an example.
- Spanning tree cannot be disconnected. If we remove one more edge from any of above spanning trees as.
- If two / more edges have same edge weight, then there would be more than two minimum spanning tree. If each edge has a distinct weight, then there will be only one / unique spanning tree.

CodeNeeraj

Applications of Spanning tree :-

1). Building a network :- Suppose there are many routers in network connected to each other, so there might be a possibility that it forms a loop.

2). Clustering :- clustering means that grouping set of objects in such way that similar objects belong to same group than to different group. our goal is to divide the n objects into k groups such that distance between different groups gets maximised.

## Searching :-

Searching is a process of finding some particular element in list. If the element is present in the list, then process is called successful and process returns location of that element, otherwise search is called unsuccessful.

There are two methods widely used as below:

- Linear search
- Binary search

1). Linear search :-

linear search is a simplest sequential search algorithm and often called sequential search. In this type of searching, we simply traverse the list completely and match each element of list with item whose location is to be found.

linear search is mostly used to search an unordered list in which items are not sorted. The algorithm is given as follows :

CodeNeuraX

Algorithm :-

LINEAR_SEARCH (A, N, VAL)

step 1 :- [INITIALIZE] SET POS = -1

step 2 :- [INITIALIZE] SET I = 1

step 3 :- Repeat step 4 while I <= N

step 4 :- IF A[I] = VAL
SET POS = I
PRINT POS
Go to step 6
[END OF IF]
SET I = I+1
[END OF LOOP].

step 5 :- IF POS = -1
PRINT "VALUE IS NOT PRESENT IN ARRAY"
[END OF IF].

step 6 :- EXIT.

Complexity of an Algorithm :-

| Complexity | Best case | Average case | Worst case |
|---|---|---|---|
| Time | O(1) | O(n) | O(n) |
| Space | | | O(1) |

C program of linear search :-

```
#include <stdio.h>
 void main ()
{
```

```
int a[10] = { 10, 23, 40, 1, 2, 0, 14, 13, 50, 9};
int item, i, flag;
printf("\n Enter Item which is to be searched \n");
scanf("%d", item);
for (i=0; i<10; i++)
  {
   if (a[i] == item)
    {
     flag = i+1;
     break;
    }
  else
     flag = 0;
}
  if (flag != 0)
   {
   printf("\n item found at location %d \n", flag);
   }
   else
   {
     printf("\n Item not found \n");
   }
}
```

Output :- Enter Item which is to be searched
20
Item not found
Enter Item which is to be searched
23
Item found at location 2.

CodeNeurax

## 2). Binary Search :-

Binary search is a search technique which works on efficiently on sorted lists. Hence in order to search an element into some list by using binary search technique, we must ensure that list is sorted.

Binary search follows divide and conquer approach in which, list is divided into two halves and item is compared with middle element of list.

Binary search Algorithm :-

BINARY_SEARCH (A, lower_bound, upper_bound, VAL)

step 1 :- [INITIALIZE] SET BEG = lower_bound
END = upper_bound, POS = -1.

step 2 :- Repeat steps 3 and 4 while BEG <= END

step 3 :- SET MID = (BEG + END)/2

step 4 :- IF A[MID] = VAL
SET POS = MID
PRINT POS Go to step 6
ELSE IF A[MID] > VAL
SET END = MID - 1
ELSE SET BEG = MID + 1

step 5 :- IF POS = -1
PRINT "VALUE IS NOT PRESENT IN ARRAY"

step 6 :- EXIT.

Complexity :-

| Sr.No. | Performance | complexity |
|---|---|---|
| 1). | Worst case | O(log n) |
| 2). | Best case | O(1) |
| 3). | Average case | O(log n) |
| 4). | Worst case space complexity | O(1) |

Example: Let us consider an array arr = {1, 5, 7, 8, 13, 19, 20, 23, 29}. Find location of item 23 in the array.

→ In 1st step :-

BEG = 0

END = 8 ron

MID = 4

a[mid] = a[4] = 13 < 23, therefore;

In second step :-

Beg = mid + 1 = 5

End = 8

mid = 13/2 = 6

a[mid] = a[6] = 20 < 23, therefore;

In third step :-

beg = mid + 1 = 7

End = 8

mid = 15/2 = 7

a[mid] = a[7].

CodeNeural

a[7] = 23 = item ;
therefore, set location = mid ;
The location of item will be 7.

item to be searched : 23

step 1 →

| 1 | 5 | 7 | 8 | 13 | 19 | 20 | 23 | 29 |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

step 2 →

| 1 | 5 | 7 | 8 | 13 | 19 | 20 | 23 | 29 |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

step 3 →

| 1 | 5 | 7 | 8 | 13 | 19 | 20 | 23 | 29 |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

In step 1: a[mid] = 13
13 < 23
beg = mid + 1 = 5
end = 8
mid = (beg + end) / 2 = 13/2 = 6

In step 2 :- a[mid] = 20
20 < 23
beg = mid + 1 = 7
end = 8
mid = (beg + end) / 2 = 15/2 = 7.

step 3 :- a[mid] = 23
23 = 23
loc = mid.

C program : Binary search

```
#include<stdio.h>
int binary search (int [], int, int, int);
void main ()
```

```
{
int arr[10] = {16, 19, 20, 23, 45, 56, 78, 90, 96, 100};
int item, location = -1;
printf(" Enter the item which you want to search");
scanf(" %d ", &item);
location = binarysearch (arr, 0, 9, item);
if (location != -1)
{
printf( "Item found at location %d ", location);
}
else
{
printf ("item not found");
}
}
int Binary search (int a[], int beg, int end, int item)
{
int mid;
if (end >= beg)
{
mid = (beg + end)/2;
if (a [mid] == item)
{
return mid+1;
}
else if (a [mid] < item)
{
return binary search (a, mid+1, end, item);
}
else
```

CodeNeurax

```
{
return binary search (a, beg, mid-1, item);
}
}
return -1;
}
```

output: Enter item which you want to search
19
Item found at location 2.

## Sorting Algorithm :-

1) Bubble sort Algorithm :-

Bubble sort algorithm is a simplest sorting algorithm. Bubble sort works on repeatedly swapping of adjacent elements until they are not in intended order. It is called as Bubble sort because moment of array elements is just like movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly the array elements in bubble sort move to end in each iteration.

It is not suitable for large data sets. The average and worst case complexity of bubble sort is $O(n^2)$, where n is number of items.

Bubble sort is majorly used where :

- complexity does not matter
- simple and shortcode is preferred.

Algorithm :-

```
begin BubbleSort(arr)
  for all array elements
  "if arr[i] > arr[i+1]
    swap(arr[i], arr[i+1])
  endif
 end for
 return arr
end Bubble sort.
```

Bubble sort complexity :-

| case | Time complexity | space complexity |
|---|---|---|
| Best case | $O(n)$ | $O(1)$. |
| Average case | $O(n^2)$ | |
| Worst case | $O(n^2)$ | |

Implementation of Bubble Sort :-

C language Implementation :-

```
#include<stdio.h>
void print(int a[], int n)
 {
  int i;
  for(i=0;i<n;i++)
   {
    printf("%d",a[i]);
   }
```

CodeNeuraX

```
}
void bubble (int a[], int n)
 {
 int i,j,temp;
 for(i=0;i<n;i++)
  {
   for (j=(i+1); j<n; j++)
   {
    if (a[j]<a[i])
     {
     temp = a[i];
     a[i] = a[j];
     a[j] = temp;
     }
  }
}
}
void main ()
 {
  int i,j, temp;
  int a[5] = { 10, 35, 32, 13, 26 };
  int n = sizeof (a) / sizeof (a0);
  printf ("Before sorting array elements are :\n");
  print (a,n);
  bubble (a,n);
  printf ("\n after sorting array elements - \n");
  print (a,n);
}
```

CodeNeurax

output :-
Before sorting array elements are -
10 35 32 13 26
After sorting array elements are -
10 13 26 32 35.

## Bucket Sort Algorithm :-

The data items in the bucket sort are distributed in term of buckets.

Bucket sort is a sorting algorithm that seprates elements into multiple groups said to be buckets. Elements in bucket sort are first uniformly divided into groups called buckets, and then they are sorted by any other sorting algorithm. After that, elements are gathered in sorted manner.

Advantages of bucket sort are :-
- Bucket sort reduces no. of comparisons
- It is asymptotically fast because of uniform distribution of elements.

Limitations of bucket sort are :
- It may or may not be a stable sorting algorithm
- It is not useful if we have a large array bcz it increases the cost.
- It is not an in-place sorting algorithm, because more some extra space is required to sort the buckets.

Codeleaf

The best and average-case complexity of bucket sort is $O(n+k)$, worst-case complexity of bucket sort is $O(n^2)$, where n is number of items.

Bucket sort is commonly used :

- with floating-point values.
- when input is distributed uniformly over a range.

Algorithm :-

Bucket sort (A[.])

1. Let B[0 .... n-1] be a new array.
2. n = length [A].
3. for i = 0 to n-1
4. make B[i] an empty list
5. for i = 1 to n
6. do insert A[i] into list B[n a[i]]
7. for i = 0 to n-1.
8. do sort list B[i] with insertion sort.
9. concatenate lists B[0], B[1] .... B[n-1] together in order.
10. END.

Complexity :-

1. Time complexity :-

| case | time complexity |
|---|---|
| Best case | $O(n+k)$. |

CodeNaurax

| | |
|---|---|
| Average case | $O(n+k)$ |
| worst case | $O(n^2)$ |

2. space complexity :-

| | |
|---|---|
| Space complexity | $O(n*k)$ |
| stable | YES |

Implementation of bucket sort in C :-

```
#include <stdio.h>
int getmax (int a[], int n)
{
 int max = a[0];
 for (int i=1; i<n; i++)
   if (a[i] > max)
     max = a[i];
   return max;
}
void bucket (int a[], int n)
 {
 int max = getmax (a,n)
 int bucket [max], i;
 for (int i = 0; i <= max; i++)
  {
```

codeleurax

```
        bucket [i]=0 ;
    }
    for(int i=0 ; i<n ; i++)
    {
      bucket [a[i]]++;
    }
    for (int i= 0; j=0 ; i<=max ;i++)
    {
    while (bucket [i] >0)
      {
      a[j++] = i ;
      bucket [i]-- ;
      }
}
}
Void printArr (int a[], int n)
  {
    for (int i=0; i<n ; i++)
    printf("%d", a[i]);
  }
int main ()
{
  int a[] = {54, 12, 84, 57, 69, 41, 9, 5};
  int n = size of(a) / size of a[0] ;
  printf ("Before sorting array elements are:-\n");
  printArr(a,n);
   bucket (a,n);
  printf ("\n After sorting array elements are:-\n");
   printArr(a,n);
  }
```

CodeNeuraX

output :-
Before sorting array elements are :-
54 12 84 57 69 41 9 5
After sorting array elements are :-
5 9 12 41 54 57 69 84.

## Heap Sort Algorithm :-

Heap sort processes the elements by creating min-heap or max-heap using the elements of the given array.

Heap sort basically recursively performs two main operations :

- Build a heap H, using the element of array.
- Repeatedly delete the root element of heap formed in 1st phase.

What is heap ?

A heap is a complete binary tree, and binary tree is a tree in which node can have utmost two children.

Algorithm :-

```
Heap Sort (arr)
BuildMaxHeap(arr)
for i = length (arr) to 2
   swap arr [i] with arr [i]
    heap-size [arr] = heap-size [arr] ? 1
    MaxHeapify (arr, 1)
End.
```

codeLeurax

BuildMaxHeap(arr):
  BuildMaxHeap(arr)
    heap-size(arr) = length(arr)
    for i = length(arr)/2 to 1.
  MaxHeapify(arr, i)
End.

Complexity :-

| Case | Time complexity | Space complexity |
|---|---|---|
| Best | O(n log n) | O(1). |
| Average | O(n log n) | |
| worst | O(n log n) | |

Implementation of Heap sort :-

```
#Include<stdio.h>
/* function to heapify a subtree. Here is 'i' the
index of root node in array a[], and 'n' is size
of heap */
void heapify(int a[], int n, int i)
 {
  int largest = i
  int left = 2*i+1
  int right = 2*i+2
  if(left<n && a[left]> a[largest]
    largest = left;
```

CodeVarsity

```
if (right<n && a[right]>a[largest]
  largest = right ;
if (largest != i)
{ int temp = a[i] ;
  a[i] = a[largest];
  a[largest] = temp;
  heapify (a, n, largest);
}
}
void heapsort (int a[], int n)
{
  for(int i = n/2-1 ; i >=0, i--)
    heapify (a, n, i);
  for (int i= n-1; i >=0 ; i--)
  { int temp = a[0];
    a[0] = a[i];
    a[i] = temp;
  heapify (a, i, 0);
}
} void printArr (int arr[], int n).
{
  for(int i= 0; i<n ; ++i)
    {
      printf ("%d ", arr[i]);
      printf (" ");
    }
} int main ()
{
 int a[] = { 48, 10, 23, 43, 28, 26, 1};
 int n = sizeof (a) / sizeof (a[0]);
 printf ("Before sorting array elements are - \n");
```

CodeNeuraX

```
printArr(a,n);
heapSort(a,n);
printf("\n After sorting array elements are -\n");
printArr(a,n);
return 0;
}
```

output: Before sorting array elements are :-
48 10 23 43 28 26 1
After sorting array elements are -
1 10 23 26 28 43 48.

## Insertion sort Algorithm :-

Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card. The idea behind the insertion sort is that first make/take one element, iterate it through sorted array. complexity of insertion sort in the average case and worst case is $O(n^2)$, where n is number of items.

Insertion sort is less efficient than the other sorting algorithms like heap sort, quick sort and merge sort etc.

Insertion sort has various advantages such as :

- simple implementation.
- Efficient for small data sets.
- Adaptive i.e it is appropriate for data sets that are already substantially sorted.

CodeMurax

complexity :-
case time complexity :
Best case o(n)
Avergge case $o(n^2)$
worst case $o(n^2)$.

space complexity o(1).

Implementation of insertion sort :-

```
#include <stdio.h>
void insert (int a[], int n)
{
  int i, j, temp;
  for (i=1; i<n, i++) {
    temp = a[i];
    j = i-1;
   while (j>=0 && temp <= a[j])
   {
    a[j+1] = a[j];
    j = j-1;
   }
   a[j+1] = temp;
 }
}
void printArr (int a[], int n)
 {
   int i;
   for(i=0; i<n; i++)
    printf ("%d", a[i]);
 }
int main()
{
  int a[] = { 12, 31, 25, 8, 32, 17};
```

CodeNeuraX

```
int n = sizeof (a) / sizeof a[0];
printf ("Before sorting array elements are -\n");
printArr (a, n);
insert (a, n);
printf ("\nAfter sorting array elements are -\n");
printArr(a, n);
return 0;
}
```

output: Before sorting array elements are -
12 31 25 8 32 17
After sorting array elements are -
8 12 17 25 31 32.

## Merge Sort Algorithm :-

Merge sort is the sorting technique that follows divide and conquer approach. This will be very helpful and interesting.

Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort elements.

Algorithm :-

arr is given array, beg is starting element and end is last element of array.

```
MERGE-SORT (arr, beg, end)
if beg < end
Set mid = (beg + end)/2.
MERGE-SORT (arr, beg, mid)
MERGE-SORT (arr, mid+1, end)
MERGE (arr, beg, mid, end)
```

end of if
End MERGE_SORT.

implementation of merge sort:-

```
/* function of merge the subarrays of a[] */
void merge (int a[], int beg, int mid, int end)
{
  int i, j, k;
  int n1 = mid - beg + 1;
  int n2 = end - mid;
  int LeftArray [n1], RightArray [n2];
  /* copy data to temp arrays */
  for(int i=0; i<n1; i++)
  LeftArray [i] = a[beg + i];
  for (int j=0; j<n2; j++)
  RightArray[j] = a[mid+1+j];
  i=0;
  j=0;
  k=beg;
while(i<n1 && j<n2)
{
  if(LeftArray[i]<=RightArray[j])
  {
    a[k] = LeftArray[i];
    i++;
  }
else
{
a[k]=RightArray[j];
j++;
}
```

```
    }
    k++;
    }
    while (i<n1)
    {
    a[k] = LeftArray[i];
     i++;
     k++;
    }
    while (j<n2)
    {
     a[k] = RightArray[j];
     j++;
     k++;
    }
}
```

Complexity :-

| case | Time complexity | space complexity |
|---|---|---|
| Best case | $o(n * \log n)$ | $o(n)$. |
| Avg. case | $o(n * \log n)$ | |
| worst case | $o(n * \log n)$ | |

CodeNeuraX

# DATA STRUCTURE CODING QUE.

Arrays by using c :

1). program to demonstrate arrays in c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
# define NUM_EMPLOYEE 10
int main (int argc, char *argv[])
{ int salary [NUM_EMPLOYEE], lcount=0,
    gcount=0 , i=0;
  printf ("Enter employee salary (MAX 10)\n");
     for (i=0; i<NUM_EMPLOYEE ; i++)
  {
    printf ("\n Enter employee salary : %od -",
          i+1) ;
    scanf ("%od ", & salary [i]);
  }
     for (i=0; i<NUM_EMPLOYEE ; i++)
  {  if (salary [i] <3000)
     lcount ++ ;
    else
      gcount ++;
  }
  printf ("\n There are {%od} employee with
        salary more than 3000 \n", gcount);
  printf ("There are {%od} employee with salary
        less than 3000 \n", lcount);
  printf ("press Enter to continue ..\n");
```

```
    getchar ();
    return 0;
}
```

2. Linked list in c++ :

```
using namespace std;
template <typename T>
class node
{
  public :
   T value;
   Node *next;
   Node *previous;
   Node (T value)
   {
     this->value = value;
   }
};
  template <typename T>
  class linked list
  {
    private :
    int size;
    Node<T> *head = NULL;
    Node<T> *tail = NULL;
    Node<T> *itr = NULL;
    public;
    Linked list()
    {
     this->size = 0;
    }
```

CodeNeurax

```
void append (T value)
{
 if (this->head == NULL)
{
 this->head - = new Node <T> (value);
 this->tail = this->head - ;
 }
else
 {
this - > tail_-> next = new node <T> (value)
this -> tail -> next -> previous = this_
                                  ->tail ;

 }
this->size - + = 1 ;
 }
void prepend (T value)
 {
void resetIterator ()
 {
 tail - = NULL ;
 }
int main (int argc, char **argv)
 {
    LinkedList <int> llist ;
    llist. append (10);
    llist. append (3);
    llist. append (1);
  cout << "printing linked list <<end l;
  cout<<end l ;
  return 0;
 }
```

CodeNeurax

37. stack implementation in c :

```
#include <stdio.h>
int MAXSIZE = 8;
int stack [8];
int top = -1;
int isempty() {
   if (top == -1)
      return 1;
   else
      return 0;
   }
int isfull() {
   if (top == MAXSIZE)
      return 1;
   else
      return 0; }
int peek() {
   return stack [top]; }
int pop() {
   int data;
   if(!isempty()) {
      data = stack [top];
      top = top-1;
      return data; }
   else {
      printf("could not retriere data, stack is empty\n");
   }
}
int push (int data) {
   if (!isfull()) {
      top = top+1;
```

CodeNeuraX

```
    stack[top] = data;
  } else {
    printf("could not insert data, stack is full\n");
  }
}
int main() {
  // push items on to the stack
  push(3);
  push(5);
  push(9);
  push(1);
  push(12);
  push(15);
  printf("Element at top of the stack : %d\n", peek());
  printf("Elements : \n");
  // print stack data
  while (!isempty()) {
    int data = pop();
    printf("%d\n", data);
  }
printf("stack full : %s\n", isfull() ? "true" : "false");
printf("stack empty : %s\n", isempty() "true", "false");
return 0;
}
```

# DATA STRUCTURES INTERVIEW QUESTIONS WITH ANSWERS

Q.1. What is data structure ?

→ A data structure is a way of organizing data that considers not only items stored, but also their relationship to each other.

Q.2. List out the areas in which data structure are applied extensively ?

→ • compiler design , • operating system,
• database system , • statistical analysis,
• numerical analysis , • artificial intelligence

Q.3. What are major data structures used in following areas RDBMS, network data model and Hierarchical data model.

→ RDBMS = array (array of structures).
network data model = graph.
Hierarchical data model = tree.

Q.4. If you are using C language to implement the heterogeneous linked list, what pointer type will you use ?

→ The heterogeneous linked list contains different data types in its nodes and we need a link, pointer to connect them. It is not possible to use ordinary pointer for this. so we go for void pointer. Void pointer is capable of storing pointer to any

CodeNeural

type as it is a generic pointer type.

Q.5. minimum number of queues needed to implement the priority queue?

→ two. one queue is used for actual storing of data and another for storing priorities.

Q.6. What is data structure used to perform recursion?

→ stack. because of its LIFO (Last IN first Out) property it remembers its 'caller'

Q.7. What are notations used in evaluation of arithmatic expressions using prefix & postfix forms?

→ Polish and Reverse polish notations.

Q.8. convert expression ((a+b)*c-(d-e)^(f+g)) to equivalent prefix and postfix notations.

→ prefix notation : - * + abc ^ - de + fg
postfix notation : ab+c*de-fg+^-

Q.9. what are methods available in storing sequential files?

→ 1. straight merging,
2. natural merging,
3. polyphase sort,
4. distribution of initial runs.

Q.10. Whether linked list is a linear or non-linear data structure?

CodeNeural

→ According to access strategies linked list is a linear one. according to storage linked list is a non linear one.

Q.11. define doubly linked list.

→ It is collection of data elements called nodes, where each node is divided into three parts:
- an info field that contains information stored in the node.
- left field that contain pointer to node on left side.
- Right field that contain pointer to node on right side.

Q.12. What are the issues that hampers efficiency in sorting a file?

→
- length of time required by programmer in coding a particular sorting program.
- amount of machine time necessary for running the particular program.
- amount of space necessary for particular pgm.
- object oriented analysis and design.

Q.13. calculate efficiency of sequential search?

→ The number of comparisons depends on where the record with argument key appears in table
- If it appears at first position then one comparison.
- If it appears at last position then n comparison.
- average = $\frac{n+1}{2}$ comparisons.

CodeNeuray

• number of comparisons in any case is O(n).

Q.14. Is any implicit arguments are passed to a function when it is called ?

→ yes, there is a set of implicit arguments that contain information necessary for function to execute and return correctly, one of them is return address which is stored within the function's data area, at time of returning to calling program address is retrived and function branches to that location.

Q.15. Paranthesis is never required in postfix or prefix expressions ? Why

→ parenthesis is not required because order of the operators in postfix / prefix expressions determines actual order of operations in evaluating expression.

Q.16. List out few of applications of tree data structure ?

→ The manipulation of arithmatic expression, symbol table construction & syntax analysis.

Q.17. List out few of applications that make use of multilinked structures ?

→ sparse matrix, Index generation.

Q.18. What is type of the algorithm used in solving 8 queens problem ?

Codingpurav

→ backtracking.

Q.19. In an AVL Tree, at what condition balancing is to be done?

→ If 'pivotal value' or height factor is greater than 1 or less than -1.

Q.20. In Rdbms, what is the efficient data structure in internal storage representation.

→ b+tree. because bt tree, all the data is stored in only in leaf nodes, that makes searching easier. this corresponds to records that shall be stored in leaf nodes.

Q.21. What is difference between array and a stack?

→ Stack follows LIFO. thus the item that is first entered would be last to be removed.

In the array, items can be entered or removed by in any order. basically, each member access is done using index. no strict order is to be followed here to remove a particular element.

Q.22. How to check whether a linked list is circular?

→ create two pointers, each set to start of list.
update each as follows:

```
while (pointer 1)
{
  pointer 1 = pointer 1 -> next;
  pointer 2 = pointer 2 -> next;
  if (pointer 2) pointer 2 = pointer 2 -> next;
```

```
if (pointer 1 == pointer 2)
{
print("circularn");
}
}
```

Q.23. What is a node class?

→ A node class is class that, relies on the base for service and implementation, provides a wider interface to users than its base class, relies primarily on virtual functions in its public interface depends on all its direct and indirect base class.

Q.24. When can you tell that a memory leak will occur?

→ a memory leak occurs when a program loses the ability to free a block of dynamically allocated memory.

Q.25. What are types of collision Resolution techniques and methods used in each of the type?

→ open addressing (closed hashing), methods used include: overflow block. closed addressing (open hashing) methods used include: linked list, binary tree

Q.26. Which is simplest file structure? (sequential, index, random).

→ Sequential is the simplest file structure.

Q.27. What are the notations, used in evaluation of arithmatic expression, using prefix and postfix forms?
→ Polish and Reverse polish notations.

Q.28. list out few of applications of tree data structure?
→ The manipulation of arithmatic expressions, symbol table construction and syntax analysis.

Q.29. difference between calloc and malloc?
→ malloc : allocate n bytes.
calloc : allocate m times n bytes initialized to 0.

Q.30. Which file contains the definition of member function
→ defination of member function for the linked list class are contained in linkedlist.cpp file.

Q.31. How is the front of the queue calculated?
→ The front of the queue is calculated by
front = (front + 1) % size.

Q.32. Why is the Isempty() member method called?
→ the isempty() member method is called within the dequeue process to determine if there is an item in dequeue to be removed i.e. isempty() is called to decide whether queue has atleast one element. This method is called by dequeue() method before returning front element.

Q.33. Which process places data at back of queue?

codeVeurax

→ enque is a process that places data at back of the queue.

Q.34. What is queue ?

→ A queue is sequential organization of data. a queue is a first in first out type of date structure. an element is inserted at last position and an element is always taken out from first position.

Q.35. What does isempty() member method determine

→ isempty() checks if stack has at least one element. this method is called by pop() before retrieving and returning top element.

Q.36. What method removes value from top of a stack ?

→ The pop() member method removes value from top of a stack, which is then returned by the pop() member method to statement that calls Pop() member method.

Q.37. What method is used to place a value onto the top of a stack ?

→ push() method, push is the direction that data is being added to stack. push() member method places a value onto the top of a stack.

Q.38. How do you assign an address to an element

of a pointer array ?

→ We can assign a memory address to an element of a pointer array by using the address operator, which is ampersand (&), in an assignment statement such as ptemployee[0] = &projects[2];

Q.39. How many parts are there in a declaration statement ?

→ There are two main parts, variable, identifier & data type and third type is optional which is type qualifier like signed / unsigned.

Q.40. list some of the static data structures in C ?

→ Some of the static data structures in C are arrays, pointers, structures etc.

Q.41. define dynamic data structure ?

→ A data structure formed when number of data items are not known in advance is known as dynamic data structure or variable size data structure.

Q.42. list some of dynamic data structures in C ?

→ some of dynamic data structures in C are linked lists, stack, queues, trees etc.

Q.43. define linear data structure.

→ linear data structures are data structures having a linear relationship between its adjacent elements.
eg : linked list.

Codelevural

Q.44. define nonlinear data structures.

→ Non linear data structure are the data structures are data structure that don't have a linear relationship between its adjacent elements but have a hierarchical relationship between the elements.
eg : trees and graphs.

Q.45. state the different types of linked lists?

→ The different types of linked list include singly linked list, doubly linked list and circular linked list.

Q.46. List the basic operations carried out in a linked list?

→
- creation of a list
- Insertion of a list.
- deletion of a node.
- modification of a node.
- traversal of a node.

Q.47. define a stack.

→ Stack is an ordered collection of an elements in which insertion and deletions are restricted to one end. The end from which elements are added and or removed is referred as top of stack.

Q.48. List out the basic operations that can be performed on a stack.

→
- push operation.

- pop operation
- peek operation
- empty check
- fully occupied check.

Q.49. State the different ways of representing expression

→
- Infix notation.
- prefix notation
- postfix notation.

Q.50. What is sequential search?

→ In sequential search each item in the array is compared with the item being searched until a match occurs.

CodeNeuraX

classmate
Date
Page

# THANK YOU !!!

keep following us on our

Social Media Channels

for More Helpful Contents !

Codeleura

# OOPs in C++

The main aim of OOP is to bind together the data and the functions that operate on them so that no other part of the code can access this data except this function.

**Class :** It is a user defined data types, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class.

**Object :** When a class is defined no memory is allocated but when it is instantiated (i.e., object is created) memory is allocated.

**Encapsulation :** In OOP, Encapsulation is defined as binding together the data and the functions that manipulates them

**Abstraction :** Abstraction means displaying only essential information and hiding the details.

- Abstraction using classes
- Abstraction using Header files (math.h → pow())

**Polymorphism :** In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

- Operator overloading
- Function overloading
  - ↳ int sum(10, 20, 30)
    int sum(10, 20)

Inheritance : The capability of a class to derive properties and characteristics from another class is called Inheritance.

- SubClass
- Superclass
- Reusability

Dynamic Binding : In dynamic binding, the code to be executed in response to function call is decided at run time.

Constructors : A constructor is a member function of a class which initializes objects of a class. In C++ constructor is automatically called when the object creates.

It has same name as class itself.

Constructor don't have a return type.

1. Default Constructor (No parameter passed)
2. Parametrized Constructor
3. Copy Constructor

Destructor in C++ : Derived class destructor will be invoked first, then the base class destructor will be invoked.

Access Modifier : Public - can be accessed by any class.

Private :- can be accessed only by a function in a class (inaccesible outside the class).

Protected :- It is also inaccessible outside the class but can be accessed by subclass of that class.

Note: If we do not specify any access modifier inside the class then by default the access modifier for the member will be private.

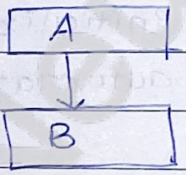
Friend class: A friend class can access private and protected members of other class in which it is declared as friend.

Ex-: friend class B;

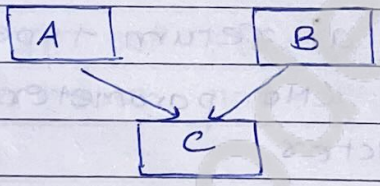
- Inheritance

```
class subclass : accessmode baseclass
{
}
```

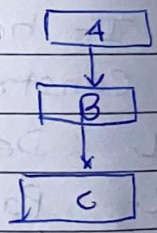
1. Single inheritance



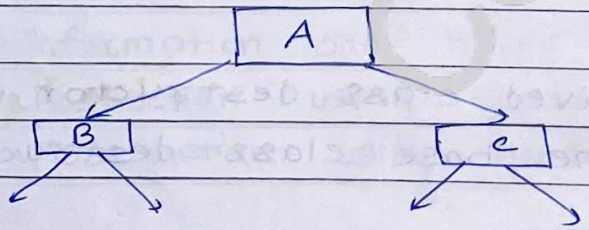

2. Multiple inheritance




3. Multilevel




4. Heerarchical inheritance




5. Hybrid

Combination of one or more type.

- Polymorphism
  - -> Compile time Poly → Operator overloading, Function overloading.
  - -> Run time Poly
    - ↳ function overriding occurs when a derived class has a definition of one or more members of base class.

## Advantages of Data Abstraction

- Avoid code duplication and inc. reusability.
- can change internal implementation of class independently.

**Structure Vs class :** Most important difference is security.
A structure is not secure and cannot hide its member function and variable while class is secure and can hide its programming & designing details.

**Local Classes in C++ :** A class declared inside a function becomes local to that function and is called local class.
All the methods of local class must be defined inside the class only.

**Virtual Function and Runtime Polymorphism :**
A virtual function is a member function which is declared within a base class and redefined (overriden) by derived class.
Functions are declared with Virtual Keyword in base class.

**Exception Handling in C++ :**
try : represent a block of code that can throw an exception.
catch : represent a block of code that get executed when error is thrown
throw : Used to throw an exception.

There is a special catch block → catch(...)
It catches all types of error.

- Inline Function

→ inline is a request not command.
It is function that is expanded in line when it is called. When the inline function is called, whole code get inserted or substituted at the point of inline function call.

```
inline return-type fun( )
{
}
```

- Function Overloading is a feature in C++ where two or more functions can have same name but different parameters.

```
void print(int i)
{
    cout << "Here is int " << i << endl;
}

void print (float i)
{
    cout << " Here is float" << i << endl;
}

int main
{
    print(10);
    print(10.12);
}
```

Differences b/w C and C++

| | C | C++ |
|---|---|---|
| 1. | C supports procedural prog. | • C++ is known as hybrid language, because it support both procedural and object oriented programming. |
| 2. | As C does not support the OOPs concept so it has no support for polymorphism, encapsulation and inheritance | • C++ has support for polymorphism, encapsulation and inheritance as it is an OOPs language. |
| 3. | C is a subset of C++ | • C++ is superset of C |
| 4. | C contains 32 keywords | • C++ contain 52 keywords (public, private, protected, try, catch, throw ...) |
| 5. | C is a function driven language | • C++ is an object driven language. |
| 6. | Function and operator overloading is not support in C. | • C++ supports function & operator overloading. |
| 7. | C does not support exception handling | • C++ supports exception handing using try and catch |

- Structure is a collection of dissimilar elements

- Static Members in C++

  - Static variable in a function : When a variable is declared as static, space for it gets allocated for the lifetime of the program. (defaut intialized to 0)
  Even if the function is called multiple times, the space for it is allocated once.

  - Static variable in a class :
    → Declared inside the class body.
  → Also known as class member variable.
  → They must be defined outside the class.
  → Static variable doesn't belong to any object, but to the whole class.
  → There will be only one copy of static member variable for the whole class.

Ex:
```
class Account
{
    private :
        int balance;
        static float roi;
    public :
        void setBalance(int b)
        {   balance = b; }
};
// intialised outside class
float Account :: roi = 3.5f;
void main
{
    Account a1;
}
```

- Object can also be declared as static.

```
static Account a1;
```

- Static function in a Class

Static member functions are allowed to access only the static data members or other static member functions.

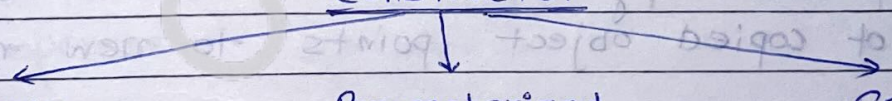
- Constructors :

→ Constructors is an special member function of the class. It is automatically invoked when an object is created.

→ It has no return type.

→ Constructor has same name as class itself.

→ If we do not specify, then c++ compiler generates a default constructor for us.




| Default | Parameterized | Copy |
|---|---|---|
| Class_name(); | class_name(parameters) | class_name(const Class_name &obj); |
| update() | update(int x, int y) | update(const update &p2) |
| { | { | { a=p2.a; |
| a=10; | a=x; | b=p2.b; |
| b=20; | b=y; | } |
| } | } | |

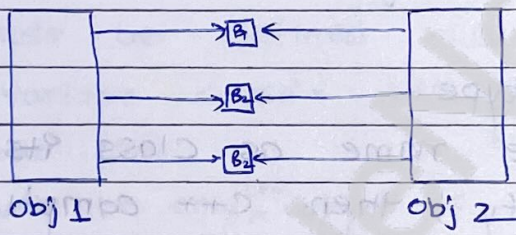
Compiler generates two constructor by itself.

1. Default Constructor
2. Copy Constructor

But if any of the constructor is created by user, then default constructor will not be created by compiler.

Construction overloading can be done just like function overloading.

Default (Compiler's) Copy constructor can done only shallow copy.



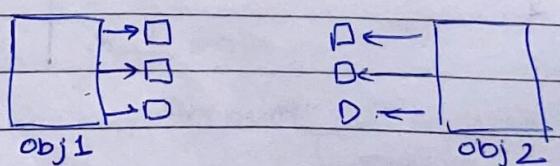

(Points to same memory location)

Deep Copy is possible only with user defined constructors. In user defined copy constructor, we make sure that pointers of copied object points to new memory location.

Can we make Copy Constructor private? Yes

Why argument to copy constructor must be passed as a reference?

Because if we pass value, then it would made to call copy constructor which becomes non-terminating.




Deep Copy

- Destructor

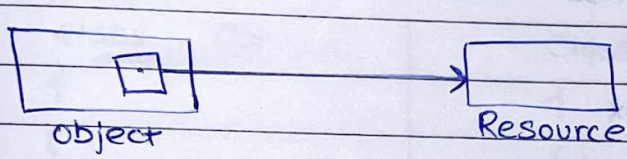
→ Destructor is a member function which destructs or deletes an object.

→ Destructor don't take any argument and don't have any return type.

→ Only one destructor is possible

→ Destructor cannot be static.

→ Actually destructor doesn't destroy object, it is the last function that invoked before object destroy.




Destructor is used, so that before deletion of obj we can free space allocated for this resource. B/c if obj gets deleted then space allocated for obj will be free but resource doesn't.

- Operator Overloading

C++ have the ability to provide special meaning to the operator.

```
class Complex
{
    ...
    ...

    Complex operator + (Complex &c1)
    {
        Complex res;
        res.a = c1.a;
        res.b = c2.b;
    }
};

int main()
{
    c = c1 + c2
}
```

As '+' can't add complex no's directly. So we can define a function with name + but we need write operator keyword before it.
So, we can use all operator like this.

## Friend Class

A friend class can access the private and protected members of other class in which it is declared as friend.
There can be friend class and friend function.

Ex:

```
class Box
{   private :
        double width ;
    public :
        friend void printWidth(Box box);
        void setWidth(double Wid);
};

void Box :: setWidth (Box double Wid)
{       width = Wid;    }

void printWidth(Box box)
{   cout << box.width ;  }

int main()
{   Box box;
    box.setWidth(14);
    printWidth(box);
}
```

# Inheritance

It is a process of inherting properties and behaviour of existing class into a new class.

| Class Base_class | class der_class : Visibility_Mode Base class |
|---|---|
| { | { |
| }; | }; |

Ex:

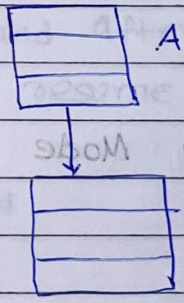
| class Car | class Sports_Car : public Car |
|---|---|
| { | { |
| }; | }; |

Types of Inheritance :

a). Single Inheritance :

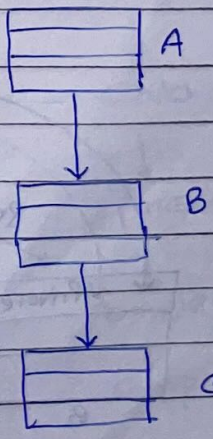
```
class B : public A
{

};
```

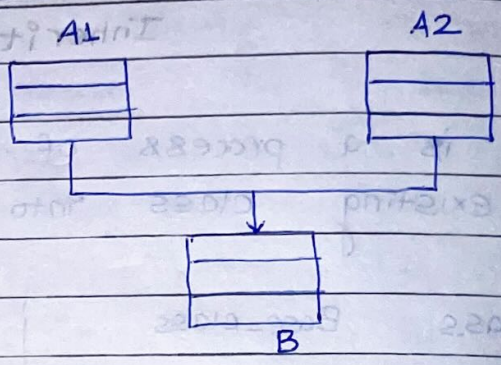



b). Multilevel Inheritance :

```
Class B : public A
{
};
class C : public B
{
};
```

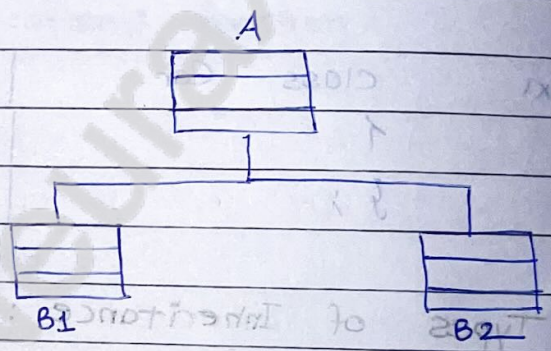


c). Multiple Inheritance



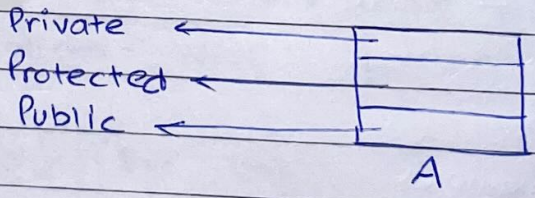

```
class A1          class A2
{                 {
};                };
  class B : public A1, public A2
  {
  };
```

d). Heirarchial Inheritance



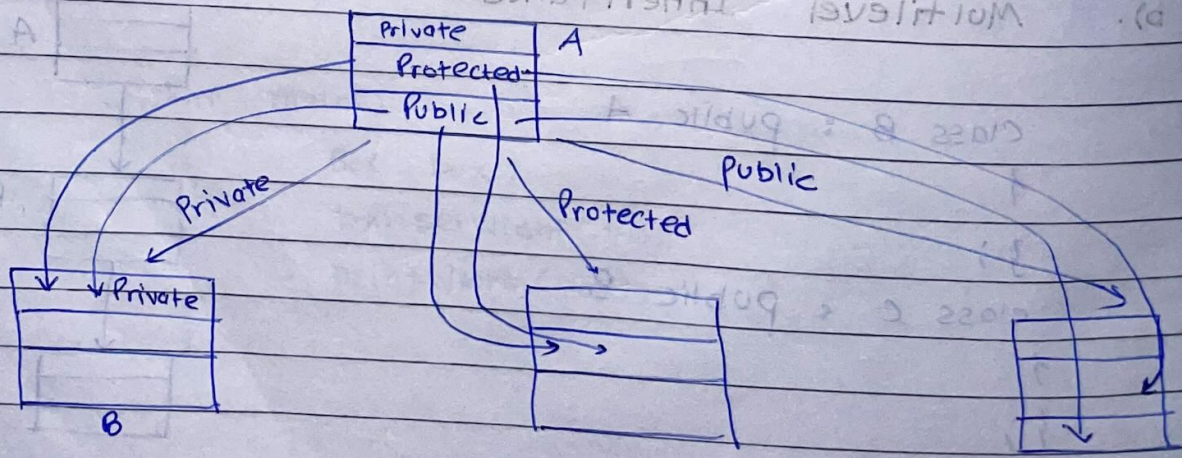

```
class B1 : public A
{
};
class B2 : public A
{
};
```

→ Visibility Mode :




A - base class

B - Sub class



If B is subclass and visibility mode is public.

```
class A : public B
{
};
```

then public member of A will be public in B, and protected will protected.

If visibility mode is private then both protected and public member of A will be private member of B

- Is a Relationship is always implemented as a public inheritance.

- Constructor and Destructor in Inheritance

First child class constructor will run during creation of object of child class, but as soon as obj is created child class constructor run and it will call constructor of it's parent class and after the execution of parent class constructor it will resume it constructor execution.

Child → B() : A() → parent constructor call

```
{
{
```

constructor exec,
obj
↓
Child const
↓
parent const
↓
complete parent
↓
complete child

While in case of destructors, first child destructor exec, then parent desc. executed.

## this pointer

Every object in C++ has access to its own address through an important pointer called this pointer.

Friend function doesn't have a 'this' pointer, b/c friends are not members of a class. Only member function have this pointer.

```
Class Box
{   private:
        int l, b, h;
    public:
        void set (int l, int b, int h)
        {   this->l = l;
            this->b = b;
            this->h = h;
        }
};
int main ()
{   Box b;
    b.set (5, 10, 4);
}
```

## Method Over Riding
(achieved at run time)

It is the redefinition of base class function in its derived class, with same return type and same parameters.

While method Overloading is achieved at Compile time.

Ex:

```
Class Car
{
    private:
        int gearno;
    public:
        void change_gear(int gear)
        {
            gear++;
        }
};

Class SportsCar : public Car
{
    void change_gear(int gear)
    {
        if(gear > 5)
            gear++;
    }
};

int main
{
    SportsCar sc;
    sc.change_gear(4);
}
```

function of sportsCar Class will be called.

While calling change_gear(), first it check if any fun with this name exist in calling class, otherwise it goes to baseClass.

Useful: like we have change_gear for all except one car which have unique method of gearChange.

# Virtual Function

A virtual function is a member function which is declared with a 'virtual keyword' in the base class and redeclared (overridden) in a derived class. When you refer to a object of derived class using pointer to a base class, you can call a virtual function of that object and execute the derived class's version of the function.

- They are used to achieve Run time Polymorphism.
- Virtual Function cannot be static and also cannot be friend function of another class.

## Compile-time (Early binding) Vs Run-time (Late Binding)

```
class base
{       public:
            virtual void print()
            {       cout<< " This is base print" << endl;
            }
            void show()
            {       Cout << " Base show fun" << endl;
            }
};
class derived
{       public:
            void print()
            {cout << " derived Print" <<endl; }
            void show()
            { cout << " derived show fun" << endl; }
};
```

```
int main()
{
    base *bptr;
    derived der;
    bptr = &der;

    bptr -> print();      // Run time
    bptr -> show();       // Compile time
}
```

Output: derived print // Late Binding
This Base show fun // Early binding

As during compiler time bptr behaviour judged on the basis of which class it belong, so bptr represent base class.

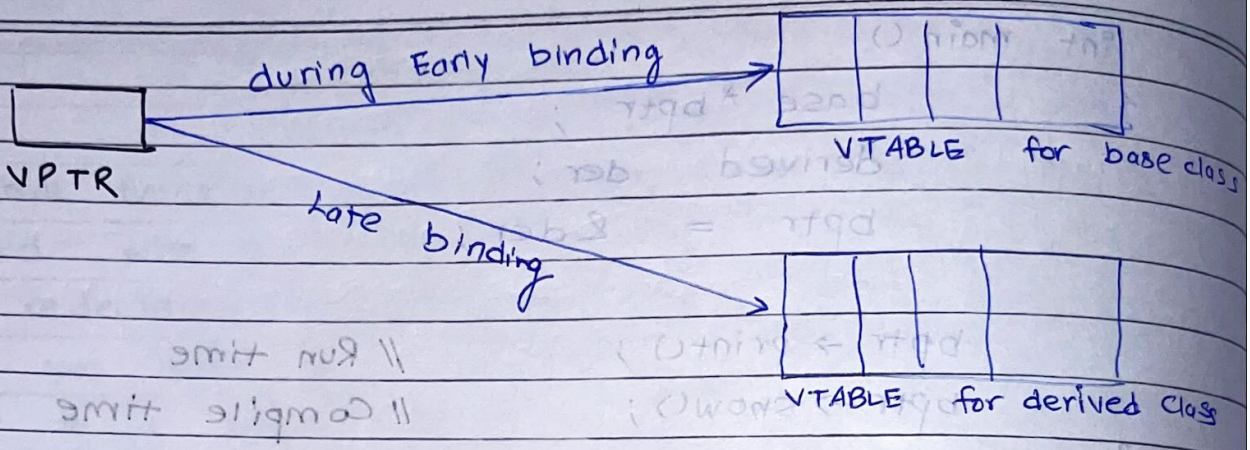
If the function is not virtual then it will allow binding at compile time and print fun of base class will get binded b/c bptr represent base class.

But at run time bptr points to the object of class derived, so it will bind function of derived at run time.

## Working of Virtual Function (VTable & VPtr)

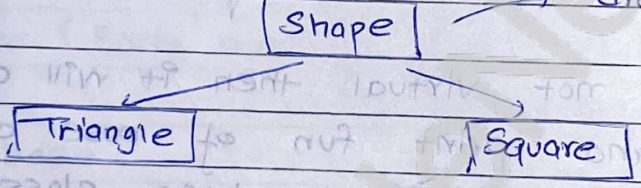
If a class contains virtual function then compiler itself does two things:

1. A virtual pointer (VPTR) is created every time obj is created for that class which contains virtual function.
2. Irrespective of object is created or not, static array of pointer called VTABLE where each cell point to each virtual function is created, in base class and derived class.




## Pure Virtual Function and abstract Class

Sometimes implementation of all function cannot be provide in the base class. Such a class is called abstract class.




A pure virtual function in C++ is a virtual function for which we don't have any implementation, we only declare it.

```
// Abstract Class
class Test
{
    public:
    virtual void fun() = 0;
};
```

Pure Virtual function

1. A class is abstract if it has at least one pure virtual function.

We cannot declare object of abstract class.
Ex: Test t; will show error

2. We can have pointer or reference of abstrac class.
3. We can access the other functions except virtual by object of its derived class.
4. If we dont override the pure virtual function in derived class then it becomes abstract.
5. An abstract class can have constructors.

(Read from GfG)

Template in C++

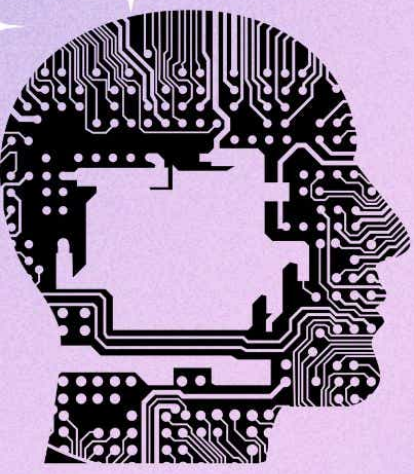
```
template <class X> int check (int a, X b)
                   X          X
{
    if (a>b)
        return a;
    else return b;
}
```

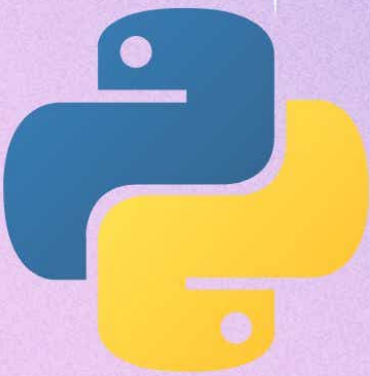
It ~~does~~ just help in data type. So that we can write generic function that can be used for different data type.

Dynamic Constructor

When allocation of memory is done dynamically using dynamic memory allocator 'new' in constructor.

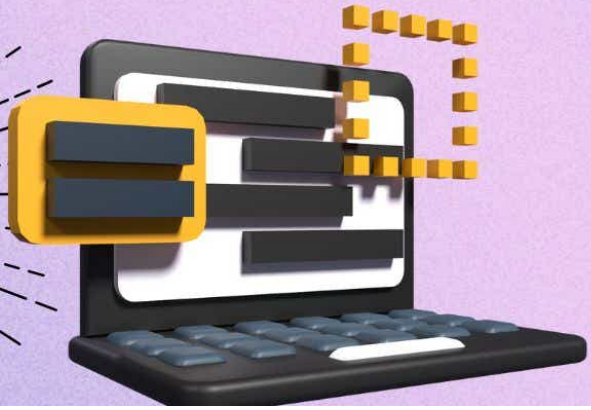
```
class geeks
{   public:
        void fun()  {  p= new char[6] ; }
};
int main()
{   geeks  g = new geeks();
}
```

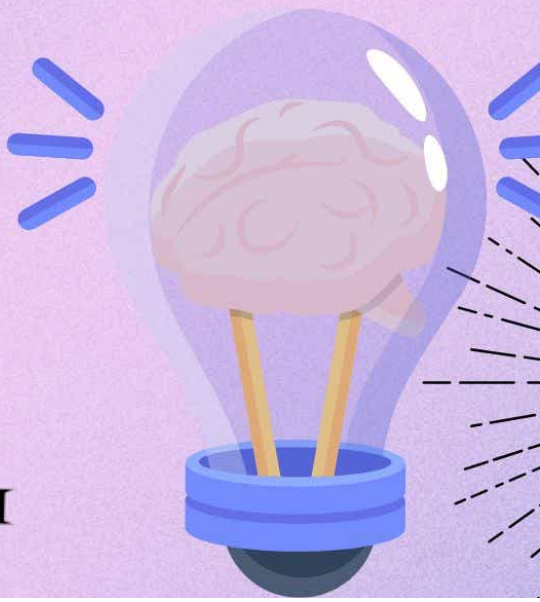




# COMPLETE PYTHON PROGRAMMING NOTES

CodeNera





HIMANI SAINI

Date .....

# PYTHON PROGRAMMING

## * Introducing Python -

→ Python is a high-level, general-purpose, and very popular programming language.
→ It is being used in web development, Machine learning applications, etc.
→ Python programs are generally smaller than the other programming languages.
→ Programmers have to type relatively less and the indentation requirement of the language, makes them readable at the time.

→ Very Simple Syntax. & Easy to learn.

```
print("Hello")
```

→ General purpose language (simple yet very powerful language)
  - → Console Application & Scripts
  - → Desktop Application
  - → Web Application
  - → Game Development
  - → Machine Learning, Deep learning, AI, Big data etc

→ Multiparadigm Support
  - → Procedural Style Programming like C.
  - → OOPs like Java.
  - → Functional programming like list.

Spiral

Date .....

→ Portable or Platform Independent
- Programs are typically first compiled into an intermediate code, then the code is run by the interpreter.

→ Dynamically Typed

```
x = 10
y = "geeks"
x = "python"
```

This feature is different from C++ & Java. They are statically Typed.
In python we don't have to specify the variable name.
But the disadvantage is in this the type is assigned in the run time of the program, so there are more chances of run time error.

→ Automatic Garbage Collection
It means when you dynamically allocate memory so you don't have to worry to realising this memory.

→ Popular Application Built in Python
Youtube, Netflix, Quora, Instagram, DropBox

=> print() in Python -
A function is a set of instructions that take some input from us, do some work on those

Date .....

parameters and produce some output.

Example - print ("Hello")
print ("Welcome", "to", "GfG")
print ()
print ("Hope you are enjoying Python")

Output - Hello
Welcome to GfG

Hope you are enjoying Python

→ ~~end~~ end and sep parameters in print()

Eg - print ("Welcome", end = " ")
print (" to GfG")

Output - Welcome to GfG

End parameter creates space and it tells what should be printing after its printing.
And in sep parameter when we have multiple input parameters we use sep to separate them. By default they are separated by space.

Eg - print ("10", "04", "2023", sep = "-")

Output - 10-04-2023

Codeslibrary

Date .....

## ⇒ Variables in Python -

Variable is a name that is used to refer to memory location. Python variable is also known as an identifier and used to hold value.

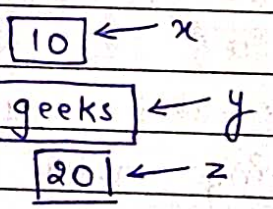
In python, we don't need to specify the type of variable because Python is a infer language and smart enough to get variable type.

Example -
```
price = 100
tax = 18
total-price = price + tax
print (total-price)
```

Output - 118

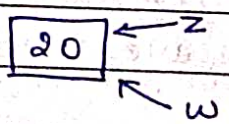
### How variables work ?

```
x = 10
y = "geeks"
z = 20
```

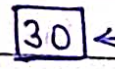



if we say, w = z

then




but if, w = 30




```
print (x)
print (y)
print (z)
print (w)
```

O/P - 10
geeks
20
30

CodeNeural

Date .....

Python is Dynamically Typed

```
x = 10
print(x)
x = "geeks"
print(x)
```

O/P - 10
geeks

NOTE - Using a variable without assigning it cause error.

Eg- print(x)
O/P - NameError: name 'x' is not assigned

=> input() in Python -

This function is used to take input from the user

Example -
```
name = input("Enter the name = ")
print("Welcome" + name)
```

O/P - Enter the name = Himani
Welcome Himani

Example - # Python program for addition

```
x = input("Enter First Number :")
y = input("Enter Second Number :")
x = int(x)
y = int(y)
res = x + y
print("Sum is", res)
```

CodeNaurax

Date .....

Output - Enter First Number = 10
Enter Second Number = 20
Sum is 30

NOTE - input() always gives us a string so whenever we need mathematical expressions we have to convert it from string.

=> Type() in Python -
Type is a built-in function that tells you data type of a variable or a value.

Eg -
```
a = 10
print (type(a))
b = 10.5
print (type(b))
c = 2+3j
print (type(c))
```
O/P - <class, 'int'>
<class, 'float'>
<class, 'complex'>

- In python we use "None" to indicate that we don't know the value yet or we didn't assign any value to this variable.

In python there is no char type, if you want char you should create single char string. (String is a sequence of char)

Eg -
```
str = "gfg"
print (type(str))
J = [10, 20, 30]
print (type(J))
```
(array and dynamically add and remove items. Items are stored in continuous locations)

CodeNeural

Date .....

```
t = (10, 20, 30)
print (type(t))
```

(It is also like a list, the difference is you can't modify once you created a tuple. These are immutable)

```
s = {10, 20, 30}
print (type(s))
```

(Set is a collection of items where all items are distinct and set is like mathematical set)

```
d = {10: "gfg", 20: "ide"}
print (type(d))
```

(dictionaries are used to do mappings. This is a collection of key: value pairs)

Eg- items : prices, rollno : name.

=> Type conversion in Python -

Implicit

Eg-
```
a = 10
b = 1.5
c = a + b
print (c)
d = True
e = a + d
print (e)
```

O/P - 11.5
11

Explicit

Eg-
```
s = "135"
j = 1 + int(s)
f = float(s)
print (j)
print (f)
```

O/P - 145
135.0

Eg-
```
s = "geeks"
print (list(s))
print (tuple(s))
print (set(s))
```

O/P - ['g', 'e', 'e', 'k', 's']
('g', 'e', 'e', 'k', 's') {'e', 'g', 'k', 's'}

CodeNeuron

Date .....

Eg-3
```
a = 20
print (bin(a))
print (hex(a))
print (oct(a))
```

O/P -
```
0b10100
0x14
0024
```

=> Comments in Python -

We may wish to describe the code we develop. We might wish to take notes of why a section of script fn, for instance. Formulas, procedures, and sophisticated business logics are typically explained with comments.

Single-line comment - #

Multi-line comment - # or """
# """

Doc-string - The string enclosed in triple quotes that comes immediately after the defined fn are called Python Docstring.

=> if, else and elif in Python -

There come situations in real life when we need to do some specific task and based on some specific conditions and, we decide what should we do next.

Similarly, there comes a situation in programming where a specific task is to be performed if a specific condition is true.

Codeverdict

Date .....

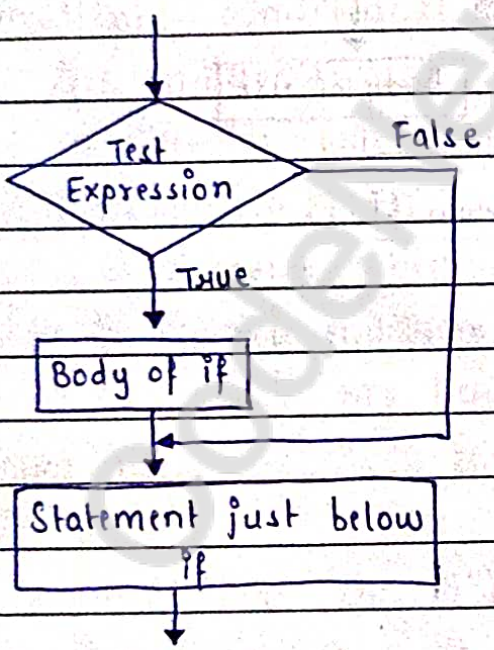
In such cases, conditional statements can be used. And the conditional statements are -

- if
- if .. else
- Nested if
- if - elif statements.

if - statement - in this if the condition is true then if part executes else the else part executes.

Syntax - if condition :
    # statements to execute if
    # condition is true

Flowchart -




Example - if 10 > 5 :
    print ("10 is greater than 5")
    print ("Program ended")

O/P - 10 is greater than 5.
    Program ended

Codeneural

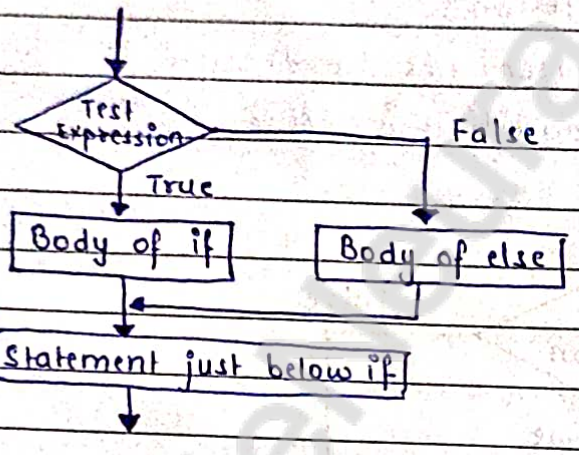
Date .....

**if-else statement** - In conditional if statement the additional block of code is merged as else statement which is performed when if condition is false.

Syntax -

```
if (condition):
    # Executes this block if
    # condition is true
else:
    # Executes this block if
    # condition is false
```

Flowchart -




Example -

```
x = 3
if x == 4:
    print ("Yes")
else:
    print ("No")
```

Example - You can also chain if...else statement with more than one condition.

```
letter = "A"
if letter == "B":
    print (" letter is B")
else:
    if letter == "C":
        print (" letter is C")
```

Date .....

```
else :
  if letter == "A":
    print ("letter is A")
  else
    print ("letter isn't A, B and C")
```

O/P - letter is A

Nested if Statement - if statement can also be checked inside other if statement. This conditional statement is called a nested if statement. This means that inner if condition will be checked only if outer if condition is true and by this, we can see multiple conditions to be satisfied.

Syntax -
```
if (condition 1):
    # Executes when condition1 is true
    if (condition 2):
        # Executes when condition 2 is true
    # if Block is end here
    # if Block is end here
```

Example -
```
num = 10
if num > 5:
    print ("Bigger than 5")
    if num <= 15:
        print ("Between 5 and 15")
```

O/P -
```
Bigger than 5
Between 5 and 15
```

if-elif Statement - shortcut of if-else chain, while using if-elif statement at the end else block is added which is performed if none of the above if-elif statement is true.

Date .....

Syntax -
```
if (condition):
    statement
elif (condition):
    statement
.
.
else:
    statement
```

Example -
```
letter = "A"
if letter == "B":
    print ("letter is B")
elif letter == "C":
    print ("letter is C")
elif letter == "A":
    print ("letter is A")
else:
    print ("letter isn't A, B or C")
```

O/P - letter is A

⇒ Arithmetic operators - They are used to perform mathem-atical operations like additional, subtraction, multip-lication, division.

There are 7 arithmetic operators in Python -

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modules
6. Exponentiation
7. Floor division

Codeleurat

Date .....

① Addition Operator - In python, + is the addition operator. It is used to add 2 values.

Example -
```
val1 = 2
val2 = 3
res = val1 + val2
print(res)
```
o/p - 5

② Subtraction Operator - used to subtract the second value from the first value.

Example -
```
val1 = 3
val2 = 2
res = val1 - val2
print(res)
```

③ Modulus Operator - % is the modulus operator used to find the remainder when first operand is divided by the second.

Example -
```
val1 = 3
val2 = 2
res = val1 % val2
print(res)
```
o/p - 1

④ Exponentiation Operator - ** is used to raise the first operand to power of second.

Example -
```
val1 = 2
val2 = 3
res = val1 ** val2
print(res)
```
o/p - 8

CodeNeural

Date .....

⑤ Floor division - // is used to find the floor of the quotient when first operand is divided by the second.

Example - val1 = 3
val2 = 2
res = val1 // val2
print (res)

o/p - 1

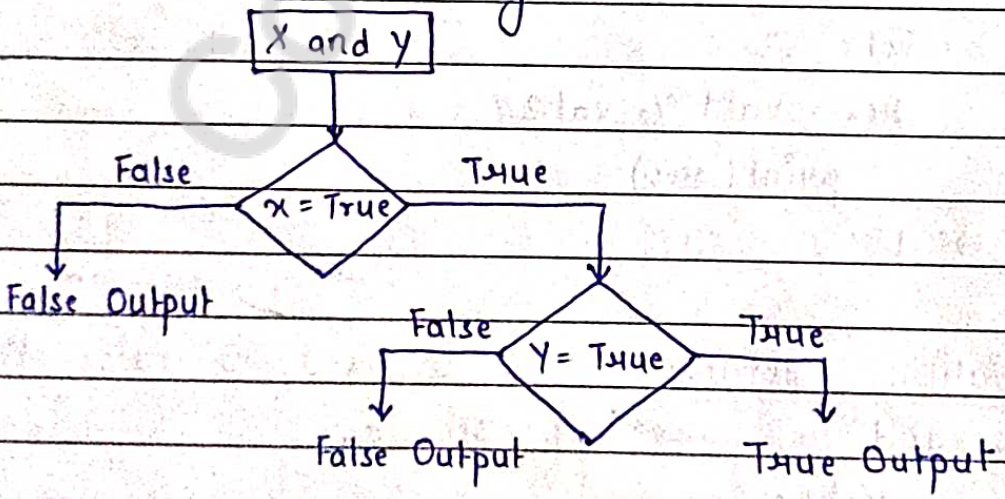
⇒ Python Logical Operators with Examples -

Operators are used to perform operations on values and variables. These are the special symbols that carry out arithmetic and logical computations. The value the operator operates on is known as Operand.

- Logical And - Logical or - Logical not

- Logical and - True if both the operands are true.

x and y.




Example - a = 10
b = 10
c = -10
if a > 0 and b > 0:

Date .....

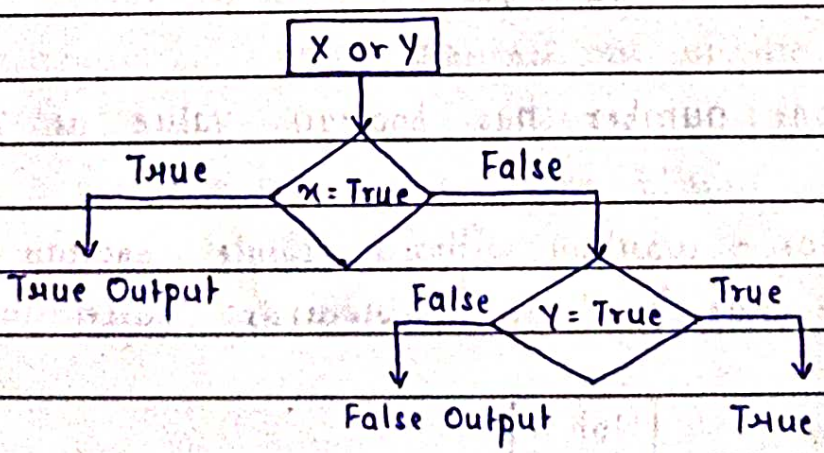
```
print ("The numbers are greater than 0")
if a>0 and b>0 and c>0:
print ("The numbers are greater than 0")
else:
    print ("Atleast one number is not greater than 0")
```

O/P - The numbers are greater than 0
Atleast one number is not greater than 0.

Example -
```
a = 10
b = 12
c = 0
if a and b and c:
    print ("All the numbers have boolean value as True")
else:
    print ("Atleast one number has boolean value as False")
```

O/P - Atleast one number has boolean value as False.

- Logical OR - It returns True if either of the operands is True.




Example -
```
a = 10
b = -10
c = 0
```

CodeNeural

Date .....

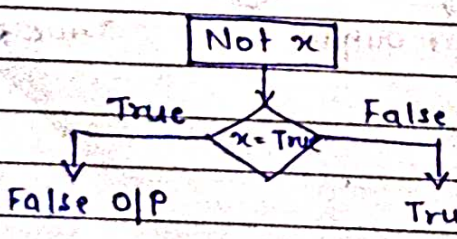
```
if a>0 or b>0:
    print ("Either of the number is greater than 0")
else:
    print ("No number is greater than 0")
if b>0 or c>0:
    print ("Either of the number is greater than 0")
else:
    print ("No number is greater than 0")
```

O/P - Either of the number is greater than 0
No number is greater than 0

Example -
```
a = 10
b = 12
c = 0
if a or b or c:
    print ("Atleast one number has boolean value as True")
else:
    print ("All the numbers have boolean value as False")
```

O/P - Atleast one number has boolean value as True.

- Logical NOT operator - works with a single boolean value. If the boolean value is True it returns False and vice-versa.




CodeNeuro

Date .....

Example - a = 10
```
if not a :
print ("Boolean value of a is True")
if not ( a%3 == 0 or a%5 == 0):
  print ("10 is not divisible by either 3 or 5")
else:
  print ("10 is divisible by either 3 or 5")
```

o/p - 10 is divisible by either 3 or 5

- Order of evaluation of logical operator -

In the case of multiple operators, Python always evaluates the expression from left to right.

Example -
```
def order(x):
  print ("Method called for value:", x)
  return True if x>0 else False
a = order
b = order
c = order
if a(-1) or b(5) or c(10):
  print ("Atleast one of the number is positive")
```

o/p - Method called for value = -1
Method called for value = 5
Atleast one of the number is positive.

for

Confidential

Date .....

## ⇒ Python Membership and Identity Operators -

Python offers two membership operators to check or validate the membership of a value. It tests for membership in a sequence, such as strings, lists or tuples.

**in operator** - The 'in' operator is used to check if a character / substring / element exists in a sequence or not. Evaluate to True if it finds the specified element in a sequence otherwise False. For eg -

```
'G' in 'GeeksforGeeks'    # Checking 'G' in string
True
'g' in 'GeeksforGeeks'    # Checking 'g' in string since pytho
is case-sensitive, return False
False
```

**Example** -

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9]
for item in list1:
    if item in list2:
        print("overlapping")
    else:
        print("not overlapping")
```

**O/P** -

```
not 100
not
not
not
not
```

Spiral

Date .....

- 'not in' operator - Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.

Example -
```
x = 24
y = 20
list = [10, 20, 30, 40, 50]
if (x not in list):
    print ("x is NOT present in given list")
else:
    print ("x is present in given list")
if (y in list):
    print ("y is present in given list")
    else:
        print ("y is NOT present in given list")
```

O/P - x is NOT present in given list.
y is present in given list

- Identity operators - They are used to compare the object if both the objects are actually of the same data type and share the same memory location.
There are different identity operators such as

'is' operator - Evaluate to True if the variables on either side of the operator point to the same object and false otherwise.

Example -
```
x = 5
y = 5
print (x is y)
id (x)
id (y)
```

Spiral

Date .....

O/P - 140704586672032
140704586672032

Here, in the given example, both the variable x and y have value 5 assigned to it and both share the same memory location, which is why return True.

'is not' operator - Evaluates to false if the variables on either side of the operator point to a different object and true otherwise.

Example -
```
x = 5
if (type(x) is not int):
    print ("true")
else:
    print ("false")
# Prints True
x = 5.6
if (type(x) is not int):
    print ("true")
else:
    print ("false")
```

O/P - False
True

⇒ Python Bitwise Operators -

Operators are used to perform operations on values and variables. These are the special symbols that carry out arithmetic and logical computations. The value the operator operates on is known as operand.

Codelearnx

Date .....

## ① Bitwise operators -

a)- **Bitwise AND (&)** - Return 1 if both the bits are 1 else 0.

Example - a = 10 = 1010 (Binary)
b = 4 = 0100 (Binary)
a & b = 1010
&
0100
= 0000
= 0 (Decimal)

b)- **Bitwise OR (|)** - Returns 1 if either of the bit is 1 else 0.

Example - a = 10 = 1010 (Binary)
b = 4 = 0100 (Binary)
a|b = 1010
|
0100
= 1110
= 14 (Decimal)

c)- **Bitwise NOT operator (~)** - Returns one's complement of the no

Example - a = 10 = 1010 (Binary)
~a = ~1010
= -(1010 + 1)
= -(1011)
= -11 (Decimal)

d)- **Bitwise (XOR) (^)** - Return 1 if one of the bits is 1 and the other is 0 else return 0.

CodeNuray

Date .....

Example - a = 10 = 1010 (Binary)
b = 4 = 0100 (Binary)
a^b = 1010
^
0100
= 1110
= 14 (Decimal)

Example - a = 10
b = 4
# Print bitwise AND operation
print ("a & b=", a & b)
# Print bitwise OR operation
print ("a|b=", a|b)
# Print bitwise NOT operation
print ("~a=", ~a)
# Print bitwise XOR operation
print ("a^b=", a^b)

o/p - a & b = 0 , a|b = 14 , ~a = -11
a^b = 14

(3) Shift Operators - These operators are used to shift the bits of a number left or right thereby multiplying or dividing the no. by two resp. They can be used when we have to multiply or divide a number by two.

a)- Bitwise Right Shift (>>) - Shifts the bits of the no. to the right and fills 0 on void left (fills 1 in the case of a negative number) as a result. Similar effect as of dividing the no. with some power of two.

Spiral

Date .....

Example - a = 10 = 0000 1010 (Binary)
a >> 1 = 0000 0101 = 5

Example - a = -10 = 1111 0110 (Binary)
a >> 1 = 1111 1011 = -5

b) - Bitwise Left Shift - (<<)
Shifts the bits of the number to the left and fills 0 on void right as a result. Similar effect as of multiplying the no. with some power of two.

Example - a = 5 = 0000 0101 (Binary)
a << 1 = 0000 1010 = 10
a << 2 = 0001 0100 = 20

Example - b = -10 = 1111 0110 (Binary)
b << 1 = 1110 1100 = -20
b << 2 = 1101 1000 = -40

③ Bitwise Operator Overloading - Operator Overloading means giving extended meaning beyond their predefined operational meaning. For example operator + is used to add two integers as well as join two string and merge two lists. It is achievable because the '+' operator is overloaded by int class and str class. You might have noticed that the same built-in operator or function shows different behavior for objects of different classes, this is called Operator Overloading.

Example -
```
class Geek():
    def __init__(self, value):
        self.value = value
    def __and__(self, obj):
        print("And operator overloaded")
```

Spiral

Date .....

```
        if isinstance (obj, Geek):
            return self.value & obj.value
        else:
            raise ValueError ("Must be a object of class Geek")
    def __or__(self, obj):
        print ("Or operator overloaded")
        if isinstance (obj, Geek):
            return self.value | obj.value
        else:
            raise ValueError ("Must be a object of class Geek")
    def __xor__(self, obj):
        print ("XOR operator overloaded")
        if isinstance (obj, Geek):
            return self.value ^ obj.value
        else:
            raise ValueError ("Must be a object of class Geek")
    def __lshift__(self, obj):
        print ("lshift operator overloaded")
        if isinstance (obj, Geek):
            return self.value << obj.value
        else:
            raise ValueError ("Must be a object of class
                                Geek")
    def __invert__(self):
        print ("Invert operator overloaded")
        return ~self.value
# Driver's code
    if __name__ == "__main__":
        a = Geek (10)
        b = Geek (12)
        print (a & b)
```

Spiral

Date .....

```
print (a|b)
print (a^b)
print (a<<b)
print (a>>b)
print (~a)
```

O/P - And operator overloaded
8
Or operator overloaded
14
Xor operator overloaded
8
lshift operator overloaded
40960
rshift operator overloaded
8
Invert operator overloaded
-11

* Loops in Python -

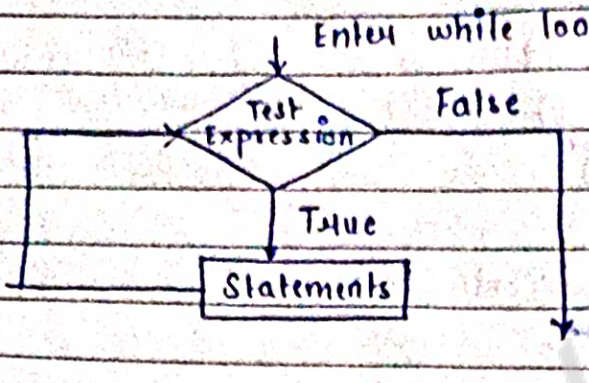
- While loop - It is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

Syntax -
```
while expression :
    statement(s)
```

Coolieyar

Date .....

## Flowchart




While loop falls under the category of indefinite iteration. Indefinite iteration means that the number of times the loop is executed isn't specified explicitly in advance.

Statements represents all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its methods of grouping statements. When a while loop is executed, expr is first evaluated in a Boolean context and if it is true, the loop body is executed. Then the expr is checked again, if it is still true then the body is executed again and this continues until the expression becomes false.

**Example-1**

```
count = 0
while (count < 3):
    count = count + 1
    print ("Hello Geek")
```

**O/P -**

```
Hello Geek
Hello Geek
Hello Geek
```

Spiral

Date .....

Example - 2

```
a = [1, 2, 3, 4]
while a:
    print (a.pop())
```

O/P -
```
4
3
2
1
```

In this, we have run a while loop over a list that will run until there is an element present in the list.

Example - 3 single statement while block

Just like the if block, if the while block consists of single statement we can declare the entire loop in a single line. If there are multiple statements in the block that make up the loop body, they can be separated by semicolons (;).

```
count = 0
while (count < 5): count += 1; print ("Hello Geek")
```

O/P -
```
Hello Geek
Hello Geek
Hello Geek
Hello Geek
Hello Geek
```

GeeksforGeeks

Date .....

## – range() in Python –

The python range() fn returns a sequence of numbers, in a given range. The most common use of it is to iterate sequences on a sequence of numbers using Python loops.

Syntax - range(start, stop, step)

Parameter - start: [optional]
stop:
step:

Return:

Example -
```
for i in range(5):
    print(i, end = " ")
print()
```

O/P - 0, 1, 2, 3, 4

## What is the use of the range() in Python ?

In simple terms, range() allows the user to generate a series of no. within a given range. Depending on how many arguments the user is passing to the fn, the user can decide where that series of numbers will begin & end as well as how big the difference will be between one no. and the next. Python range() fn takes can be initialized in 3 ways -

- range (stop) takes one argument.
- range (start, stop) takes two argument.
- range (start, stop, step) takes three argument.

Spiral

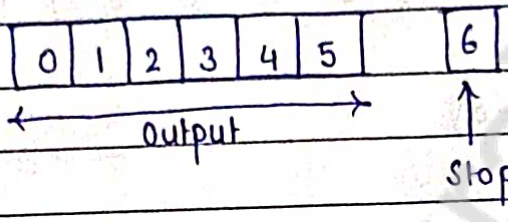
Date .....

a)- range (stop) -

When the user call range() with one argument, the user will get a series of numbers that starts at 0 and includes every whole number up to, but not including, the number that the user has provided as the stop.

Example -

Python range(6)



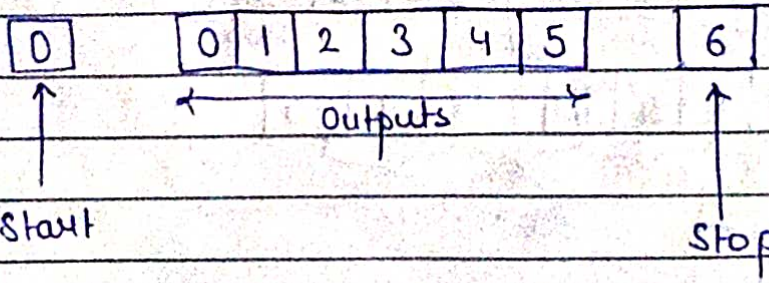

Example - for i in range(6):
print(i, end=" ")
print()

O/P - 0 1 2 3 4 5

b)- range (start, stop) -

When the user call range() with two arguments, the user gets to decide not only where the series of no. stops but also where it starts, so the user don't have to start at 0 all the time. Users can use range() to generate a series of numbers from X to Y.

Python range (0, 6)




Spiral

Date .....

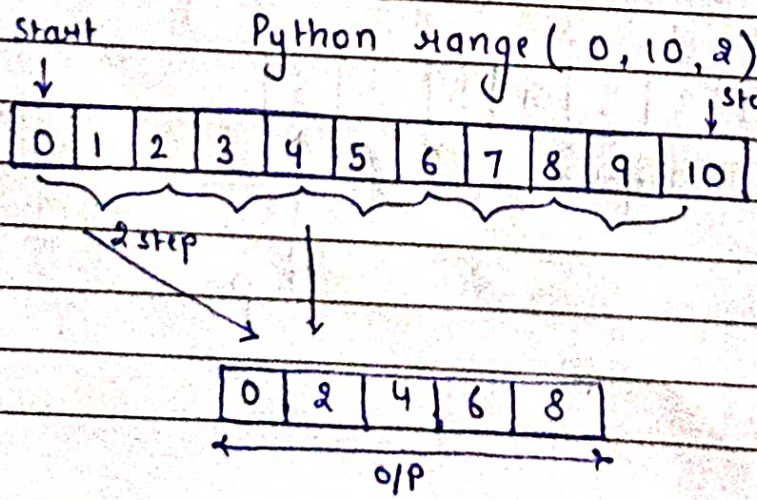
Example - Demonstration of Python range (start, stop)

```
# printing a natural
# number from 5 to 20
for i in range (5, 20):
    print (i, end = " ")
```

O/P - 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

c)- range (start, stop, step)

When the user called range() with three arguments, the user can choose not only where the series of numbers will start and stop, but also how big the difference will be between the one no. and the next. If the user doesn't provide the step, then range() will automatically behave as if the step is 1. In this example, we are printing even no. between 0 and 10, so we choose our starting point from 0 (start=0) and stop the series at 10 (stop=10). For printing an even number the difference between one number and the next number must be 2 (step=2) after providing a step we get the following output (0, 2, 4, 6, 8).




Spiral

Date .....

Example - Demonstration of Python range (start, stop, step)

```
for i in range (0,10,2):
    print(i, end = " ")
    print()
```

O/P - 0, 2, 4, 6, 8

## For Loop in Python -

It is used for sequential traversal i.e., it is used for iterating over an iterable like String, Tuple, List, Set or Dictionary.

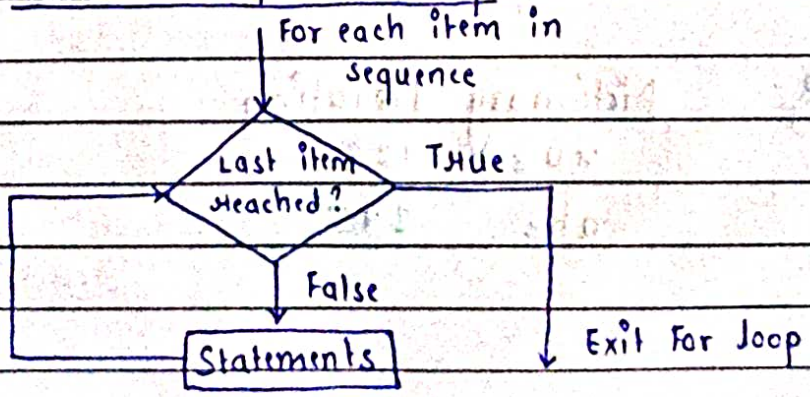
In python, there is no C style for loop, i.e., for (i=0; i<n; i++). There is a "for" loop which is similar to each loop in other languages.

NOTE - In Python, for loops only implements the collection-based iteration.

Syntax -

```
for var in iterable:
    # statements
```

### Flowchart of For loop




Spiral

Date .....

Here the iterable is a collection of objects like lists, tuples. The indented statements inside the for loop are executed once for each item in an iterable. The variable var takes the value of the next item of the iterable each time through the loop.

Example -
```
# Python program to illustrate
# iterating over a list
l = ["geeks", "for", "geeks"]
for i in l:
    print(i)
```

O/P -
```
geeks
for
geeks
```

Example - Loop in Dictionary

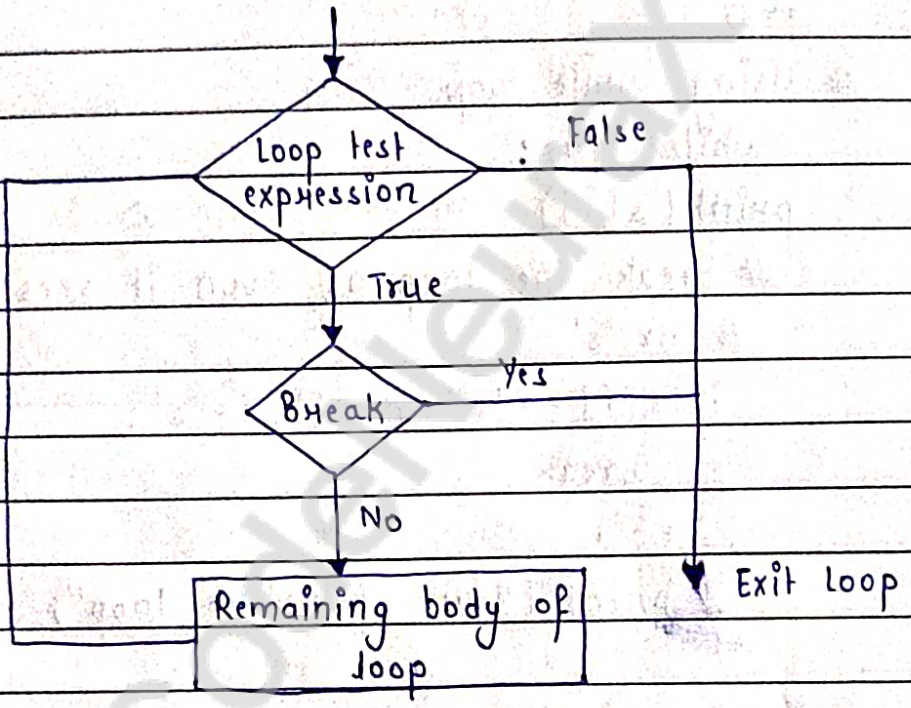
```
# iterating over dictionary
print ("Dictionary Iteration")
d = dict()
d['xyz'] = 123
d['abc'] = 345
for i in d:
    print ("%s %d" % (i, d[i]))
```

O/P -
```
Dictionary Iteration
xyz 123
abc 345
```

Spiral

Date .....

* break in python - It is used to bring the control out of the loop when some external condition is triggered. break statement is put inside the loop body (generally after if condition). It terminates the current loop, i.e., the loop in which it appears, and resumes execution at the next statement immediately after the end of that loop. If the break statement is inside a nested loop, the break will terminate the innermost loop.




Example -
```
for i in range (10):
    print(i)
    if i == 2:
        break
```

o/p - 0
1
2

Example - 2
```
s = 'geeksforgeeks'
# Using for loop
```

Spiral

Date .....

```
for letter in s:
    print(letter)
# break the loop as soon it sees 'e'
 # or 's'
 if letter == 'e' or letter == 's'
     break
print("Out of for loop")
  print()
   i=0
 # using while loop
    while True:
   print(s[i])
    # break the loop as soon it sees 'e'
     # or 's'
     if s[i] == 'e' or s[i] == 's':
         break
     i+=1
      print("Out of while loop")
```

o/p -
```
g
e
Out of for loop
g
e
Out of while loop
```

* Continue Statement in Python - It is a loop control statement that forces to execute the next iteration of the loop while skipping the rest of the code inside the loop for the current iteration only, i.e.,

Date .....

when the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped for the current iteration and the next iteration of the loop will begin.

Syntax - while True :

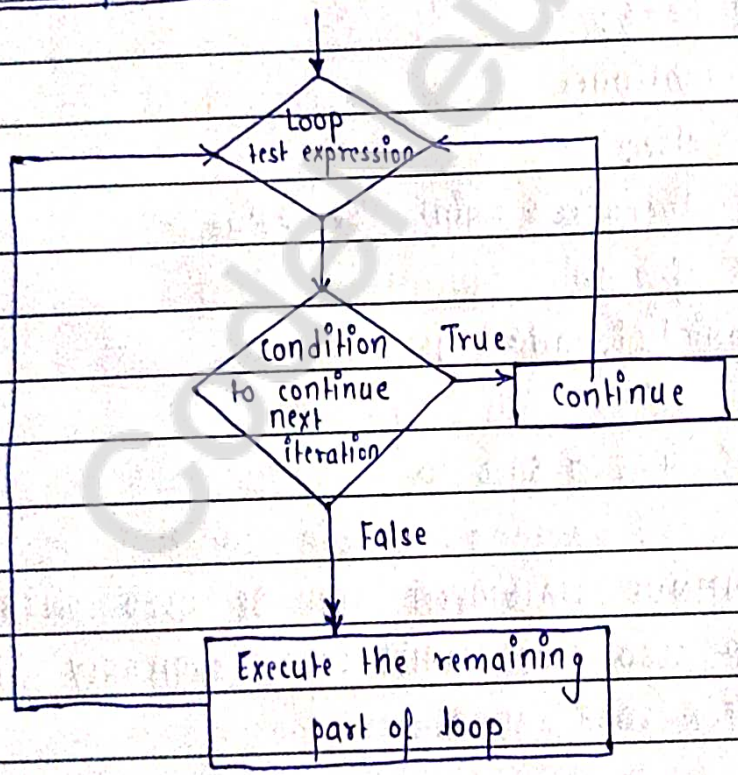
.....

if x == 10 :

continue

print(x)

Flowchart of Continue statement -




Example - for var in "Geeksforgeeks":

if var == "e":

continue

print (var)

o/p - G
K
S
f
|
0
r
g
k
s

Spiral

Date .....

Explanation - Here we are skipping the print of character 'e' using if-condition checking and continue statement.

Example - 2

Consider a situation when you need to write a program which prints the number from 1 to 10, but not 6.

```
# loop from 1 to 10
for i in range (1, 11)
    # if i is equal to 6,
    # continue to next iteration
    # without printing
    if i == 6:
        continue
    else:
    # otherwise print the value
    # of i
    print (i, end = " ")
```

o/p - 1 2 3 4 5 7 8 9 10

NOTE - THE CONTINUE STATEMENT CAN BE USED WITH ANY OTHER LOOP ALSO LIKE "WHILE LOOP" SIMILARLY AS IT IS USED WITH "FOR LOOP" ABOVE.

* Nested Loop - With for and while loops we can create nested loop. Nested loop means loops inside a loop. For example, while loop inside the for loop, for loop inside the for loop, etc.

CodeNeural

Date .....

Syntax - Outer-loop Expression :

Inner-loop Expression :
Statement inside inner-loop

Statement inside Outer-loop

Example - x = [1, 2]
y = [4, 5]
for i in x :
for j in y :
print ( i, j)

O/P - 14
15
24
25

Example - 2

```
# Multiplication Table
# Running outer loop from 2 to 3
for i in range (2, 4):
    # Printing inside the outer loop
    # Running inner loop from 1 to 10
    for j in range (1, 11):
        # Printing inside the inner loop
        print (i, "*", j, "=", i*j)
    # Printing inside the outer loop
    print ()
```

O/P - 2 * 1 = 2 ..... 2 * 10 = 20
3 * 1 = 3 ..... 3 * 10 = 30

CodeNeural

Date .....

Example -
```
# Initialize list1 and list2
# with some strings
list1 = ['I am', 'You are']
list2 = ['healthy', 'fine', 'geek']
# Store length of list2 in list2_size
list2_size = len(list2)
# Running outer for loop to
# iterate through a list1
for item in list1:
    # Printing outside inner loop
    print("start outer for loop")
    # initialize couter i with 0
    i=0
    # Running inner while loop to
    # iterate through a list2
    while (i< list2_size):
        # Printing inside inner loop
        print(item, list2[i])
        # Incrementing the value of i
        i = i+1
    # printing outside inner loop
    print("end for loop")
```

O/P -
```
start outer for loop
I am healthy
I am fine
I am geek
end for loop
start outer for loop
You are healthy
You are fine
```

Spiral

Date .....

You are geek
end for loop

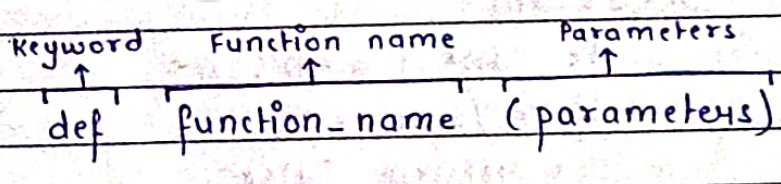
In this example, we are initializing two lists with some strings. Store the size of list2 in 'list2_size' using len() function and using it in the while loop as a counter. After that run an outer for loop to iterate over list1 and inside that loop run an inner while loop to iterate over list2 using list indexing inside that we are printing each value of list2 for every value of list1.

* Functions in Python -

Python Functions is a block of statements that return the specific task.

The idea is to put some commonly or repeatedly done tasks together and make a function so that instead of writing the same code again and again for different inputs, we can do the function calls to reuse code contained in it over and over again.

Syntax - Python functions




codeurfan

Date .....

Example - def fun():
print("Welcome to GFG")
# Driver code to call a function
fun()

O/P - Welcome to GFG.

- Defining and calling a function with parameters -

Syntax - def function_name(parameter: data_type) ->
return_type:
"""Doctring"""
# body of the function
return expression

Example - # some more functions
def is_prime(n):
if n in [2, 3]:
return True
if (n==1) or (n%2==0):
return False
x = 3
while x*x <= n:
if n%x==0:
return False
x+=2
return True
print(is_prime(78), is_prime(79))

O/P - False True

GeeksNeural

Date .....

## ⇒ Arguments of a Python Function -

Arguments are the values passed inside the paranthesis of the function. A function can be have any number of arguments separated by a comma.

Example -
```
# A simple Python Function to check
# whether x is even or odd
def evenOdd(x):
    print("even")
else:
    print("odd")
# Driver code to call the function
evenOdd(2)
evenOdd(3)
```

o/p - even
odd

## - Types of Arguments -

① Default argument - It is a parameter that assumes a default value if a value is not provided in the function call for that argument.

Example -
```
def myFun(x, y=50):
    print("x:", x)
    print("y:", y)
# Driver code (We call myFun() with
# only argument)
myFun(10)
```

o/p - x=10
y=50

Spiral

Date .....

② Keyword arguments -

The idea is to allow the caller to specify the argument name with values so that caller does not need to remember the order of parameters.

Example -
```
def student ( firstname , lastname ):
    print ( firstname , lastname )
# Keyword arguments
student ( firstname = 'Geeks' , lastname = 'for' )
student ( lastname = 'for' , firstname = 'Geeks' )
```

O/P -
```
Geeks for
Geeks for
```

③ Variable- length arguments -

We can pass a variable number of arguments to a function using special symbol. There are two special symbols -

- * args ( Non-keyword Arguments)
- ** kwargs ( keyword Arguments)

Example -
```
def myFun ( *argv ):
    for arg in argv :
        print (arg)
myFun ('Hello', 'Welcome', 'to', 'GeeksforGeeks')
```

O/P -
```
Hello
Welcome
to
GeeksforGeeks
```

GeeksforGeeks

Date .....

* Docstring - The first string after the function is called the Document string or Docstring in short. This is used to describe the functionality of the function. The use of docstring in fn is optional but it is considered a good practice.

Syntax - print(function_name.__doc__)

Example - def evenOdd(x):
    """Function to check whether x is even or odd"""
    if (x%2==0):
        print("even")
    else:
        print("odd")
# Driven code to call the function
print(evenOdd.__doc__)

O/P - Function to check if the number is even or odd.

* Return statement in Python function -
The function return statement is used to exit from a function and go back to the function caller and return the specified value or data item to the caller.

Syntax - return [expression_list]

The return statement can consist of a variable, an

Date .....

expression, or a constant which is returned to the end of the function execution. If none of the above is present with the return statement a None object is returned.

Example -
```
def square_value(num):
    """This function returns the square value
       of the entered number"""
    return num**2
print(square_value(2))
print(square_value(4))
```

O/P - 4
16

- Pass by Reference or pass by value -

One important thing to note is, in Python every variable name is a reference. When we pass a variable to a function, a new reference to the object is created.

Example -
```
def myFun(x):
    x[0] = 20
# Driver Code (Note the lst is modified after
# function call
lst = [10, 11, 12, 13, 14, 15]
myFun(lst)
print(lst)
```

O/P - [20, 10, 11, 12, 13, 14, 15]

* Strings in Python -

Strings are array of bytes representing Unicode characters.

Date .....

Example - "Geeksforgeeks" or 'Geeksforgeeks'

Python does not have a character data type, a single character is simply a string with a length of 1. Square braces can be used to access elements in string.

Example - print ("A Computer Science portal for geeks")

o/p - A Computer Science portal for geeks.

- Creating a String in Python -

String in python can be created using single, double or even triple quotes.

Example -

```
String1 = 'Welcome to the Geeks World'
print ("String with the use of single quotes:")
print (String1)
# Creating a String
# with double quotes
String1 = "I m a Geek"
print ("\n String with the use of Double quote")
print (String1)
# Creating a String with triple quotes
String1 = '''I'm a "Geek"'''
print ("\n String with the use of Triple
                                    Quotes:")
print (String1)
# Creating String with triple
# Quotes allows multiple lines
String1 = '''Geeks
For
Life'''
print ("\n Creating a multiline String:")
```

Original

Date .....

print (String 1)

O/P - String with the use of Single Quotes:
Welcome to the Geeks world
String with the use of double quotes:
I'm a Geek
String with the use of Triple Quotes:
I'm a "Geek"
Creating a multiline String:
Geeks
For
Life

## - Accessing characters in Python String -

In python, individual characters of a string can be accessed by using the method of Indexing. Indexing allows negative address references to access characters from the back of the string, eg. -1 refers to the last char, -2 refers to the second last char and so on.

While accessing an index out of the range will cause an IndexError. Only Integers are allowed to be passed as an index, float or other types that will cause a TypeError.

Example -
```
String 1 = "Geeks For Geeks"
print ("Initial String:")
print (String 1)
# Printing First character
print ("\n First character of String is:")
print (String 1[0])
```

Spiral

Date .....

```
# Printing Last Character
print ("\n Last character of String is :")
print (String1[-1])
```

O/P - Initial String :
GeeksForGeeks
First character of String is :
G
Last character of String is :
S

- Reversing a Python String -

```
# Program to reverse a string
gfg = "geeksforgeeks"
print (gfg[::-1])
```

O/P - skeegrofskeeg

- String Slicing -

To access a range of characters in the String, the method of slicing is used. Slicing in a String is done by using a Slicing operator (colon).

Example -
```
String1 = "GeeksforGeeks"
print (" Initial String :")
print (String1)
print ("\n Slicing characters for 3-12 :")
print (String1[3:12])
print ("\n Slicing characters between" +
       "3rd and 2nd last character :")
```

Code Kurax

Date .....

print(String1[3:-2])

O/P - Initial String:
GeeksForGeeks
Slicing characters from 3-12:
ksForGeek
Slicing characters between 3rd and 2nd last char:
ksForGee

- chr() function -
It returns a string from a Unicode code integer.
Eg - print(chr(97))

O/P - a

* Escape Sequencing in Python -

While printing String with single and double quote in it cause SyntaxError because string already contains Single and Double Quotes and hence cannot be printed with the use of either of these. Hence, to print such a string either Triple Quotes are used to Escape sequences are used to print such strings.

- String Concatenation using + Operator -
This operator can be used to add multiple string together. However, the arguments must be a string. Here, the + operator combines the string that is stored in the var1 and var2 and stores in another variable var3.

NOTE - STRINGS ARE IMMUTABLE, THEREFORE, WHENEVER IT IS

CodeNeurax

Date .....

CONCATENATED, IT IS ASSIGNED A NEW VARIABLE.

- index() method - It allows a user to find the index of the first occurence of an existing substring inside a given string.

Syntax - string_obj.index(substring, begp, endp)

Parameters - • substring - The string to be searched for.
• begp - (default: 0): This fn specifies the position from where search has to be started.
• endp (default: length of string): This fn specifies the position from where search has to end.

Return - Returns the 1st position of substring found.

Exception - Raise valueError if argument string is not found or index is out of range.

- rindex() method - It returns the highest index of the substring inside the string if the substring is found. Otherwise, it raises ValueError.

⇒ Case Changing of Strings -

- lower() - Convert all uppercase characters in a string into lowercase.
- upper() - Convert all lowercase characters in a string into uppercase.
- title() - Convert string to title case.

Coolgenral

Date .....

⇒ Built in methods - starts with() & endswith()

Syntax - startswith() - str.startswith(search-string, start, end)

endwith() - str.endswith(search-string, start, end)

- Python String find() method - It returns the lowest index or first occurence of the substring if it is found in a given string. If it is not found, then it returns -1.

Syntax - str-obj.find(sub, start, end)

sub → Substring that need to be searched

* List Introduction -

Python lists are just like dynamically sized arrays, declared in other language (C++, Java). In a simple language, it is a collection of things, enclosed with [] and separated by commas.

The list is a sequence data type which is used to store the collection of data. Tuples and Strings are other types of sequence data types.

Eg - Var = ["Geeks", "For", "Geeks"]
print(Var)

O/P - ['Geeks', 'For', 'Geeks']

Spiral

Date .....

- Lists need not to be homogeneous always which makes it the most powerful tool in Python.
- A single list may contain Data Types like Int, Str, as well as objects.
- Lists are mutable, and hence, they can be altered even after their creation.

* Creating a List in Python -

List can be created by just placing the sequence inside the square braces []. Unlike sets, a list doesn't need a built-in function for the creation of a list.

Example -
```
List = []
print ("Blank List:")
print ("List")
List = [10, 20, 30]
print ("\n List of numbers:")
print (List)
```

O/P -
```
Blank list:
[]
List of numbers:
[10, 20, 30]
```

- Accessing element from the list -
```
List = ["Geeks", "For", "Geeks"]
print ("Accessing a element:")
print (List[0])
print (List[2])
```

Spiral

Date .....

- Negative Indexing

- Getting the size of the list -

```
List 1 = [ ]
print ( len (List 1))
List 2 = [ 1, 2, 3]
print ( len (list 2))
```

O/P - 0
3

- Adding Elements to a list -

Using append ( ) method -

- Only 1 element can be added at a time, for the additional of multiple elements with the append() method, loops are used.
- Tuples can be added to the list with the use of append method because tuples are immutable.

Example -

```
List = [ ]
print ("Initial blank list :")
print (List)
List. append (1)
List. append (2)
List. append (4)
print ("\n List After Addition of three elements!")
print ( List)
for i in range (1, 4):
    List. append (i)
```

Spiral

Date .....

```
print("\n List after Addition of elements from 1-3:")
    print(List)
List.append((5,6))
    print("\n List after Addition of a Tuple:")
    print(List)
List2 = ['For', 'Geeks']
    List.append(List2)
    print("\n List after Addition of a List:")
    print(List)
```

O/P - Initial blank List:
[]
List after Addition of Three elements:
[1, 2, 4]
List after Addition of elements from 1-3:
[1, 2, 4, 1, 2, 3]
List after Addition of a Tuple:
[1, 2, 4, 1, 2, 3, (5, 6)]
List after Addition of a List:
[1, 2, 4, 1, 2, 3, (5, 6), ['For', 'Geeks']]

- Using insert() method -
It inserts a given element at a given index in a list using Python.

Syntax - list_name.insert(index, element)
index - at which the element has to be inserted.
element - to be inserted in the list.
Returns - Does not return any value.

Coolneural

Date .....

Example -

```
lis = ['Geeks', 'Geeks']
lis.insert(1, "For")
print(lis)
```

O/P - ['Geeks', 'For', 'Geeks']

- count() method - returns the count of how many times a given object occurs in a List.

Syntax - list_name.count(object)

Parameters -

- object - is the item whose count is to be returned.

Exception -

- TypeError - Raises TypeError if more than 1 parameter is passed in count() method.

Example -

```
list2 = ['a', 'a', 'a', 'b', 'b', 'a', 'c', 'b']
print(list2.count('b'))
```

O/P - 3

- del keyword - del is a keyword and remove(), pop() are in-built method.
- remove() method delete values or object from the list using value.
- the del and pop() delete values or objects from the list using an index.
- The del keyword deletes any variable, or list of values from a list.

codeLerax

Date .....

Syntax -

```
del list_name[index]    # To delete single valu
del list_name           # To delete whole list
```

- The remove() method removes the first matching values from the list.

Syntax - list_name.remove(value)

- The pop() method like the del keyword deletes value at a particular index. But the pop() method returns deleted value from the list.

Syntax - list_name.pop(index)

Example -

```
number = [1,2,3,2,3,4,5]
# use remove()
number.pop(3)
print(number)
# use remove()
number.pop(-1)
print(number)
number.pop(0)
print(number)
```

o/p -

```
[1,2,3,3,4,5]
[1,2,3,3,4]
[2,3,3,4]
```

- max() method used to compute the maximum of the values passed in its argument and the lexicographically largest value if strings are

Spirat

Date .....

passed as arguments.

Example - print ("Maximum of 4, 12, 43.3, 19 and 100 is :", end = " ")
print (max (4, 12, 43.3, 19, 100))

O/P - Maximum of 4, 12, 43.3, 19, 100 is = 100

- min() method is used to compute the minimum of the values passed in its arguments.

- sort() function - can be used to sort a List in ascending, descending, or user-defined order.

Syntax - List_name. sort ( reverse = True/ False, key = myFunc)

- reverse() method - is a inbuilt method in python that reverses objects of the list in place i.e., it doesn't use any extra space but it just modifies the original list.

Syntax - list_name. reverse()

- Tuples in Python -

Python Tuple is a collection of objects separated by commas. In some ways, a tuple is similar to a list in terms of indexing, nested objects, and repetition but a tuple is immutable, unlike lists which are mutable.

To create a Tuple we will use () operators.

var = ("Geeks", "for", "Geeks")
print (var)

Confidential

Date .....

- Accessing Values in Tuples -

Method - 1 Using Positive Index
Method - 2 Using Negative Index

* Sets in Python -
It is an unordered collection data type that is iterable, mutable and has no duplicate elements.
Sets are represented by {}.

Since sets are unordered, we can't access items using indexes like we do in lists.

Example - var = {"Geeks", "for", "Geeks"}
print (type(var))

o/p - <class 'set'>

- Frozen set are immutable objects that only support methods and operators that produce a result without affecting the frozen set or sets to which they are applied. It can be done with frozenset() method.

- Methods for Sets -

- Adding elements to sets - set.add() function
- Union operation on sets - union() function
- Intersection operation - intersection() or & operator
- Difference - difference() or - operator
- Clearing sets - Clear() method

Spiral

Date .....

## * Dictionary in Python -

It is a collection of keys values, used to store data values like a map, which, unlike other data types which hold only a single value as an element.

- Dictionary holds key : value pair.

Example - Dict : {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print (Dict)

### - Creating a Dictionary -

A dictionary can be created by placing a sequence of elements with curly {} braces, separated by 'comma'. Dictionary holds pairs of values, one being the key and the other corresponding pair element being its key: Value. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable.

NOTE - Dictionary keys are case sensitive, the same name but different cases of key will be treated distinctly.

Example - Dict = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print ("\n Dictionary with the use of Integer keys:")
print (Dict)
Dict = {'Name': 'Geeks', 1: [1, 2, 3, 4]}
print ("\n Dictionary with the use of Mixed keys:")
print (Dict)

Spiral

Date .....

O/P - Dictionary with the use of Integer keys:
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Dictionary with the use of Mixed keys:
{'Name': 'Geeks', 1: [1, 2, 3, 4]}

- Dictionary can also be created by the built-in function dict(). An empty dictionary can be created by just placing to curly braces{}.

- Nested Dictionary -

```
Dict = {1: 'Geeks', 2: 'For', 3: {'A': 'Welcome',
                                  'B': 'To', 'C': 'Geeks'}}
print(Dict)
```

O/P - {1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}

- Adding elements to a Dictionary -
One value can be added to a Dictionary by defining value along with the key. Eg- Dict[key] = 'Value'.

Updating an existing value in a Dictionary can be done by using the built-in update() method.
Nested key values can be also added to an existing Dictionary.

- Accessing elements to a Dictionary -
In order to access the item of a dictionary refer to its key name. Key can be used inside square braces.

Codejeet

Date .....

- There is also a method called get() method that will be help in accessing the elements from a dictionary. This method accepts key as argument and return the value.

⇒ Dictionary methods -

① clear() - Remove all the elements from the dictionary.
② copy() - Return a copy of the dictionary.
③ get() - Return the value of specified key.
④ items() - Return a list containing a tuple for each key value pair.
⑤ keys() - Returns a list containing dictionaries keys
⑥ pop() - Remove the element with specified key
⑦ popitem() - Remove the last inserted key-value pair.
⑧ update() - Updates dictionary with specified key-value pairs.
⑨ values() - Returns a list of all the values of dictionary.

* Slicing (List, Tuple And String) -

Slicing in python is a feature that enables accessing parts of the sequence. In slicing a string, we create a substring, which is essentially a string that exists within another string. We use slicing when we require a part of the string and not the complete string.

Syntax - string [start : end : step]
- start - We provide the starting index.
- end - We provide the end index.
- step - It is an optional argument that determine

Spiral

Date .....

the increment between each index for slicing.

- Slicing in List -

```
lst = [50, 70, 30, 20, 90, 10, 50]
print(lst[1:5])
```

O/P - [70, 30, 20, 90]

- Slicing in Tuple -

```
tup = (22, 3, 45, 4, 2.4, 2, 56, 890, 1)
print(tup[1:4])
```

O/P - (3, 45, 4)

* Comprehension in Python -

A python list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterate over each element in the Python list.

- Python List comprehension provides a much more short syntax for creating a new list based on the values of an existing list.

Syntax -

```
newList = [expression(element) for element in
                    oldList if condition]
```

Spiral

Date .....

## - List Comprehensions Vs For Loop -

There are various ways to iterate through a list. However, the most common approach is to use the for loop.

```
List = []
for character in 'Geeks 4 Geeks!':
    List.append(character)
print(List)
```

O/P - ['G','e','e','k','s','4','G','e','e','k','s','!']

## - Python Dictionary Comprehension -

Here we have two lists named keys and values and we are iterating over them with the help of zip() function.

```
keys = ['a','b','c','d','e']
values = [1,2,3,4,5]
myDict = {k:v for (k,v) in zip(keys,value)}
print(myDict)
```

O/P - {'a':1, 'b':2, 'c':3, 'd':4, 'e':5}

Spiral

Date .....

* Python OOPs Concepts and Class -

OOPs is a programming paradigm that uses object and classes in programming. It aims to implement real-world entities like, inheritance, polymorphism, encapsulations, etc. in the programming. The main concept of OOPs is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

Main Concepts of OOPs -

- Class
- Objects
- Polymorphism
- Encapsulation
- Inheritance
- Data Abstraction

- Class - A class is a collection of objects. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.

- Classes are created by keyword class.
- Attributes are the variables that belongs to a class.
- Attributes are always public and can be accessed using the dot (.) operator. Eg - Myclass.Myattribute

Syntax - class ClassName :
    # Statement-1
    :
    # Statement-N

CodeNeura

Date .....

- Objects - Object is an entity that has a state and behavior associated with it. It may be any real-world object like a mouse, keyboard, chair, table, etc. Integers, strings, floating-point numbers, even arrays, and dictionaries, are all objects.

An object consists of -

- State - It is represented by the attribute of an object. It also reflects the properties of an object.
- Behavior - It is represented by the method of an object. It also reflects the response of an object to other objects.
- Identity - It gives a unique name to an object & enables one object to interact with other objects.

```
obj = Dog()
```

Before diving deep into objects and classes let us understand some basic keywords that will we used while working with objects and classes.

① The self

- Class method must have an extra first parameter in the method definition. We do not give a value for this parameter when we call the method, Python provides it
- If we have a method that takes no arguments, then we still have to have one arguments.

Spiral

Date .....

- This is similar to this pointer in C++. When we call a method of this object as myobject.method(arg1, arg2), this is automatically converted by Python into MyClass.method(myobject, arg1, arg2).

② The __init__ method -

It is similar to constructions in C++ & Java. It is run as soon as an object of a class is instantiated. The method is useful to do any initialization you want to do with your objects.

- Encapsulation in Python -

It is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit.
In this, the variables of a class is hidden from other classes, and can be accessed only through the methods of their current class.

```
class Base:                          # Base class
    def __init__(self):
        self._a = 2                  # Protected member
class Derived(Base):                 # Derived class
    def __init__(self):
        # Calling constructor of Base class
        Base.__init__(self)
        print("Calling protected member of base class:",
              self._a)
        # Modify the protected variable:
        self._a = 3
```

Spiral

Date .....

```
print("Calling modified protected member outside
                class:", self._a)
obj1 = Derived()
obj2 = Base()
print("Accessing protected member of obj1:",
                obj1._a)
print("Accessing protected member of obj2:",
                obj2._a)
```

## - Class Instance Attribute -

Class attributes belong to the class itself they will be shared by all the instances. Such attributes are defined in the class body parts usually at the top, for legibility.

```
class Sampleclass:
    count = 0
    def increase(self):
        Sampleclass.count += 1
s1 = Sampleclass()
s1.increase()
print(s1.count)
s2 = Sampleclass()
s2.increase()
print(s2.count)
print(Sampleclass.count)
```

O/P - 1
2
2

Spiral

Date .....

Unlike class attributes, instance attributes are not shared by objects. Every object has its own copy of the instance attribute.

To list the attributes of an instance/ object, we have two functions -

① vars() - This function is displays the attribute of an instance in the form of dictionary.

② dir() - This function displays more attributes than vars functions, as it is not limited to instance. It displays the class attributes as well. It also displays the attributes of its ancestor classes.

* Class Member Access -

Attribute of a class are function objects that defines corresponding methods of its instance. They are used to implement access controls of the classes.

Built-in functions -

① getattr() - used to access the attribute of object.
② hasattr() - used to check if an attribute exist or not.
③ setattr() - used to set an attribute.
④ delattr() - used to delete an attribute.

GoodNotes

Date .....

* Class method vs static method -

Class method is a method that is bound to the class and not the object of the class.

They have the access to the state of the class as it takes a class parameter that points to the class and not the object instance.

It can modify a class state that would apply across all the instances of the class. For eg, it can modify a class variable that will be applicable to all the instances.

Static method doesn't receive an implicit first argument.

Syntax - class C(object):

@staticmethod

def fun(arg1, arg2, ...):

...

returns : a static method for function fun.

A static method is also a method that is bound to the class and not the object of the class.

A static method can't access or modify the class state.

It is present in a class because it makes sense for the method to be present in class.

- Class method vs static method -

1. A class method takes cls as the first parameter while a static method needs no specific parameters.
2. A class method can access or modify the class state while a static method can't access or modify it.
3. In general, static methods know nothing about the class state. They are utility-type methods that take some

Date .....

parameters and work upon those parameters. On the other hand class methods must have class as a parameter.

* Inheritance -

It is the capability of one class to derive or inherit the properties from another class.

Advantages -

- It represents real-world relationship well.
- It provides reusuability of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
- It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.

```
class Person(object):
    # Constructor
    def __init__(self, name):
        self.name = name
    # To get name
    def getName(self):
        return self.name
    # To check if this person is an employee
    def isEmployee(self):
        return False
# Inherited or Subclass
class Employee(Person):
    # Here we return true
    def isEmployee(self):
        return True
```

Spiral

Date .....

```
# Driver code
emp = Person ("Geek 1")
print (emp. getName(), emp. isEmployee())
    emp = Employee ("Geek 2")
print (emp. getName(), emp. isEmployee())
```

O/P - ('Geek 1', False)
('Geek 2', True)

- What is object class ?
Like Java object class, in python, object is a root of all classes.

Example - class subclass_name (superclass_name):

```
class Person (object):
    def __init__ (self, name, idnumber):
        self.name = name
        self. idnumber = idnumber
    def display (self):
        print (self. name)
        print ( self. idnumber)
    # child class
        class Employee (Person):
    def __init__ (self, name, idnumber, salary, post):
        self. salary = salary
            self. post = post
    Person. __init__ (self, name, idnumber)
        a = Employee ('Rahul', 8853, 200000, "Intern")
        a. display ()
```

Spiral

Date .....

## - Different forms of Inheritance -

1. Single Inheritance - When a child class inherits from only one parent class, it is called single inheritance.

2. Multiple Inheritance - When a child class inherits from multiple parent classes, it is called multiple inheritance.

```
class Base1(object):
    def __init__(self):
        self.str1 = "Geek1"
        print("Base1")
class Base2(object):
    def __init__(self):
        self.str2 = "Geek2"
        print("Base 2")
class Derived(Base1, Base2):
    def __init__(self):
# Calling constructor of Base1 and Base2 classes
        Base1.__init__(self)
        Base2.__init__(self)
        print("Derived")
    def printStrs(self):
        print(self.str1, self.str2)
ob = Derived()
ob.printStrs()
```

O/P - Base1
Base2
Derived

| |
|---|
| Base1 |
| Base2 |
| Derived |
| Geek1 Geek2 |

Spiral

Date .....

③ Multi-level Inheritance -
When we have a child and grandchild relationship.

④ Hierarchical Inheritance -
More than one derived classes are created from a single base.

⑤ Hybrid Inheritance - This form combines more than one form of inheritance. Basically, it is a blend of more than one type of inheritance.

Private members of parent class -
We don't always want the instance variables of the parent class to be inherited by the child class i.e., we can make some of the instance variables of the parent class private, which won't be available to the child class.

* Polymorphism -
It means having many forms. In programming, polymorphism means the same function name (but different signatures) being used for different types.

Example - Inbuilt polymorphism function -

```
print(len("geeks"))
print(len([10, 20, 30]))
```

O/P - 5
3

Spiral

Date .....

Example of user-defined polymorphism function -

```
def add (x, y, z = 0):
    return x+y+z
print (add (2,3))
print (add (2,3,4))
```

O/P - 5
9

- Polymorphism with class method -

```
class India():
    def capital (self):
print("New Delhi is a capital of India")
    def language (self):
print("Hindi is the most widely spoken langu-
                    -age in India")
    def type(self):
print ("India is a developing country")

class USA():
    def capital (self):
print ("Washington, D.C. is the capital of USA")
    def type(self):
        print ("USA is a developed country")
obj_ind = India()
obj_usa = USA()
    for country in (obj_ind, obj_usa):
        country. capital()
        country. language()
        country. type()
```

Date .....

- **Polymorphism with Inheritance -**

Polymorphism lets us define methods in the child class that have the same name as the methods in the parent class. In inheritance, the child class inherits the methods from the parent class. However, it is possible to modify a method in a child class that it has inherited from the parent class.

This is particularly useful in cases where the method inherited from the parent class doesn't quite fit the child class. In such cases, we re-implement the method in the child class. This process of re-implementing a method in the child class is known as Method Overriding.

```
class Bird:
    def intro(self):
print("There are many types of birds")
    def flight(self):
print("Most of the bird can fly but some
                    cannot")
class sparrow(Bird):
    def flight(self):
        print("Sparrows can fly")
class ostrich(Bird):
    def flight(self):
        print("Ostriches cannot fly")
obj_bird = Bird()
obj_spr = sparrow()
obj_ost = ostrich()
obj_bird.intro()
obj_bird.flight()
```

Spiral

Date .....

```
obj-spy.intro()
obj-spy.flight()
obj-ost.intro()
obj-ost.flight()
```

## * Abstract Class -

An abstract class can be considered as a blueprint for other classes. It allows you to create a set of methods that must be created within any child classes built from the abstract class. A class which contains one or more abstract methods is called an abstract class. An abstract method is a method that has a declaration but does not have an implementation. While we are ~~declass~~ designing large functional units we use an abstract class.

- By defining an abstract base class, you can define a common Application Program Interface (API) for a set of subclasses.

Example -

```
class Polygon(ABC):
    @abstractmethod
    def noofsides(self):
        pass
class Triangle(Polygon):
    # overriding abstract method
    def noofsides(self):
        print("I have 3 sides")
```

Spiral

Date .....

```
class Pentagon(Polygon):
    # overriding abstract method
    def noofsides(self):
        print("I have 5 sides")
class Hexagon(Polygon):
    # overriding abstract method
    def noofsides(self):
        print("I have 6 sides")
# Driver code
R = Triangle()
R.noofsides()
K = Quadrilateral()
K.noofsides()
R = Pentagon()
R.noofsides()
K = Hexagon()
K.noofsides()
```

O/P - I have 3 sides
I have 4 sides
I have 5 sides
I have 6 sides

CodeNCrypt